



การพัฒนาระบบต้นแบบของสภาพแวดล้อมอัจฉริยะ สำหรับผู้ใช้อีกรถเข็น

โดย

นายวรพล แสงสระศรี

นายศุภวิชญ์ อัสวลาทยอง

โครงการนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิศวกรรมศาสตรบัณฑิต
สาขาวิศวกรรมไฟฟ้าและคอมพิวเตอร์
คณะวิศวกรรมศาสตร์ มหาวิทยาลัยธรรมศาสตร์
ปีการศึกษา 2568
ลิขสิทธิ์ของมหาวิทยาลัยธรรมศาสตร์

การพัฒนาระบบต้นแบบของสภาพแวดล้อมอัจฉริยะ สำหรับผู้ใช้งานรถเข็น

โดย

นายวรพล แสงสระศรี

นายศุภวิชญ์ อัครลาภทอง

โครงการนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิศวกรรมศาสตรบัณฑิต
สาขาวิศวกรรมไฟฟ้าและคอมพิวเตอร์
คณะวิศวกรรมศาสตร์ มหาวิทยาลัยธรรมศาสตร์
ปีการศึกษา 2568
ลิขสิทธิ์ของมหาวิทยาลัยธรรมศาสตร์

Prototyping Development of Smart Environment for Wheelchair User

BY

Mr. Worapon Saengsrasi

Mr. Supawit Ausawalaithong

A PROJECT SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF BACHELOR OF ENGINEERING
IN ELECTRICAL AND COMPUTER ENGINEERING
FACULTY OF ENGINEERING
THAMMASAT UNIVERSITY
ACADEMIC YEAR 2025
COPYRIGHT OF THAMMASAT UNIVERSITY

มหาวิทยาลัยธรรมศาสตร์

คณะวิศวกรรมศาสตร์

โครงการ

ของ

นายวรพล แสงสระศรี

นายศุภวิชญ์ อัครลาภทอง

เรื่อง

การพัฒนาระบบต้นแบบของสภาพแวดล้อมอัจฉริยะสำหรับผู้ใช้อัจฉริยะ

ได้รับการตรวจสอบและอนุมัติ ให้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิศวกรรมศาสตรบัณฑิต

เมื่อวันที่ 8 มิถุนายน พ.ศ. 2569

อาจารย์ที่ปรึกษาโครงการ

(ผู้ช่วยศาสตราจารย์ ดร. ศุภชัย วรพจน์พิศุทธิ์)

หัวหน้าภาควิชาวิศวกรรมไฟฟ้าและคอมพิวเตอร์

(ผู้ช่วยศาสตราจารย์ ดร. ศุภชัย วรพจน์พิศุทธิ์)

หัวข้อโครงการ	การพัฒนาระบบต้นแบบของสภาพแวดล้อมอัจฉริยะสำหรับผู้ใช้อัจฉริยะ
ชื่อผู้เขียน	นายวรพล แสงสระศรี นายศุภวิชญ์ อัสวลาทยอง
ชื่อปริญญา	วิศวกรรมศาสตรบัณฑิต
สาขาวิชา/คณะ/มหาวิทยาลัย	วิศวกรรมไฟฟ้าและคอมพิวเตอร์ คณะวิศวกรรมศาสตร์ มหาวิทยาลัยธรรมศาสตร์
อาจารย์ที่ปรึกษาโครงการ	ผู้ช่วยศาสตราจารย์ ดร. ศุภชัย วรพจน์พิศุทธิ์
ปีการศึกษา	2568

บทคัดย่อ

โครงการนี้เสนอแพลตฟอร์ม WheelSense ระบบต้นแบบสภาพแวดล้อมอัจฉริยะสำหรับผู้ใช้อัจฉริยะในสถานดูแลผู้สูงอายุ โดยบูรณาการการรับรู้การเคลื่อนไหว การระบุตำแหน่งภายในอาคาร การบริหารงานเชิงปฏิบัติการ และการสนับสนุนการตัดสินใจของผู้ดูแล เข้าเป็นสายงานข้อมูลครบวงจร ภายใต้โครงสร้างพื้นฐานที่ติดตั้งและดูแลรักษาได้จริง เพื่อแก้ปัญหาความปลอดภัย การสื่อสารข้ามกะ และการบันทึกข้อมูลแบบกระจายในบ้านพักคนชราของประเทศไทย

สถาปัตยกรรมของระบบประกอบด้วยสี่ชั้นที่ทำงานประสานกัน ได้แก่ ชั้นอุปกรณ์ภาคสนามสำหรับตรวจจับการเคลื่อนที่และวัดสัญญาณชีพพื้นฐาน ชั้นโหนดภายในอาคารสำหรับสแกนสัญญาณวิทยุ (BLE beacon) และเสริมภาพบริบท ชั้นเซิร์ฟเวอร์ภายในสถานดูแลสำหรับรับเข้า จัดเก็บ และให้บริการข้อมูลผ่าน API ที่มีการยืนยันตัวตน และชั้นส่วนติดต่อผู้ใช้ ประกอบด้วยแดชบอร์ดเว็บตามบทบาทและแอปพลิเคชันบนสมาร์ตโฟน ระบบระบุตำแหน่งใช้แนวทางลายนิ้วมือสัญญาณวิทยุ (RSSI fingerprinting) ร่วมกับอัลกอริทึมเพื่อนบ้านใกล้สุด (K-Nearest Neighbors) ในระดับโซนและห้อง ส่วนผู้ช่วยปัญญาประดิษฐ์ (AI) แบบประมวลผลภายในเครื่อง (on-device) ถูกบูรณาการผ่าน Model Context Protocol (MCP) ภายใต้กระบวนการเสนอ-ยืนยัน-ดำเนินการ เพื่อให้การกระทำที่เปลี่ยนสถานะของระบบต้องได้รับการยืนยันก่อนเสมอ และเพื่อรักษาความเป็นส่วนตัวของข้อมูลผู้พักอาศัยภายในสถานดูแล

การประเมินระบบต้นแบบดำเนินการที่มหาวิทยาลัยธรรมศาสตร์ครอบคลุมสี่มิติ ได้แก่ ความทนทานของการจำแนกตำแหน่งจากชุดข้อมูลภาคสนาม; ความต่อเนื่องและเวลาแฝงของสายงานข้อมูลครบวงจร ความสอดคล้องในการเลือกเครื่องมือและเวลาแฝงของผู้ช่วย AI จากชุดสถานการณ์

ทดสอบ และประสบการณ์ผู้ใช้ตามบทบาทจากแบบสอบถามออนไลน์ นอกจากนี้ โครงการได้นำเสนอระบบต้นแบบต่อผู้บริหารและผู้ดูแลของบ้านพักคนชราवासนะเวศม์ เพื่อเก็บข้อเสนอแนะเชิงการใช้งานจริง โดยไม่มีการติดตั้งระบบ ณ สถานที่ดังกล่าว

ผลการพัฒนาสะท้อนความเป็นไปได้ของการบูรณาการหลายชั้นเทคโนโลยีเข้าเป็นสายงานเดียว ภายใต้ข้อจำกัดด้านต้นทุน ความซับซ้อนของการติดตั้ง และความเป็นส่วนตัวของข้อมูลประเด็นที่ต้องพัฒนาต่อยอด ได้แก่ การสอบเทียบเวลาระหว่างองค์ประกอบย่อย การขยายชุดกรณีทดสอบให้สะท้อนสภาพงานจริงมากยิ่งขึ้น และการปรับปรุงประสบการณ์ใช้งานให้เหมาะสมกับผู้ดูแลในบริบทที่หลากหลาย ซึ่งเป็นแนวทางสำคัญสำหรับการนำระบบไปประยุกต์ใช้ในที่พักอาศัย โรงพยาบาล และสถานดูแลผู้สูงอายุของประเทศไทยในระยะถัดไป

คำสำคัญ: สภาพแวดล้อมอัจฉริยะ, ผู้ใช้เก้าอี้รถเข็น, บลูทูธพลังงานต่ำ, ระบบระบุตำแหน่งในอาคาร, อินเทอร์เน็ตของสรรพสิ่ง

Title	Prototyping Development of Smart Environment for Wheelchair User
Author	Mr. Worapon Saengsrasi Mr. Supawit Ausawalaithong
Degree	Bachelor of Engineering
Major Field/Faculty/University	Electrical and Computer Engineering Faculty of Engineering Thammasat University
Advisor	Asst. Prof. Dr. Supachai Worapotphisut
Academic Year	2025

ABSTRACT

This project presents WheelSense, a prototype smart environment for wheelchair users in elderly care facilities. The system integrates motion sensing, indoor positioning, operational management, and caregiver decision support into an end-to-end data pipeline under deployable infrastructure. It aims to address safety limitations, cross-shift communication gaps, and paper-based recording practices prevalent in Thai nursing homes.

The architecture comprises four coordinated layers: (1) edge devices for motion detection and vital sign sensing; (2) indoor nodes for BLE beacon scanning and contextual enhancement; (3) a care-facility server for data ingestion, storage, and authenticated APIs; and (4) user interfaces including role-based web dashboards and mobile applications. Indoor positioning employs RSSI fingerprinting with K-Nearest Neighbors (KNN) for zone and room-level inference. An on-device AI assistant is integrated via the Model Context Protocol (MCP) under a propose-confirm-execute workflow, ensuring state-changing actions require prior confirmation while preserving resident data privacy.

System evaluation at Thammasat University covers four dimensions: positioning robustness from field data; end-to-end pipeline continuity and latency; AI tool-selection consistency and latency from template-based test scenarios; and

role-based user experience from online questionnaires. The prototype was also presented to administrators and caregivers at Vasanthawet Elderly Home to collect practical feedback without on-site installation.

Development results demonstrate the feasibility of integrating multi-layer technologies into a unified pipeline under cost, installation complexity, and data privacy constraints. Future work includes timestamp alignment across subsystems, expanding test cases to reflect real-world conditions, and improving user experience for caregivers in diverse contexts. These approaches are critical for deploying the system in residences, hospitals, and elderly care facilities in Thailand.

keyword: Smart environment, wheelchair user, Bluetooth Low Energy, indoor positioning, Internet of Things

กิตติกรรมประกาศ

คณะผู้จัดทำขอขอบพระคุณ ผู้ช่วยศาสตราจารย์ ดร. ศุภชัย วรพจน์พิศุทธิ์ อาจารย์ที่ปรึกษาโครงการ ที่ได้ให้คำแนะนำ ข้อเสนอแนะ และคำปรึกษาทางวิชาการอย่างต่อเนื่องตลอดการดำเนินงาน ทั้งในด้านการออกแบบระบบ การเลือกใช้เทคโนโลยี และการแก้ไขปัญหาทางเทคนิค จนโครงการสามารถพัฒนาได้อย่างเป็นระบบ

คณะผู้จัดทำขอขอบพระคุณ รองศาสตราจารย์ ดร. สายรัก สอาดไพร คณาจารย์ และเพื่อนร่วมงานจากคณะสหเวชศาสตร์ มหาวิทยาลัยธรรมศาสตร์ ที่ให้ความอนุเคราะห์ด้านข้อมูล คำแนะนำ และมุมมองการใช้งานจริงของผู้ใช้เก่าอีรธเซ็น ซึ่งเป็นประโยชน์อย่างยิ่งต่อการปรับปรุงระบบต้นแบบให้เหมาะสมและปลอดภัยมากยิ่งขึ้น

คณะผู้จัดทำขอขอบคุณทุกท่านที่ให้การสนับสนุนตลอดระยะเวลาการดำเนินโครงการ และหวังเป็นอย่างยิ่งว่าโครงการนี้จะเป็นส่วนหนึ่งในการพัฒนาเทคโนโลยีเพื่อยกระดับคุณภาพชีวิตของผู้ใช้เก่าอีรธเซ็นและผู้พิการในอนาคต

สารบัญ

บทคัดย่อภาษาไทย	หน้า (1)
บทคัดย่อภาษาอังกฤษ	(3)
กิตติกรรมประกาศ	(5)
สารบัญตาราง	(8)
สารบัญรูป	(11)
สัญลักษณ์และคำย่อ	16
บทที่ 1 บทนำ	17
1.1 ที่มาและความสำคัญ	17
1.2 วัตถุประสงค์	19
1.3 ขอบเขตงานวิจัย	19
1.4 ประโยชน์ที่คาดว่าจะได้รับ	20
1.5 แผนการดำเนินงาน	21
บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง	22
2.1 แนวคิดระบบดูแลผู้สูงอายุและผู้ใช้รถเข็น	22
2.2 เทคโนโลยี IoT และ Sensor สำหรับระบบดูแล	23
2.3 อุปกรณ์ฮาร์ดแวร์หลักที่เกี่ยวข้อง	24
2.4 เครื่องมือซอฟต์แวร์และไลบรารีที่ใช้พัฒนา	27
2.5 การระบุตำแหน่งภายในอาคารด้วย BLE RSSI	29
2.6 การประมวลผลข้อมูล IMU และการเคลื่อนไหว	31
2.7 Machine Learning สำหรับจำแนกพฤติกรรมหรือความเสี่ยง	33
2.8 Retrieval-Augmented Generation และ Retrieval Logic	34
2.9 Neuro-symbolic AI และ AI Pipeline แบบ Rule + Model	35
2.10 งานวิจัยที่เกี่ยวข้อง	36
2.11 ช่องว่างของงานวิจัยเดิม	37
บทที่ 3 วิธีการออกแบบและพัฒนาระบบ	39
3.1 ภาพรวมของระบบ	39
3.2 การเชื่อมต่ออุปกรณ์ภาคสนามและ Mobile Gateway	40
3.3 M5StickC Plus2 BLE Wheelchair Sensor	43
3.4 Polar Verity Sense และ Flutter Mobile Gateway	46
3.5 Indoor Localization	48

3.9	การทดสอบโครงงาน	61
3.10	แนวทางการประเมินผลระบบ	63
บทที่ 4	ผลการทดลองและการประเมินผล	65
4.1	ภาพรวมการทดลอง	65
4.2	การทดสอบ Sensor และอุปกรณ์ภาคสนาม	66
4.3	การทดสอบ Indoor Localization ด้วย KNN	70
4.4	การทดสอบ Mobile Gateway และ Server	72
4.5	การประเมิน Dashboard/HMI และ SUS Score	77
4.6	การทดสอบ AI: Inter-Rater Reliability	86
4.7	การทดสอบ AI: Text-Distance / Cosine Similarity	87
4.8	การทดสอบ AI: Latency Analysis	87
บทที่ 5	สรุปผลการดำเนินงาน อุปสรรค และข้อเสนอแนะ	89
5.1	สรุปผลการดำเนินงาน	89
5.2	อุปสรรคและข้อจำกัดของระบบ	91
5.3	ข้อเสนอแนะ	93
	รายการอ้างอิง	96
	ภาคผนวก	100
	รายละเอียดบริบทโครงงาน	101
ก.1	ปัญหาทางวิศวกรรมที่ซับซ้อน (IEA Complex Engineering Problems)	101
ก.2	องค์ความรู้และเครื่องมืออุปกรณ์ที่ใช้	104
ก.3	มาตรฐาน กฎหมาย และข้อบังคับที่เกี่ยวข้อง	106
ก.4	ผู้มีส่วนได้ส่วนเสีย ผลกระทบ และความยั่งยืน	107
ก.5	การบริหารงานวิศวกรรมและโครงการ	109
	ตารางข้อมูลการทดลอง Sensor	112
	Raw data และ Summary ของ KNN	114
	ตัวอย่างชุดคำสั่ง AI Test	116
	ตัวอย่าง Schema, API และ MQTT Payload	120
	ภาพหน้าจอ Dashboard และ Mobile Gateway	123
	Flow Diagram และ Firmware Loop เพิ่มเติม	126
	แบบสอบถามประเมินความพึงพอใจ Google Form	130
	ภาพประกอบการสำรวจพื้นที่และการติดตั้งต้นแบบ	132
	Comparative Performance Evaluation of BLE-Based Indoor Motion Tracking	134

สารบัญตาราง

	หน้า
1.1	แผนการดำเนินงานของโครงการ 21
2.1	บทบาทของเทคโนโลยี IoT และ sensor ในระบบดูแล 25
2.2	สเปคสำคัญของอุปกรณ์ฮาร์ดแวร์หลัก 26
2.3	เครื่องมือซอฟต์แวร์และไลบรารีหลักที่ใช้ในระบบต้นแบบ 28
2.4	ตัวอย่าง feature จาก IMU ที่ใช้กับการวิเคราะห์การเคลื่อนไหว 32
2.5	การแบ่งบทบาทระหว่าง neural และ symbolic component 35
2.6	การจัดกลุ่มงานวิจัยที่เกี่ยวข้องกับระบบต้นแบบ 37
3.1	หน้าที่หลักของเฟิร์มแวร์ M5StickC Plus2 ในระบบปัจจุบัน 44
3.2	กลุ่มข้อมูลที่ mobile gateway รวมก่อนส่งเข้าสู่ระบบ 48
3.3	บริการหลักของ platform services ในระบบ WheelSense 50
3.4	ตัวอย่างกลุ่มข้อมูลที่จัดเก็บใน PostgreSQL และ JSONB 52
3.5	บทบาทผู้ใช้ หน้าจอหลัก และหน้าที่ที่รองรับบน dashboard 54
3.6	กลยุทธ์การทดสอบโครงการ 62
3.7	แนวทางการประเมินผลที่ใช้ต่อเนื่องไปยังบทผลการทดลอง 64
4.1	ภาพรวมขอบเขตการทดลองของระบบ WheelSense 66
4.2	ผลสรุปการทดสอบอุปกรณ์ภาคสนาม 67
4.3	ผลการตรวจจับสถานะการเคลื่อนไหวจากหน้าต่างข้อมูล IMU 68
4.4	ผลการทดสอบระยะทางและความเร็ว 69
4.5	ผลการตรวจ feature ความเร่งจาก IMU 70
4.6	ผลการทดสอบ RSSI/KNN จากชุดข้อมูลที่มีหลักฐานประกอบ 72
4.7	ผลเปรียบเทียบ KNN กับ max-RSSI baseline 72
4.8	ผลการทดสอบ mobile gateway และ latency ของ server path 74

4.9	ผลการทดสอบ latency ของ server path	76
4.10	ผลการทดสอบ latency ของ query ฐานข้อมูล	76
4.11	ผลสรุป usability ของ dashboard/HMI จากหลักฐานปัจจุบัน	77
4.12	ผลการทำภารกิจบน dashboard และ mobile gateway	78
4.13	ผลการทดสอบ AI Inter-Rater Reliability ตามระดับความกำกวม	86
4.14	ผลการตรวจสอบ execution gate ของ AI/MCP pipeline	86
4.15	ผลการทดสอบ Text-Distance / Cosine Similarity ของคำตอบ AI	87
4.16	ผลการทดสอบ latency ของ AI/MCP pipeline (n=30)	88
5.1	สรุปผลการดำเนินงานตามส่วนของระบบ	90
ข.1	ข้อมูลสรุปของ Wheelchair Sensor และการส่ง telemetry	112
ข.2	ข้อมูลหน้าตาการเคลื่อนไหวจาก IMU	113
ข.3	ข้อมูลสรุปของ Polar Verity Sense และ Mobile Gateway	113
ค.1	จำนวนข้อมูลที่ใช้ในชุดทดลอง RSSI/KNN	114
ค.2	ผลเปรียบเทียบ KNN กับ max-RSSI baseline	114
ค.3	ผลเปรียบเทียบ KNN และ XGBoost ในชุดทดลองเดียวกัน	114
ค.4	Confusion matrix ของ KNN ใน Case 1	115
ค.5	Confusion matrix ของ KNN ใน Case 2	115
ค.6	Confusion matrix รายตำแหน่งของ KNN ใน Case 3	115
ง.1	ผลสรุปชุดทดสอบ AI ตามระดับความกำกวม	116
ง.2	ผลการผ่าน execution gate ของชุดทดสอบ AI	116
ง.3	ตัวอย่างคำสั่งกลุ่ม No-ambiguous	117
ง.4	ตัวอย่างคำสั่งกลุ่ม Low-ambiguous	118
ง.5	ตัวอย่างคำสั่งกลุ่ม Medium-ambiguous	119
ง.6	ตัวอย่างคำสั่งกลุ่ม High-ambiguous	119
จ.1	ตัวอย่าง MQTT topic ที่ใช้กับระบบต้นแบบ	121
ฉ.1	ข้อมูลประกอบการประเมิน Dashboard/HMI และ SUS score	123
ฉ.2	ข้อมูล task scenario ที่ใช้ประเมิน Dashboard/HMI	123

ซ.1	ลำดับ firmware loop ของ M5StickC Plus2	126
ซ.2	ผล latency ของ server path ที่ใช้ประกอบบทที่ 4	126
ซ.3	ผล end-to-end scenario ของระบบต้นแบบ	127
ซ.4	ลำดับการทำงานของ Flutter Mobile Gateway	128
ซ.5	ลำดับ server ingest และการนำข้อมูลไปใช้	128
ซ.6	ลำดับ AI/MCP pipeline แบบ 5 layer	129

สารบัญรูป

	หน้า
1.1	บริบทการดูแลผู้สูงอายุและผู้พิการที่ใช้เก้าอี้รถเข็นในสถานดูแลของประเทศไทย
1.2	อาคารและพื้นที่บริการหลักของบ้านพักคนชราวาระสนะเวศม์ที่ใช้เป็นบริบทศึกษา
1.3	การใช้กระดานบันทึกข้อมูลแบบแอนะล็อกที่พบในหน่วยงานจริง
2.1	กระบวนการสื่อสาร BLE: advertising สำหรับ RSSI และ GATT สำหรับข้อมูล sensor
2.2	รูปแบบ publish/subscribe ของ MQTT ในระบบดูแล: gateway ส่งข้อมูลผ่าน broker ไปยัง server และ dashboard
2.3	ตัวอย่างหน้าจอ dashboard ของระบบ smart environment ที่รวมสถานะจากอุปกรณ์ IoT หลายชนิด
2.4	อุปกรณ์ฮาร์ดแวร์หลักที่เกี่ยวข้องกับระบบต้นแบบ
2.5	กระบวนการระบุตำแหน่งภายในอาคารด้วย BLE RSSI และ fingerprinting
2.6	แนวคิด Retrieval Logic ในระบบ RAG
2.7	แนวคิด AI pipeline แบบผสม rule และ model สำหรับระบบที่เรียกใช้เครื่องมือ
3.1	สถาปัตยกรรมโดยรวมของระบบ WheelSense จากอุปกรณ์ภาคสนามไปยัง mobile gateway, platform services, dashboard และ AI
3.2	ลำดับการปรับแนวทางการเชื่อมต่ออุปกรณ์ภาคสนามจากการทดลองภายในพื้นที่ทดสอบ
3.3	วงรอบเฟิร์มแวร์ Node T-SimCam สำหรับ config portal, HTTP status, MQTT status/ack/control และการส่งภาพแบบ capture/stream
3.4	ขั้นตอนการเชื่อมต่อรวมระหว่างอุปกรณ์ภาคสนาม mobile gateway และบริการส่วนกลาง

3.5	ตัวอย่างตำแหน่งติดตั้ง M5StickC Plus2 บนรถเข็นสำหรับอ่านค่าการเคลื่อนไหวและส่งข้อมูลผ่าน BLE ไปยังโทรศัพท์	43
3.6	แผนผังวงรอบการทำงานของเฟิร์มแวร์ M5StickC Plus2 แบบ BLE wheelchair sensor	45
3.7	การใช้งาน Polar Verity Sense ร่วมกับสมาร์ตโฟนที่ทำหน้าที่เป็น mobile gateway	46
3.8	หน้าจอ /WheelSensor สำหรับตรวจสอบสถานะ BLE, Polar, M5StickC Plus2 และการส่งข้อมูลขึ้น server	47
3.9	วงรอบการทำงานของ Flutter mobile gateway สำหรับรวมข้อมูล Polar, M5StickC Plus2, RSSI และ step counter	47
3.10	แผนผังพื้นที่ทดลอง 4 zone พร้อมตำแหน่ง BLE node กึ่งกลางแต่ละพื้นที่สำหรับเก็บค่า RSSI	48
3.11	ตัวอย่างตำแหน่ง BLE node ในพื้นที่ทดลองสำหรับเก็บค่า RSSI ภายในอาคาร	49
3.12	Flowchart การส่งข้อมูล RSSI และการประเมินตำแหน่งด้วย KNN หรือ max-RSSI fallback	49
3.13	ลำดับการประมวลผลข้อมูลฝั่ง server ตั้งแต่ ingest, validate, store, query ไปจนถึง dashboard และ AI	50
3.14	แผนภาพ schema ฐานข้อมูลที่ใช้เชื่อม workspaces, users, patients, devices และข้อมูล telemetry	51
3.15	HMI role flow ของระบบ WheelSense แสดงการผ่านสิทธิ์ บริบท workspace และ service ที่ dashboard เรียกใช้งาน	55
3.16	โครงสร้าง AI pipeline 5 layer โดย Layer 3 ทำหน้าที่เตรียม behavioral context นอกเส้นทาง execute หลัก	57
3.17	Layer 1 Intent Router สำหรับจับเจตนาแบบ deterministic ก่อนใช้ semantic matching และ clarification	57
3.18	Layer 2 Context Assembler สำหรับดึงและจัดแพ็คเกจบริบทก่อนส่งเข้า LLM	58

3.19	Layer 4 Constrained LLM สำหรับสังเคราะห์คำตอบหรือ execution candidate ภายใต้ schema ที่กำหนด	58
3.20	Layer 5 Execution Gate สำหรับตรวจ schema, policy และยืนยันการสั่งงานผ่าน MCP tool	59
3.21	Layer 3 Behavioral State สำหรับเตรียมบริบทพฤติกรรมจากข้อมูล sensor และเหตุการณ์ของผู้ใช้	59
3.22	Context Window Management สำหรับจำกัดบริบทให้มีขนาดเหมาะสม และตรวจสอบย้อนกลับได้	60
4.1	การติดตั้ง M5StickC Plus2 บนรถเข็นสำหรับทดสอบข้อมูล IMU และ telemetry	67
4.2	กราฟสรุป accuracy ของการตรวจจับสถานะการเคลื่อนไหวจาก IMU	68
4.3	ตัวอย่างหน้าจอ mobile gateway ที่ใช้ตรวจสอบสถานะตำแหน่งและข้อมูลจาก BLE node	71
4.4	ตัวอย่างสัญญาณ RSSI ต่อเนื่องจากชุดทดลอง Case 1, Case 2 และ Case 3	71
4.5	หน้าจอ mobile gateway สำหรับตรวจสอบการเชื่อมต่อ Polar Verity Sense และสถานะส่งข้อมูล	73
4.6	ตัวอย่างการตรวจสอบบริการบน Raspberry Pi 5 และ Docker runtime ระหว่างการทดสอบ server	74
4.7	หน้าจอ monitor ของ Raspberry Pi 5 ระหว่างรัน service และ container ของระบบ	75
4.8	กราฟสรุปการกระจายของ latency ในเส้นทาง MQTT ระหว่างการทดสอบ	75
4.9	กราฟสรุปค่า P50 และ P95 ของ server path จากการทดสอบ ingest, API, alert และ room prediction	76
4.10	กราฟสรุป task completion ของ dashboard/HMI จากผู้เข้าร่วมทดสอบ 15 คน	78
4.11	หน้าจอ HMI บทบาท Admin	80
4.12	หน้าจอ HMI บทบาท Head Nurse	81

4.13	หน้าจอ HMI บทบาท Supervisor	82
4.14	หน้าจอ HMI บทบาท Observer	83
4.15	หน้าจอ HMI บทบาท Patient	84
4.16	หน้าจอ EaseAI บน Dashboard	85
4.17	กราฟประกอบการวิเคราะห์เวลาแฝงของการประมวลผล LLM ใน pipeline	88
ข.1	กราฟ accuracy ของหน้าต่างข้อมูล IMU ตามสถานะการเคลื่อนไหว	112
ฉ.1	กราฟ task completion ของ dashboard และ mobile gateway ในการประเมิน HMI	124
ฉ.2	หน้าจอ dashboard สำหรับ head nurse ในการติดตามสถานะผู้พักอาศัยและ alert	124
ฉ.3	หน้าจอประกอบ workflow สำหรับตรวจ alert ข้อมูลสถานะพื้นที่ และการติดตามงานดูแล	125
ช.1	กราฟ P50/P95 ของ server path สำหรับ ingest, API, alert และ room prediction	127
ช.1	ตัวอย่างหน้าจอแบบสอบถามประเมินความพึงพอใจส่วนต้น	130
ช.2	ตัวอย่างหน้าจอแบบสอบถามประเมินความพึงพอใจส่วนท้าย	131
ฅ.1	บริบทพื้นที่และการติดตั้ง node อ้างอิงในสถานดูแล	132
ฅ.2	ตัวอย่างการติดตั้งอุปกรณ์ในพื้นที่ทดสอบและบนเก้าอี้รถเข็น	132
ฅ.3	บริบทผู้ใช้และการทดลองใช้อุปกรณ์ต้นแบบ	133
ฅ.4	ภาพประกอบการสรุปแนวคิดและ workflow ระหว่างการออกแบบระบบ	133
ฅ.5	ภาพประกอบการรับฟังความคิดเห็นและการลงพื้นที่	133

สัญลักษณ์และคำย่อ

สัญลักษณ์/คำย่อ	ความหมาย
AAL	Ambient Assisted Living (สภาพแวดล้อมช่วยเหลือนการดำรงชีวิต)
ABAC	Attribute-Based Access Control (การควบคุมการเข้าถึงตามคุณลักษณะ)
ACID	Atomicity, Consistency, Isolation, Durability
ACL	Access Control List (รายการควบคุมการเข้าถึง)
API	Application Programming Interface
BLE	Bluetooth Low Energy (บลูทูธพลังงานต่ำ)
HTTP	Hypertext Transfer Protocol
IMU	Inertial Measurement Unit (หน่วยวัดความเร่ง-หมุน)
IoT	Internet of Things (อินเทอร์เน็ตของสรรพสิ่ง)
IRB	Institutional Review Board (คณะกรรมการจริยธรรมการวิจัย)
IRR	Inter-Rater Reliability (ความสอดคล้องระหว่างผู้ประเมิน)
JSON	JavaScript Object Notation
JSONB	JSON Binary (ของ PostgreSQL)
JWT	JSON Web Token
KNN / k -NN	k -Nearest Neighbors (เพื่อนบ้านใกล้สุด k ตัว)
LLM	Large Language Model (แบบจำลองภาษาขนาดใหญ่)
MCP	Model Context Protocol (โพรโทคอลบริบทสำหรับเครื่องมือของโมเดล)
MQTT	Message Queuing Telemetry Transport
NASA-TLX	NASA Task Load Index
OAuth	Open Authorization (มาตรฐานการอนุญาตแบบเปิด)
PPG	Photoplethysmography (การตรวจวัดสัญญาณชีพด้วยแสง)
QoS	Quality of Service (คุณภาพการบริการของ MQTT)
RBAC	Role-Based Access Control (การควบคุมการเข้าถึงตามบทบาท)
REST	Representational State Transfer
RSSI	Received Signal Strength Indicator (ค่าความแรงสัญญาณที่ได้รับ)
SSE	Server-Sent Events
SUS	System Usability Scale (มาตรวัดความสามารถในการใช้งาน)
TLS	Transport Layer Security

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

จากสถานการณ์คนพิการในประเทศไทย ปี พ.ศ. 2567 [1] พบว่ามีประชากรผู้พิการกว่า 2.2 ล้านคน โดยเฉพาะผู้พิการทางการเคลื่อนไหว ภาครัฐจึงได้มีนโยบายส่งเสริมคุณภาพชีวิตหลายด้าน ทั้งการสร้างอาชีพผ่านรายงานการจ้างงานคนพิการ [2] การสนับสนุนสิทธิและการใช้ชีวิตอิสระตามอนุสัญญาว่าด้วยสิทธิคนพิการ (Convention on the Rights of Persons with Disabilities: CRPD) [3] รวมถึงการประยุกต์ใช้เทคโนโลยีเพื่อการดำรงชีวิตอิสระสำหรับผู้สูงอายุและผู้พิการในแนวคิด Ambient Assisted Living (AAL) [4] เพื่อลดอุปสรรคในการดำเนินชีวิต นอกจากนี้ยังมีการส่งเสริมการมีส่วนร่วมทางสังคมตามแผนพัฒนาการกีฬาแห่งชาติ ฉบับที่ 6 [5] และการสร้างสภาพแวดล้อมที่ปราศจากอุปสรรคตามแผนพัฒนาคุณภาพชีวิตคนพิการแห่งชาติ ฉบับที่ 5 [6]

ผู้ใช้เก้าอี้รถเข็นและผู้สูงอายุในสถานดูแล เช่น บ้านพักคนชราवासนะเวชม์ ซึ่งเป็นบริบทศึกษาของโครงการนี้ ต้องพึ่งพาผู้ดูแลและเผชิญข้อจำกัดด้านการเคลื่อนที่และความปลอดภัย



รูปที่ 1.1: บริบทการดูแลผู้สูงอายุและผู้พิการที่ใช้เก้าอี้รถเข็นในสถานดูแลของประเทศไทย

จากการลงพื้นที่สำรวจอาคารและพื้นที่บริการหลัก พบว่ามีผู้พักอาศัยกระจายอยู่ในหลายอาคาร ทำให้ผู้ดูแลไม่สามารถทราบตำแหน่งและสภาพของผู้ใช้เก้าอี้รถเข็นทุกคนได้ทันเวลา ซึ่ง

อาจนำไปสู่อุบัติเหตุ เช่น การหกล้ม [7]



รูปที่ 1.2: อาคารและพื้นที่บริการหลักของบ้านพักคนชราวาสนะเวศม์ที่ใช้เป็นบริบทศึกษา

นอกจากนี้ การทำงานของผู้ดูแลยังมีข้อจำกัดจากการบันทึกข้อมูลแบบแอนะล็อก เช่น กระดานหรือกระดาษ ซึ่งเสี่ยงต่อการสูญหายและไม่สามารถนำมาวิเคราะห์แนวโน้มสุขภาพได้ ปริมาณงานที่มากยังนำไปสู่ความล้าจากการแจ้งเตือน (alert fatigue) [8]



รูปที่ 1.3: การใช้กระดานบันทึกข้อมูลแบบแอนะล็อกที่พบในหน้างานจริง

แม้ปัจจุบันจะมีระบบอัตโนมัติในบ้านและระบบระบุตำแหน่งภายในอาคารหลายรูปแบบ [9], [10], [11] แต่มักไม่สอดคล้องกับพฤติกรรมของผู้ใช้เก้าอี้รถเข็น มีความซับซ้อน หรือมีต้นทุนสูง โครงการนี้จึงเสนอการพัฒนาแพลตฟอร์ม WheelSense ซึ่งเป็นระบบสภาพแวดล้อมอัจฉริยะแบบบูรณาการที่เชื่อมโยงอุปกรณ์บนเก้าอี้รถเข็น อุปกรณ์สวมใส่ โหนดภายในอาคาร และเซิร์ฟเวอร์ขอบ เพื่อสนับสนุนการทำงานของผู้ดูแลและเพิ่มความปลอดภัยให้แก่ผู้ใช้งานอย่างมีประสิทธิภาพ

1.2 วัตถุประสงค์

1. เพื่อพัฒนาระบบต้นแบบสภาพแวดล้อมอัจฉริยะสำหรับผู้ใช้อีอาร์เอ็นที่สามารถรับข้อมูลการเคลื่อนไหว สัญญาณชีพ และบริบทของพื้นที่ภายในอาคารได้อย่างต่อเนื่อง
2. เพื่อพัฒนาระบบระบุตำแหน่งภายในอาคารระดับโซนหรือห้อง โดยอาศัยสัญญาณ BLE และการประมวลผลบนเซิร์ฟเวอร์
3. เพื่อพัฒนาแพลตฟอร์มซอฟต์แวร์สำหรับจัดเก็บ ประมวลผล และแสดงผลข้อมูลแก่ผู้ใช้งานตามบทบาท ผ่านเว็บ Dashboard และแอปพลิเคชันบนอุปกรณ์เคลื่อนที่
4. เพื่อบูรณาการผู้ช่วยปัญญาประดิษฐ์ภายใต้กรอบเสนอ-ยืนยัน-ดำเนินการ เพื่อช่วยสรุปข้อมูลและสนับสนุนการตัดสินใจของผู้ดูแลอย่างปลอดภัย
5. เพื่อประเมินระบบต้นแบบในด้านความถูกต้องของการระบุตำแหน่ง ความเสถียรของการไหลของข้อมูล ประสิทธิภาพของส่วนบริการ และประสบการณ์ผู้ใช้ของผู้ดูแล

1.3 ขอบเขตงานวิจัย

1. พัฒนาชุดอุปกรณ์ภาคสนามสำหรับติดตั้งบนเก้าอี้รถเข็นและอุปกรณ์สวมใส่ที่ใช้เก็บข้อมูลการเคลื่อนไหวและสัญญาณชีพพื้นฐาน
2. พัฒนาระบบระบุตำแหน่งภายในอาคารระดับโซนหรือห้อง โดยใช้ข้อมูล BLE RSSI และข้อมูลอ้างอิงจากพื้นที่ที่กำหนด
3. พัฒนาเว็บ Dashboard และแอปพลิเคชันบนอุปกรณ์เคลื่อนที่สำหรับการใช้งานตามบทบาทของผู้ดูแล ผู้ปฏิบัติงาน และผู้เกี่ยวข้อง
4. บูรณาการผู้ช่วยปัญญาประดิษฐ์แบบภายในระบบ โดยจำกัดการกระทำที่มีผลเปลี่ยนสถานะให้อยู่ภายใต้ขั้นตอนยืนยันก่อนดำเนินการ
5. ประเมินระบบต้นแบบภายใต้สภาพแวดล้อมทดลองและการใช้งานสาธิตในบริบทสถานดูแล โดยไม่มุ่งหมายเป็นการทดลองทางคลินิกหรือการรับรองเป็นเครื่องมือแพทย์

1.4 ประโยชน์ที่คาดว่าจะได้รับ

ประโยชน์ต่อสังคม

1. ช่วยยกระดับคุณภาพการดูแลผู้สูงอายุและผู้ใช้เก้าอี้รถเข็นในสถานดูแล โดยเพิ่มความสามารถในการเฝ้าระวัง ความปลอดภัย การติดตามตำแหน่ง และการรับรู้เหตุผิดปกติได้รวดเร็วยิ่งขึ้น
2. ช่วยลดภาระงานซ้ำซ้อนของผู้ดูแลและสนับสนุนการประสานงานข้ามเวรด้วยข้อมูลที่ค้นย้อนกลับได้ ซึ่งเอื้อต่อการจัดการทรัพยากรบุคคลอย่างมีประสิทธิภาพมากขึ้น

องค์ความรู้ที่ได้รับ

1. ได้องค์ความรู้ด้านการออกแบบระบบสภาพแวดล้อมอัจฉริยะที่บูรณาการอุปกรณ์ภาคสนาม บริการส่วนกลาง และส่วนติดต่อผู้ใช้เข้าด้วยกันในบริบทการดูแล
2. ได้องค์ความรู้ด้านการประยุกต์ใช้ผู้ช่วยปัญญาประดิษฐ์อย่างปลอดภัยในงานเชิงปฏิบัติการที่เกี่ยวข้องกับข้อมูลผู้พักอาศัยและการตัดสินใจของผู้ดูแล

1.5 แผนการดำเนินงาน

การดำเนินงานวิจัยครอบคลุมระยะเวลา 12 เดือน โดยแบ่งเป็นช่วงการวิเคราะห์ปัญหา การศึกษาเอกสารที่เกี่ยวข้อง การออกแบบและสร้างระบบต้นแบบ การบูรณาการและทดสอบ ตลอดจนการวิเคราะห์ผลและจัดทำเล่มวิทยานิพนธ์ ดังรายละเอียดในตารางที่ 1.1

ตารางที่ 1.1: แผนการดำเนินงานของโครงการ

กิจกรรม	เดือนที่											
	1	2	3	4	5	6	7	8	9	10	11	12
1. วิเคราะห์ปัญหาและเก็บบริบทหน้างาน	→	→										
2. ศึกษาวรรณกรรมและทฤษฎีที่เกี่ยวข้อง	→	→	→									
3. ออกแบบสถาปัตยกรรมระบบและ data flow		→	→	→								
4. พัฒนา wheelchair sensor และ wearable gateway			→	→	→	→						
5. พัฒนาระบบระบุตำแหน่งภายในอาคาร				→	→	→	→					
6. พัฒนา backend, database และ MQTT service				→	→	→	→	→				
7. พัฒนา dashboard และ HMI ตามบทบาทผู้ใช้					→	→	→	→	→			
8. บูรณาการ AI/MCP pipeline และ execution gate						→	→	→	→			
9. ทดสอบ sensor, localization, gateway และ server								→	→	→		
10. ประเมิน dashboard, AI และสรุปผลการทดลอง									→	→	→	
11. จัดทำและปรับปรุงวิทยานิพนธ์										→	→	→

หมายเหตุ: → หมายถึงช่วงเวลาที่ยังดำเนินกิจกรรม

บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

บทนี้รวบรวมแนวคิด ทฤษฎี และงานวิจัยที่เกี่ยวข้องกับระบบดูแลผู้สูงอายุและผู้
ใช้รถเข็นในสภาพแวดล้อมอัจฉริยะ โดยเน้นเฉพาะพื้นฐานที่จำเป็นต่อการออกแบบระบบ ได้แก่
แนวคิด Ambient Assisted Living, เทคโนโลยี IoT และเซ็นเซอร์, การระบุตำแหน่งภายในอาคาร
ด้วย BLE RSSI, การประมวลผลข้อมูล IMU, Machine Learning สำหรับจำแนกพฤติกรรม,
Retrieval-Augmented Generation และแนวคิด Neuro-symbolic AI แบบผสมระหว่างกฎกับ
โมเดลภาษา

2.1 แนวคิดระบบดูแลผู้สูงอายุและผู้ใช้รถเข็น

การดูแลผู้สูงอายุและผู้ใช้รถเข็นเป็นงานที่ต้องติดตามหลายมิติพร้อมกัน ทั้งตำแหน่งของ
ผู้พักอาศัย การเคลื่อนไหว ความผิดปกติของสัญญาณชีพ การตอบสนองต่อเหตุฉุกเฉิน และภาระงาน
ของบุคลากรดูแล ในบริบทของสังคมสูงวัย จำนวนผู้สูงอายุที่ต้องพึ่งพาผู้ดูแลมีแนวโน้มเพิ่มขึ้นต่อเนื่อง
ขณะที่บุคลากรในสถานดูแลมีข้อจำกัดด้านเวลาและจำนวนคน การออกแบบระบบช่วยดูแลจึงควร
ช่วยลดงานตรวจสอบซ้ำ ๆ ของผู้ดูแล และทำให้ข้อมูลสำคัญถูกรวบรวมอยู่ในจุดที่สามารถมองเห็นและ
ตัดสินใจได้เร็วขึ้น [12], [13]

แนวคิด Ambient Assisted Living (AAL) เป็นแนวคิดที่ใช้เทคโนโลยีสารสนเทศและ
สภาพแวดล้อมอัจฉริยะเพื่อสนับสนุนการใช้ชีวิตประจำวันของผู้สูงอายุหรือผู้ที่มีข้อจำกัดด้านร่างกาย
ระบบในกลุ่มนี้มักประกอบด้วยเซ็นเซอร์ อุปกรณ์สื่อสาร ฐานข้อมูล และกลไกแจ้งเตือน เพื่อช่วยให้ผู้
ดูแลรับรู้เหตุการณ์สำคัญได้ทันเวลาโดยไม่ต้องตรวจด้วยตนเองตลอดเวลา [4] ในงานด้านผู้ใช้อรถเข็น
ข้อมูลตำแหน่งและการเคลื่อนไหวมีความสำคัญมากเป็นพิเศษ เพราะสามารถสะท้อนรูปแบบกิจกรรม
ความเสี่ยงในการหยุดนิ่งนานผิดปกติ หรือการเคลื่อนที่ออกจากบริเวณที่กำหนด

Smart environment เป็นการขยายแนวคิด AAL ไปสู่ระดับพื้นที่ โดยทำให้อุปกรณ์
และบริการภายในอาคารเชื่อมโยงกัน เช่น เซ็นเซอร์ประจำตัวผู้ใช้ อุปกรณ์สวมใส่ จุดอ้างอิงสัญญาณ
ภายในอาคาร ระบบแจ้งเตือน และ dashboard สำหรับผู้ดูแล จุดสำคัญไม่ใช่เพียงการเก็บข้อมูลให้ได้

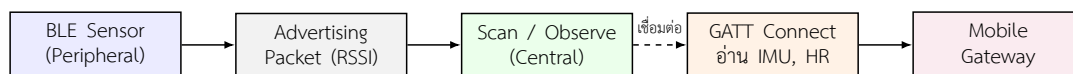
มากที่สุด แต่เป็นการจัดข้อมูลให้มีบริบท เช่น ใครอยู่ที่ไหน กำลังเคลื่อนไหวหรือหยุดนิ่ง มีสัญญาณชีพผิดปกติหรือไม่ และควรส่งต่อเหตุการณ์ใดให้ผู้ดูแลตรวจสอบก่อน

จากแนวคิดดังกล่าว ระบบดูแลผู้สูงอายุและผู้ใช้รถเข็นจึงควรมีคุณสมบัติหลัก 4 ประการ ได้แก่ การรับข้อมูลจากเซ็นเซอร์ที่สัมพันธ์กับสถานะจริงของผู้ใช้ การส่งข้อมูลเข้าสู่ระบบกลางอย่างต่อเนื่อง การจัดเก็บและแสดงผลข้อมูลในรูปแบบที่ผู้ดูแลอ่านได้เร็ว และการมีกลไกช่วยจัดลำดับความสำคัญของเหตุการณ์ก่อนส่งให้มนุษย์ตรวจสอบ ทั้งหมดนี้เป็นพื้นฐานของการออกแบบระบบต้นแบบในงานนี้ แต่ในบทนี้จะอธิบายในระดับทฤษฎีเพื่อไม่ให้ซ้ำกับรายละเอียดการพัฒนาในบทถัดไป

2.2 เทคโนโลยี IoT และ Sensor สำหรับระบบดูแล

Internet of Things (IoT) หมายถึงการเชื่อมต่ออุปกรณ์กายภาพเข้ากับระบบสารสนเทศเพื่อเก็บ ส่ง และประมวลผลข้อมูลจากสภาพแวดล้อมจริง อุปกรณ์ IoT ในระบบดูแลผู้สูงอายุอาจเป็นเซ็นเซอร์ติดรถเข็น อุปกรณ์สวมใส่ โทรศัพท์มือถือ หรือ gateway ภายในพื้นที่ทดลอง จุดร่วมของอุปกรณ์เหล่านี้คือมีข้อจำกัดด้านพลังงาน ขนาด หน่วยความจำ และคุณภาพสัญญาณ จึงต้องเลือก protocol และโครงสร้างข้อมูลให้เหมาะกับงาน [14]

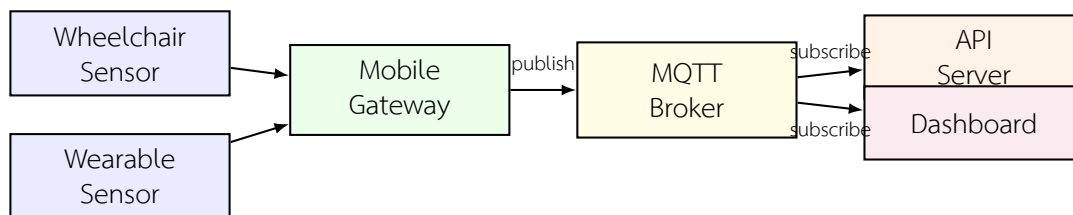
Bluetooth Low Energy (BLE) เป็นเทคโนโลยีสื่อสารระยะใกล้ที่เหมาะสมกับอุปกรณ์ใช้พลังงานต่ำ BLE รองรับทั้งการโฆษณาแพ็กเก็ตแบบ advertising และการเชื่อมต่อผ่าน Generic Attribute Profile (GATT) ทำให้อุปกรณ์สามารถส่งข้อมูลสั้น ๆ เช่น RSSI, battery, IMU หรือค่าจาก sensor ไปยัง gateway ได้ โดยค่า Received Signal Strength Indicator (RSSI) สามารถนำไปใช้เป็นข้อมูลประกอบการประเมินระยะหรือห้องที่อุปกรณ์อยู่ได้ แม้ค่า RSSI จะมีความผันผวนจากสิ่งกีดขวาง การสะท้อนของสัญญาณ และตำแหน่งของอุปกรณ์ [15]



รูปที่ 2.1: กระบวนการสื่อสาร BLE: advertising สำหรับ RSSI และ GATT สำหรับข้อมูล sensor

MQTT เป็น protocol แบบ publish/subscribe ที่นิยมในระบบ IoT เพราะ payload มีขนาดเล็กและแยกผู้ส่งกับผู้รับออกจากกัน อุปกรณ์หรือ gateway สามารถ publish telemetry ไปยัง topic ที่กำหนด ขณะที่ server หรือ dashboard subscribe เฉพาะ topic ที่ต้องใช้ หลักการนี้เหมาะกับระบบที่มีข้อมูลหลายชนิด เช่น ตำแหน่ง สัญญาณชีพ และสถานะอุปกรณ์ เพราะทำให้เพิ่ม

แหล่งข้อมูลใหม่ได้โดยไม่ต้องผูกทุก service เข้าหากันโดยตรง [16], [17]



รูปที่ 2.2: รูปแบบ publish/subscribe ของ MQTT ในระบบดูแล: gateway ส่งข้อมูลผ่าน broker ไปยัง server และ dashboard

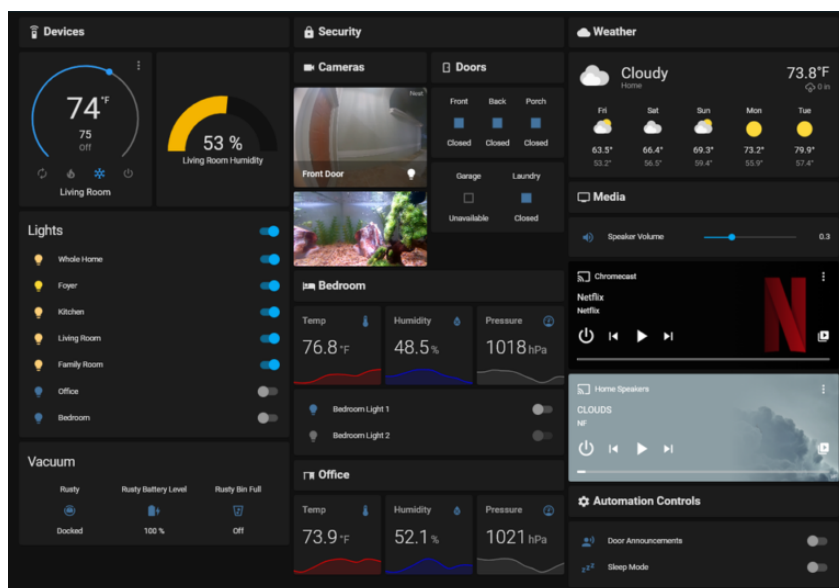
เซ็นเซอร์ IMU และ wearable sensor เป็นแหล่งข้อมูลสำคัญในงานดูแลผู้สูงอายุ IMU มักประกอบด้วย accelerometer และ gyroscope สำหรับวัดความเร่งและอัตราการหมุน ส่วน wearable sensor เช่นอุปกรณ์วัดแสงสะท้อนจากผิวหนังด้วยหลักการ Photoplethysmography (PPG) สามารถใช้ติดตาม heart rate, pulse-to-pulse interval หรือข้อมูลสัญญาณชีพที่สัมพันธ์กับภาวะร่างกายได้ [18], [19] อย่างไรก็ตาม ข้อมูลเหล่านี้ควรถูกใช้เป็นข้อมูลประกอบการประเมินเบื้องต้น ไม่ใช่การวินิจฉัยทางการแพทย์

ในระบบที่ต้องประมวลผลภายในพื้นที่ทดลอง แนวคิด edge หรือ local server มีบทบาทในการลดการพึ่งพา cloud ภายนอก ทำให้ข้อมูลส่วนบุคคลไม่จำเป็นต้องออกนอกระบบ และช่วยให้ dashboard หรือระบบแจ้งเตือนทำงานได้แม้ internet ภายนอกไม่เสถียร โครงสร้างทั่วไปมักประกอบด้วย gateway ที่รับข้อมูลจากอุปกรณ์ภาคสนาม server ที่ตรวจสอบและจัดเก็บข้อมูล ฐานข้อมูลสำหรับ telemetry และ frontend/dashboard สำหรับแสดงผล

2.3 อุปกรณ์ฮาร์ดแวร์หลักที่เกี่ยวข้อง

M5StickC Plus2 เป็นบอร์ดไมโครคอนโทรลเลอร์ขนาดเล็กที่มี ESP32, หน้าจอ, ปุ่ม, แบตเตอรี่ และ IMU ในตัว จึงเหมาะกับการทดลองเป็น sensor ดิทรถเซ็นเพื่ออ่านค่าการเคลื่อนไหวและสถานะแบตเตอรี่ ข้อดีของอุปกรณ์ลักษณะนี้คือมีขนาดเล็ก ติดตั้งง่าย และสามารถส่งข้อมูลผ่าน BLE ไปยังอุปกรณ์กลางได้ ในบริบทของงานนี้ M5StickC Plus2 ถูกพิจารณาในฐานะ wheelchair sensor ที่อ่านข้อมูล IMU/battery และส่ง payload ไปยัง mobile gateway ไม่ใช่อุปกรณ์ที่ต้องทำหน้าที่เป็น gateway หลักของระบบทั้งหมด [20]

LilyGO T-SimCam เป็นบอร์ดที่มีความสามารถด้าน cellular และ camera module จึงเคยเหมาะกับแนวคิด node หรือ camera node ในต้นแบบช่วงแรก โดยเฉพาะการทดลอง



รูปที่ 2.3: ตัวอย่างหน้าจอ dashboard ของระบบ smart environment ที่รวมสถานะจากอุปกรณ์ IoT หลายชนิด

ตารางที่ 2.1: บทบาทของเทคโนโลยี IoT และ sensor ในระบบดูแล

เทคโนโลยี	หลักการสำคัญ	เหตุผลที่เกี่ยวข้องกับระบบดูแล
BLE	สื่อสารระยะใกล้ ใช้พลังงานต่ำ รองรับ advertising และ GATT	ใช้เชื่อม sensor กับ mobile gateway และใช้ RSSI เป็นข้อมูลตำแหน่งภายในอาคาร
MQTT	publish/subscribe ผ่าน broker	ส่ง telemetry หลายชนิดไปยัง server โดยไม่ผูกอุปกรณ์เข้ากับ service โดยตรง
IMU	วัดความเร่งและอัตราการหมุนหลายแกน	ใช้วิเคราะห์การเคลื่อนไหวของรถเข็นหรือกิจกรรมพื้นฐาน
Wearable sensor	วัดข้อมูลชีวสัญญาณ เช่น HR, RR interval และ PPG	เพิ่มข้อมูลสถานะร่างกายเพื่อประกอบการเฝ้าระวัง
Edge/local server	ประมวลผลและจัดเก็บข้อมูลในพื้นที่	ลด latency และลดการพึ่งพาระบบ cloud ภายนอก

โครงข่ายที่ต้องส่งข้อมูลหรือภาพผ่าน node ภาคสนาม อย่างไรก็ตาม อุปกรณ์ประเภทนี้มีความซับซ้อนด้านการติดตั้ง พลังงาน และการจัดเส้นทางข้อมูลมากกว่า sensor ขนาดเล็ก ในระบบสุดท้ายจึงควรถูกอธิบายในฐานะอุปกรณ์ที่เคยใช้ในต้นแบบหรือแนวคิด node เดิม ไม่ใช่แกนหลักของ architecture ปัจจุบัน [21]

Polar Verity Sense เป็นอุปกรณ์สวมใส่แบบ optical armband สำหรับอ่านข้อมูล heart rate และข้อมูลที่เกี่ยวข้องกับ PPG ผ่านการเชื่อมต่อ BLE และ Polar SDK อุปกรณ์นี้มีบทบาทในการเพิ่มข้อมูลด้านสัญญาณชีพให้ระบบดูแล เช่น HR, RR interval หรือ PPG stream โดยการอ่านค่าระดับลึกต้องอาศัย SDK ที่ผู้ผลิตเตรียมไว้ ข้อมูลจากอุปกรณ์สวมใส่จึงช่วยเสริมข้อมูลการเคลื่อนไหวจากรถเข็น แต่ยังคงถูกตีความอย่างระมัดระวังในฐานะข้อมูลประกอบ ไม่ใช่ผลวินิจฉัย [18], [22]

Raspberry Pi 5 RAM 8 GB เป็น single-board computer ที่มีพลังประมวลผลเพียงพอสำหรับบทบาท local/edge server ในระบบต้นแบบ เช่น API service, database, MQTT broker, dashboard และ AI runtime ขนาดเล็ก การใช้เครื่องลักษณะนี้ช่วยให้ระบบทดลองสามารถทำงานในพื้นที่ได้โดยไม่ต้องพึ่ง cloud ทั้งหมด และยังเปิดโอกาสให้ทดสอบการ deploy ด้วย Docker, การจัดเก็บข้อมูลใน PostgreSQL และการเรียกใช้ AI/MCP ภายในระบบเดียวกัน [23]

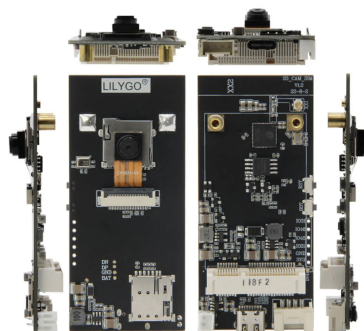
ตารางที่ 2.2: สเปคสำคัญของอุปกรณ์ฮาร์ดแวร์หลัก

คุณสมบัติ	M5StickC Plus2	Polar Verity Sense	Raspberry Pi 5
MCU / CPU	ESP32-PICO-V3	Nordic nRF5340	BCM2712 Cortex-A76
Sensor	IMU 6 แกน (MPU-6886)	PPG (HR, RR interval)	–
การเชื่อมต่อ	BLE 4.2, WiFi	BLE, ANT+	WiFi 5, GbE, BLE 5.0
แบตเตอรี่	200 mAh LiPo	110 mAh (สวมใส่ 20+ ชม.)	ต้องใช้ไฟภายนอก 5V/5A
RAM	520 KB SRAM	–	8 GB LPDDR4X
บทบาทในระบบ	Wheelchair sensor	Wearable sensor	Edge / local server

นอกจากอุปกรณ์หลักข้างต้น ระบบดูแลยังอาจมีอุปกรณ์รอง เช่น BLE beacon หรือ node อ้างอิงสัญญาณ โทรศัพท์มือถือที่ทำหน้าที่ gateway เครื่องชาร์จ แหล่งจ่ายไฟสำรอง และอุปกรณ์เครือข่ายภายในอาคาร อุปกรณ์เหล่านี้มีผลต่อความเสถียรของระบบ แต่ในเชิงทฤษฎีสามารถ



(a) M5StickC Plus2



(b) LilyGO T-SimCam



(c) Polar Verity Sense



(d) Raspberry Pi 5

รูปที่ 2.4: อุปกรณ์ฮาร์ดแวร์หลักที่เกี่ยวข้องกับระบบต้นแบบ

จัดเป็นส่วนสนับสนุนของ sensor layer, network layer และ edge layer ตามบทบาทที่กล่าวไว้ก่อนหน้านี้

2.4 เครื่องมือซอฟต์แวร์และไลบรารีที่ใช้พัฒนา

นอกจากฮาร์ดแวร์แล้ว ระบบต้นแบบยังอาศัยเครื่องมือซอฟต์แวร์หลายชนิดที่ทำงานร่วมกัน ตารางที่ 2.3 สรุปเครื่องมือหลักที่เกี่ยวข้องกับการพัฒนาระบบ โดยแต่ละเครื่องมือถูกเลือกจากเหตุผลเชิงเทคนิคและข้อจำกัดของ prototype

Flutter เป็น SDK สำหรับพัฒนาแอปพลิเคชันมือถือแบบ cross-platform ที่ใช้ภาษา Dart สามารถเชื่อมต่อ BLE ผ่าน plugin และสร้าง UI สำหรับแสดงสถานะ sensor ได้ ในระบบนี้ Flutter ถูกใช้พัฒนา mobile gateway ที่รับข้อมูลจาก sensor ตีตรถเข็นและ wearable sensor ผ่าน BLE แล้วส่งต่อไปยัง server ผ่าน MQTT หรือ REST API [24]

Next.js เป็น React-based web framework ที่รองรับ server-side rendering และ API routes ใช้พัฒนา dashboard สำหรับ admin และ caregiver ให้สามารถดูข้อมูลผู้พักอาศัย

ตารางที่ 2.3: เครื่องมือซอฟต์แวร์และไลบรารีหลักที่ใช้ในระบบต้นแบบ

เครื่องมือ	ประเภท	หน้าที่	เหตุผลที่เลือกใช้
Flutter	Mobile SDK	พัฒนา mobile gateway app สำหรับ Android	รองรับ BLE plugin, cross-platform, UI สำหรับ caregiver
Next.js	Web framework	พัฒนา dashboard สำหรับ admin และ caregiver	Server-side rendering, real-time update
FastAPI	Backend API	RESTful API, WebSocket, business logic	Async, type-safe, รองรับ Python AI ecosystem
PostgreSQL	Database	จัดเก็บ telemetry, user, alert ด้วย JSONB	รองรับ JSON payload ยืดหยุ่น
Docker	Containerization	Deploy service ทั้งหมดบน Raspberry Pi 5	แยก environment, ทำซ้ำได้
Ollama + Gemma 3 4B	AI runtime	Local LLM สำหรับ AI assistant	ทำงานบน edge ได้, ไม่ต้องพึ่ง cloud
FastMCP	MCP server	กำหนด tool contract สำหรับ AI เรียกใช้	สอดคล้องกับ MCP specification
PlatformIO	Embedded IDE	พัฒนา firmware สำหรับ M5StickC Plus2	รองรับ ESP32, จัดการ library อัตโนมัติ

สถานะ sensor ตำแหน่งภายในอาคาร และประวัติการแจ้งเตือน [25]

FastAPI เป็น Python web framework แบบ asynchronous ที่รองรับ type hint และ auto-documentation ทำหน้าที่เป็น backend หลักของระบบ รับ telemetry จาก gateway, ประมวลผล business logic, จัดเก็บข้อมูลใน PostgreSQL และให้บริการ WebSocket สำหรับ real-time update [26]

PostgreSQL เป็นระบบฐานข้อมูลเชิงสัมพันธ์แบบ open source ที่รองรับ JSONB ทำให้สามารถจัดเก็บ telemetry payload ที่มีโครงสร้างยืดหยุ่นได้โดยไม่ต้องเปลี่ยน schema ทุกครั้งที่เพิ่มข้อมูลใหม่ [27]

Docker เป็นเครื่องมือ containerization ที่ช่วยให้ deploy service ทุกส่วนบน Raspberry Pi 5 ด้วย docker-compose ทำให้แต่ละ service แยก environment และทำซ้ำได้ [28]

Ollama เป็น runtime สำหรับเรียกใช้ Large Language Model บน local machine ในระบบนี้ใช้ร่วมกับ Gemma 3 4B เพื่อให้ AI assistant ทำงานบน edge server ได้โดยไม่ต้องส่ง

ข้อมูลไปยัง cloud ภายนอก [29]

FastMCP เป็น Python library สำหรับสร้าง MCP server ตาม Model Context Protocol specification ทำให้ AI client สามารถเรียกใช้เครื่องมือภายในระบบผ่าน tool contract ที่มีการกำหนด input/output schema ชัดเจน [30]

2.5 การระบุตำแหน่งภายในอาคารด้วย BLE RSSI

การระบุตำแหน่งภายในอาคารมีความแตกต่างจากการระบุตำแหน่งกลางแจ้ง เนื่องจาก GPS ทำงานได้ไม่ดีในอาคารที่มีผนัง เพดาน และสัญญาณสะท้อนจำนวนมาก ระบบภายในอาคาร จึงมักใช้สัญญาณวิทยุจาก WiFi, BLE, UWB หรือ RFID แทน สำหรับระบบดูแลผู้สูงอายุ BLE เป็นตัวเลือกที่เหมาะสมกับต้นแบบเพราะอุปกรณ์มีราคาต่ำ ใช้พลังงานน้อย และสามารถวัด RSSI ได้จาก อุปกรณ์รับสัญญาณทั่วไป

แนวคิดพื้นฐานของ RSSI คือความแรงของสัญญาณที่ตัวรับวัดได้จะลดลงเมื่อระยะห่างจากตัวส่งเพิ่มขึ้น แบบจำลองที่ใช้บ่อยคือ log-distance path loss model ดังสมการ (2.1) [31]

$$d = d_0 \times 10^{\frac{P(d_0) - P(d)}{10n}} \quad (2.1)$$

เมื่อ d คือระยะที่ต้องการประมาณ, d_0 คือระยะอ้างอิง, $P(d_0)$ คือ RSSI ที่ระยะอ้างอิง, $P(d)$ คือ RSSI ที่วัดได้ และ n คือ path loss exponent ของสภาพแวดล้อม แม้สมการนี้ช่วยอธิบายความสัมพันธ์ระหว่าง RSSI กับระยะทางได้ แต่ในอาคารจริง RSSI มีความผันผวนสูงจาก multipath, สิ่งกีดขวาง และทิศทางของเสาอากาศ ดังนั้นการคำนวณระยะตรง ๆ แล้วหาจุดตัดแบบ trilateration จึงไม่ใช่แกนหลักที่เหมาะสมกับระบบนี้

วิธี fingerprinting แก้ปัญหาดังกล่าวโดยแบ่งงานเป็นสองช่วง ช่วงแรกคือ offline phase ที่วัดค่า RSSI จากตำแหน่งอ้างอิงหลายจุดและบันทึกเป็น radio map ช่วงที่สองคือ online phase ที่นำ RSSI ที่วัดได้จริงมาเปรียบเทียบกับ radio map เพื่อหาตำแหน่งที่มีรูปแบบสัญญาณใกล้เคียงที่สุด แนวคิดนี้ถูกใช้ในงาน indoor localization อย่างแพร่หลายตั้งแต่งานกลุ่ม RADAR และต่อ ยอดมาในระบบ BLE RSSI หลายรูปแบบ [32]

K-Nearest Neighbors (KNN) เป็นวิธีพื้นฐานที่ใช้กับ fingerprinting ได้ตรงไปตรงมา โดยแทนค่า RSSI ที่วัดได้เป็นเวกเตอร์ $\mathbf{x}$ และเปรียบเทียบกับ fingerprint ที่เก็บไว้เป็น $\mathbf{r}_j$ ระยะห่าง

ระหว่างเวกเตอร์คำนวณได้ตามสมการ (2.2) [33]

$$D_j = \sqrt{\sum_{i=1}^m (x_i - r_{j,i})^2} \quad (2.2)$$

เมื่อ m คือจำนวน beacon หรือ node อ้างอิง, x_i คือ RSSI ที่วัดได้จาก node ที่ i และ $r_{j,i}$ คือค่า RSSI ของ fingerprint ลำดับที่ j จากนั้นเลือก fingerprint ที่มีระยะน้อยที่สุดจำนวน K จุด เพื่อนำมาประเมินห้องหรือพิกัด การใช้ distance-weighted KNN ช่วยให้น้ำหนักของจุดที่ใกล้เคียงกว่าใน feature space มากกว่าจุดที่อยู่ไกลกว่า ดังสมการ (2.3)

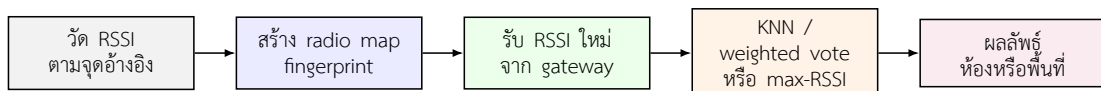
$$w_j = \frac{1}{D_j + \varepsilon}, \quad \hat{c} = \arg \max_c \sum_{j \in N_K} w_j \cdot I(c_j = c) \quad (2.3)$$

เมื่อ N_K คือชุด fingerprint ใกล้เคียงที่สุดจำนวน K จุด, c_j คือคลาสหรือห้องของ fingerprint จุดที่ j , $I(\cdot)$ คือ indicator function และ ε เป็นค่าขนาดเล็กเพื่อป้องกันการหารด้วยศูนย์

ในกรณีที่ข้อมูล fingerprint ไม่ครบหรือค่า RSSI ไม่เหมาะกับการคำนวณ KNN ระบบสามารถใช้ max-RSSI เป็น baseline หรือ fallback ได้ แนวคิดคือเลือก node ที่มีค่า RSSI สูงที่สุดแล้วใช้ตำแหน่งหรือห้องของ node นั้นเป็นค่าประมาณ ดังสมการ (2.4)

$$i^* = \arg \max_i P_i, \quad \hat{c}_{\max} = c_{i^*} \quad (2.4)$$

วิธี max-RSSI มีความเรียบง่ายและเหมาะสมเป็น fallback แต่โดยทั่วไปจะไวต่อสัญญาณสะท้อนหรือการบังสัญญาณมากกว่า fingerprinting ดังนั้นในระบบต้นแบบจึงควรใช้ KNN เป็นวิธีหลักเมื่อมีข้อมูลเพียงพอ และใช้ max-RSSI ในกรณีที่ข้อมูลไม่ครบหรือโมเดลไม่สามารถประเมินได้อย่างมั่นใจ



รูปที่ 2.5: กระบวนการระบุตำแหน่งภายในอาคารด้วย BLE RSSI และ fingerprinting

2.6 การประมวลผลข้อมูล IMU และการเคลื่อนไหว

IMU เป็นเซ็นเซอร์ที่ใช้วัดการเคลื่อนไหวเชิงกล โดย accelerometer วัดความเร่งเชิงเส้นในแกน x, y, z และ gyroscope วัดอัตราการหมุนรอบแกนต่าง ๆ ข้อมูลทั้งสองชนิดสามารถใช้ตรวจสอบการเปลี่ยนแปลงสถานะของรถเข็นหรือกิจกรรมของผู้ใช้ได้ เช่น การเริ่มเคลื่อนที่ การหยุดนิ่ง การเคลื่อนที่ต่อเนื่อง หรือการสั่นผิดปกติ หลักการประมวลผลข้อมูล IMU จึงมักเริ่มจากการลด noise การคำนวณ feature ในช่วงเวลา และการสรุปสถานะจาก feature เหล่านั้น [34]

สำหรับ accelerometer สามารถคำนวณ magnitude ของความเร่งรวมได้ตามสมการ (2.5) เพื่อรวมข้อมูลสามแกนให้เป็นค่าเดียวที่อ่านง่ายขึ้น

$$|\mathbf{a}_t| = \sqrt{a_{x,t}^2 + a_{y,t}^2 + a_{z,t}^2} \quad (2.5)$$

ข้อมูลที่วัดต่อเนื่องมักถูกแบ่งเป็น sliding window ความยาว W ตัวอย่าง แล้วคำนวณค่าสถิติ เช่น ค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐาน ดังสมการ (2.6)

$$\mu = \frac{1}{W} \sum_{t=1}^W x_t, \quad \sigma = \sqrt{\frac{1}{W-1} \sum_{t=1}^W (x_t - \mu)^2} \quad (2.6)$$

อีก feature หนึ่งที่ใช้บ่อยคือจำนวนครั้งที่สัญญาณเปลี่ยนเครื่องหมายหรือ zero-crossing ซึ่งสะท้อนความถี่ของการสั่นหรือการเปลี่ยนทิศทางในช่วงเวลานั้น ดังสมการ (2.7)

$$ZC = \sum_{t=2}^W I(x_t x_{t-1} < 0) \quad (2.7)$$

ในงานที่นำ IMU ไปติดตั้งกับล้อรถเข็น ข้อมูลจาก gyroscope สามารถใช้ประมาณความเร็วและระยะทางของล้อได้ หากให้ ω_t เป็นอัตราการหมุนที่ผ่านการกรองแล้วในหน่วย rad/s และ R เป็นรัศมีล้อ ความเร็วเชิงเส้นโดยประมาณคำนวณได้จากสมการ (2.8)

$$v_t = R\omega_t \quad (2.8)$$

จากนั้นสามารถประมาณความเร็วและระยะทางสะสมได้ตามสมการ (2.9)

$$a_t = \frac{v_t - v_{t-1}}{\Delta t}, \quad s_t = s_{t-1} + v_t \Delta t \quad (2.9)$$

ในทางปฏิบัติ ข้อมูล gyroscope ต้องผ่านการชดเชย offset และตัวกรอง เช่น Exponential Moving Average (EMA) เพื่อให้ค่าที่ใช้คำนวณความเร็วไม่แกว่งมากเกินไป สมการทั่วไปของ EMA แสดงในสมการ (2.10)

$$\hat{\omega}_t = \alpha \omega_t + (1 - \alpha) \hat{\omega}_{t-1} \quad (2.10)$$

เมื่อ α เป็นค่าปรับแต่งของตัวกรอง และ $\hat{\omega}_t$ คือสัญญาณหลังกรอง นอกจากนี้ยังกำหนด deadband เพื่อตัด drift ขนาดเล็กขณะรถเข็นหยุดนิ่ง ดังสมการ (2.11)

$$\omega'_t = \begin{cases} 0, & |\hat{\omega}_t| < \tau \\ \hat{\omega}_t, & |\hat{\omega}_t| \geq \tau \end{cases} \quad (2.11)$$

โดย τ คือ threshold ของ deadband ที่ต้องปรับให้เหมาะกับอุปกรณ์และตำแหน่งติดตั้ง การนำสูตรเหล่านี้ไปใช้จึงต้องประเมินร่วมกับการทดสอบจริง เพราะ IMU มี drift, noise, การสั่นจากพื้น และความคลาดเคลื่อนจากการติดตั้ง

ตารางที่ 2.4: ตัวอย่าง feature จาก IMU ที่ใช้กับการวิเคราะห์การเคลื่อนไหว

Feature	ข้อมูลที่ใช้	ความหมายเชิงพฤติกรรม
Acceleration magnitude	a_x, a_y, a_z	ระดับแรงสั่นหรือการเปลี่ยนแปลงการเคลื่อนไหวโดยรวม
Mean และ standard deviation	ค่าใน sliding window	ความเข้มข้นและความแปรปรวนของการเคลื่อนไหว
Zero-crossing	สัญญาณหลังกรองในแต่ละแกน	ความถี่ของการสลับทิศทางหรือการสั่น
Velocity และ distance	Gyroscope และรัศมีล้อ	ความเร็วและระยะทางโดยประมาณของรถเข็น

2.7 Machine Learning สำหรับจำแนกพฤติกรรมหรือความเสี่ยง

Machine Learning ในระบบดูแลผู้สูงอายุทำหน้าที่แปลงข้อมูล sensor ที่มีความถี่สูง และอ่านยากให้เป็นสถานะหรือความเสี่ยงที่ผู้ดูแลเข้าใจได้ง่ายขึ้น ตัวอย่างเช่น แปลงข้อมูล IMU เป็นสถานะกำลังเคลื่อนที่ หยุดนิ่ง หรือมีการสั่นผิดปกติ หรือรวมข้อมูลการเคลื่อนไหวกับข้อมูลสัญญาณชีพเพื่อช่วยจัดลำดับความสำคัญของเหตุการณ์ ขั้นตอนสำคัญคือ feature engineering ซึ่งนำข้อมูลดิบมาสรุปเป็นตัวแปร เช่น mean, standard deviation, magnitude, zero-crossing, velocity, distance, heart rate และ RR interval

XGBoost เป็นอัลกอริทึม gradient tree boosting ที่ได้รับความนิยมเพราะรองรับ feature หลายชนิด จัดการความสัมพันธ์ไม่เชิงเส้นได้ดี และให้ผลลัพธ์ที่ดีความเชิง feature importance ได้ระดับหนึ่ง โมเดลประกอบด้วย decision tree หลายต้น โดยผลทำนายเกิดจากผลรวมของต้นไม้ทั้งหมด ดังสมการ (2.12) [35]

$$\hat{y}_i = \sum_{m=1}^M f_m(\mathbf{x}_i), \quad f_m \in \mathcal{F} \quad (2.12)$$

เมื่อ $\mathbf{x}_i$ คือ feature vector ของตัวอย่างที่ i , f_m คือ decision tree ต้นที่ m และ M คือจำนวนต้นไม้ทั้งหมด โดยการฝึกโมเดลจะพยายามลด loss ของข้อมูลฝึกพร้อมกับ regularization เพื่อลด overfitting ดังสมการ (2.13)

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{m=1}^M \Omega(f_m) \quad (2.13)$$

หากโมเดลให้ผลลัพธ์เป็น score ของแต่ละคลาส สามารถแปลงเป็นความน่าจะเป็นด้วย softmax ได้ตามสมการ (2.14)

$$p(y = c|\mathbf{x}) = \frac{\exp(z_c)}{\sum_{q=1}^C \exp(z_q)} \quad (2.14)$$

การประเมินโมเดล classification ควรพิจารณามากกว่า accuracy เพียงค่าเดียว เพราะข้อมูลพฤติกรรมหรือความเสี่ยงอาจไม่สมดุลระหว่างคลาส ค่า precision, recall และ F1-score จึงใช้ประกอบกันตามสมการ (2.15) [36]

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}, \quad Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN} \quad (2.15)$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (2.16)$$

สำหรับระบบดูแล ความผิดพลาดของโมเดลมีความหมายเชิงความปลอดภัยต่างกัน

False negative อาจทำให้ระบบพลาดเหตุการณ์สำคัญ ขณะที่ false positive อาจเพิ่มภาระผู้ดูแลจาก alert ที่ไม่จำเป็น ดังนั้นการใช้โมเดลควรถูกออกแบบให้เป็นตัวช่วยคัดกรองเบื้องต้นและให้มนุษย์ตรวจสอบ ไม่ใช่กลไกตัดสินใจแทนผู้ดูแลทั้งหมด

2.8 Retrieval-Augmented Generation และ Retrieval Logic

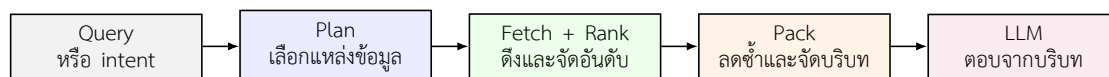
Retrieval-Augmented Generation (RAG) เป็นแนวคิดที่ให้โมเดลภาษาใช้ข้อมูลจากแหล่งความรู้ภายนอกก่อนสร้างคำตอบ แทนที่จะตอบจากความรู้ที่อยู่ในพารามิเตอร์ของโมเดลเพียงอย่างเดียว ขั้นตอนทั่วไปคือรับ query, ค้นเอกสารหรือข้อมูลที่เกี่ยวข้อง, จัดแพ็คเกจบริบทให้กระชับและส่งบริบทนั้นให้ LLM สังเคราะห์คำตอบ วิธีนี้ช่วยลดปัญหาการตอบจากข้อมูลที่ไม่อยู่ในบริบทและทำให้ระบบสามารถอ้างอิงข้อมูลที่เปลี่ยนแปลงได้ง่ายขึ้น [37]

ในเชิงทฤษฎี RAG-Sequence สามารถเขียนแบบย่อได้ตามสมการ (2.17) โดย x คือ query, z คือเอกสารหรือบริบทที่ retrieve ได้ และ y คือคำตอบที่โมเดลสร้าง

$$p_{\text{RAG-Seq}}(y|x) \approx \sum_{z \in \text{top-}k(p_\eta(\cdot|x))} p_\eta(z|x) \prod_{i=1}^N p_\theta(y_i|x, z, y_{<i}) \quad (2.17)$$

เมื่อ $p_\eta(z|x)$ คือความน่าจะเป็นของ retriever ที่เลือกบริบท z จาก query และ p_θ คือ generator หรือ LLM ที่สร้างคำตอบจาก query และบริบทที่ได้ จุดสำคัญคือคุณภาพของคำตอบไม่ได้ขึ้นกับ LLM เพียงอย่างเดียว แต่ขึ้นกับการเลือกบริบท การจัดลำดับข้อมูล และการตัดข้อมูลที่ไม่จำเป็นออกจาก context window ด้วย

ในระบบที่เกี่ยวข้องกับข้อมูลผู้ป่วยหรือผู้พักอาศัย retrieval logic ควรมีข้อจำกัดชัดเจน



รูปที่ 2.6: แนวคิด Retrieval Logic ในระบบ RAG

เช่น ดึงเฉพาะข้อมูลที่มีผู้ใช้มีสิทธิ์เห็น ดึงเฉพาะช่วงเวลาที่เกี่ยวข้อง และไม่ส่งข้อมูลดิบมากเกินไปเข้าไปใน context window การแยกขั้นตอน retrieval ออกจากขั้นตอน generation ยังช่วยให้ตรวจสอบได้ว่าคำตอบหนึ่งเกิดจากบริบทใด ซึ่งสำคัญต่อความโปร่งใสและการแก้ไขข้อผิดพลาด

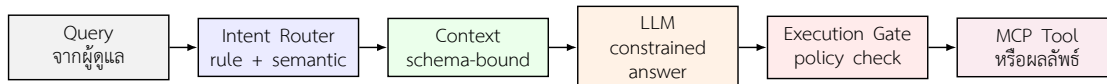
2.9 Neuro-symbolic AI และ AI Pipeline แบบ Rule + Model

Neuro-symbolic AI คือแนวคิดที่รวมความสามารถของ neural model เช่น deep learning หรือ LLM เข้ากับ symbolic component เช่น กฎ ตรรกะ schema knowledge graph หรือ policy ที่ตรวจสอบได้ งานวิจัยในสาขานี้มองว่าการใช้โมเดลเรียนรู้เพียงอย่างเดียวอาจมีข้อจำกัดด้านความโปร่งใสและการควบคุม ขณะที่ระบบ rule-based เพียงอย่างเดียวอาจไม่ยืดหยุ่นพอสำหรับภาษาธรรมชาติและบริบทที่ซับซ้อน การผสมทั้งสองส่วนจึงมีเป้าหมายเพื่อให้ระบบเข้าใจข้อมูลไม่เป็นโครงสร้างได้ดีขึ้น แต่ยังคงอยู่ภายใต้ข้อจำกัดที่ตรวจสอบได้ [38], [39]

สำหรับระบบดูแลที่ใช้ LLM และเครื่องมือภายนอก แนวคิดนี้สามารถนำมาใช้ในลักษณะ neuro-symbolic-inspired ได้ กล่าวคือ neural component ทำหน้าที่เข้าใจภาษา สรุบบริบท และสร้างคำตอบภาษาธรรมชาติ ส่วน symbolic component ทำหน้าที่กำหนด intent schema, policy, role permission, tool contract และ execution gate เพื่อป้องกันคำสั่งที่ไม่ผ่านเงื่อนไข ระบบลักษณะนี้ไม่ควร claim ว่าเป็น neuro-symbolic system เต็มรูปแบบ แต่สามารถอธิบายได้ว่าได้รับแรงบันดาลใจจากการแยกหน้าที่ระหว่าง model reasoning และ rule-based control

ตารางที่ 2.5: การแบ่งบทบาทระหว่าง neural และ symbolic component

ส่วนประกอบ	ตัวอย่างหน้าที่	ประโยชน์ต่อระบบดูแล
Neural	เข้าใจภาษาธรรมชาติ จัดกลุ่มความหมาย สังเคราะห์คำตอบ	รองรับคำถามหรือคำสั่งของผู้ดูแลที่มีรูปแบบหลากหลาย
Symbolic	intent rule, schema, policy, permission, tool contract	จำกัดขอบเขตการเข้าถึงข้อมูลและการสั่งงานให้ตรวจสอบได้
Execution gate	ตรวจ payload ก่อนเรียกใช้ tool หรือ MCP server	ลดความเสี่ยงจาก hallucination หรือคำสั่งที่ไม่ตรงสิทธิ์



รูปที่ 2.7: แนวคิด AI pipeline แบบผสม rule และ model สำหรับระบบที่เรียกใช้เครื่องมือ

Model Context Protocol (MCP) เป็นแนวทางหนึ่งในการกำหนดรูปแบบการเชื่อมต่อระหว่าง AI client กับเครื่องมือหรือ resource ภายนอก โดยช่วยให้ tool มี contract ชัดเจนและสามารถควบคุมการเข้าถึงผ่าน server ได้ [30], [40] เมื่อใช้ร่วมกับ execution gate ระบบสามารถแยกขั้นตอน “เสนอคำสั่ง” ออกจาก “อนุญาตให้สั่งงานจริง” ได้ชัดเจน ซึ่งเหมาะกับงานดูแลผู้สูงอายุ ที่การทำงานผิดพลาดอาจส่งผลกระทบต่อความปลอดภัยหรือความเป็นส่วนตัวของข้อมูล

2.10 งานวิจัยที่เกี่ยวข้อง

งานวิจัยด้าน indoor localization ด้วย BLE RSSI และ fingerprinting แสดงให้เห็นว่า RSSI สามารถใช้เป็นสัญญาณประกอบการระบุตำแหน่งภายในอาคารได้ แต่มีความผันผวนสูงและต้องอาศัยวิธีการลดผลกระทบจากสภาพแวดล้อม งานของ Bahl และ Padmanabhan ในระบบ RADAR เป็นตัวอย่างสำคัญของการใช้ radio map และ fingerprinting เพื่อหาตำแหน่งภายในอาคาร [32] ส่วนงานวิเคราะห์ RSSI ในอาคารชี้ให้เห็นว่าค่า RSSI ได้รับผลกระทบจากตำแหน่งอุปกรณ์ สิ่งกีดขวาง และการสะท้อนของสัญญาณ [31] ทำให้การใช้ KNN และ fallback เช่น max-RSSI มีความเหมาะสมกับ prototype ที่ต้องรองรับความไม่แน่นอนของข้อมูล

งานวิจัยด้าน Human Activity Recognition (HAR) จาก wearable sensor และ IMU มักใช้กระบวนการแบ่งสัญญาณเป็น window แล้วคำนวณ feature เชิงสถิติ เช่น mean, variance, energy, magnitude และ zero-crossing ก่อนป้อนเข้าสู่โมเดลจำแนกพฤติกรรม [34] แนวทางนี้สอดคล้องกับการใช้ข้อมูลจาก accelerometer และ gyroscope บนรถเข็น เพราะข้อมูลดิบอ่านตีความยาก แต่เมื่อแปลงเป็น feature แล้วสามารถใช้จำแนกสถานะการเคลื่อนไหวหรือความผิดปกติได้ชัดเจนขึ้น

งานด้าน wearable health และ PPG สนับสนุนการใช้ sensor สวมใส่เพื่อติดตามข้อมูลชีวสัญญาณ เช่น heart rate, RR interval, PPG และข้อมูลการเคลื่อนไหว แต่ข้อจำกัดสำคัญคือคุณภาพสัญญาณขึ้นกับตำแหน่งสวมใส่ การเคลื่อนไหวของร่างกาย และ algorithm ของผู้ผลิต [18], [19] ดังนั้นการใช้ข้อมูล wearable ในระบบดูแลจึงควรเป็นข้อมูลประกอบการเฝ้าระวังและแจ้ง

เดือน ไม่ใช่การวินิจฉัยทางการแพทย์

งานด้าน Retrieval-Augmented Generation แสดงให้เห็นว่าการดึงบริบทจากแหล่งข้อมูลภายนอกก่อนให้โมเดลภาษาตอบ สามารถเพิ่มความเฉพาะเจาะจงและความสอดคล้องของคำตอบในงานที่ต้องใช้ความรู้จำนวนมาก [37] สำหรับระบบดูแล แนวคิดนี้นำมาประยุกต์ได้กับการดึงข้อมูลผู้พักอาศัย ข้อมูล sensor ล่าสุด ประวัติการแจ้งเตือน หรือ policy ก่อนสร้างคำตอบให้ผู้ดูแล

งานด้าน neuro-symbolic และ tool-use AI เน้นการผสมความสามารถของโมเดลกับกฎหรือ schema ที่ตรวจสอบได้ เพื่อเพิ่มความน่าเชื่อถือและลดความเสี่ยงจากการตัดสินใจแบบ black box [38], [39] เมื่อระบบต้องเรียกใช้เครื่องมือผ่าน MCP หรือ API ภายนอก การมี execution gate และ tool contract จึงเป็นกลไกสำคัญที่ทำให้การใช้ AI อยู่ในขอบเขตที่ปลอดภัยกว่า LLM ที่สามารถสั่งงานได้โดยไม่มีขั้นตรวจสอบ

ตารางที่ 2.6: การจัดกลุ่มงานวิจัยที่เกี่ยวข้องกับระบบต้นแบบ

กลุ่มงานวิจัย	ประเด็นสำคัญ	การเชื่อมโยงกับงานนี้
Indoor localization / BLE RSSI / KNN	RSSI ผันผวน ต้องใช้ fingerprinting หรือวิธี fallback	ใช้ KNN และ max-RSSI เพื่อประเมินพื้นที่ในอาคาร
Human activity recognition / IMU	แปลงสัญญาณเป็น feature ก่อนจำแนกพฤติกรรม	ใช้ feature จาก IMU และข้อมูลการเคลื่อนไหวของรถเข็น
Wearable health / PPG	วัด HR, RR interval และ PPG แต่ต้องระวัง noise	ใช้ข้อมูล wearable เป็นบริบทประกอบ ไม่ใช่ผลวินิจฉัย
RAG	ดึงบริบทก่อนให้ LLM ตอบ	ลดการตอบนอกข้อมูลและควบคุม context ของผู้พักอาศัย
Neuro-symbolic / tool-use AI	ใช้ rule, schema และ policy ควบคู่กับโมเดล	ใช้ execution gate และ MCP tool contract ก่อนสั่งงานจริง

2.11 ช่องว่างของงานวิจัยเดิม

จากการทบทวนทฤษฎีและงานวิจัยที่เกี่ยวข้อง พบว่างานจำนวนมากมักพัฒนาเฉพาะส่วนใดส่วนหนึ่งของระบบ เช่น งาน indoor localization ที่เน้นตำแหน่งจาก RSSI, งาน HAR ที่เน้นจำแนกกิจกรรมจาก IMU, งาน wearable health ที่เน้นสัญญาณชีพ, งาน dashboard ที่เน้นการแสดงผล หรือระบบ AI ที่เน้นการตอบคำถามและการเรียกใช้เครื่องมือ แต่ในบริบทการดูแลผู้สูงอายุ หรือผู้ไร้รถเข็น ผู้ดูแลต้องการภาพรวมที่เชื่อมข้อมูลเหล่านี้เข้าด้วยกันมากกว่าผลลัพธ์แยกส่วน

ช่องว่างสำคัญคือการรวม wheelchair sensor, wearable sensor, mobile gateway, indoor localization, dashboard ตามบทบาทผู้ใช้ และ AI/MCP tool pipeline ไว้ใน prototype เดียว โดยยังคงขอบเขตว่าเป็นระบบต้นแบบเพื่อสนับสนุนการดูแล ไม่ใช่เครื่องมือแพทย์ที่ผ่านการรับรอง การออกแบบลักษณะนี้ต้องจัดสมดุลระหว่างความสามารถด้าน sensor, ความเสถียรของการส่งข้อมูล, ความชัดเจนของ dashboard, และความปลอดภัยของ AI ที่อาจเข้าถึงข้อมูลหรือเครื่องมือในระบบ

ดังนั้น บทถัดไปจะนำแนวคิดจากบทนี้ไปออกแบบระบบในรูปแบบปัญหาไปสู่แนวทางแก้ไข โดยอธิบายว่าระบบถูกปรับจากข้อจำกัดของ prototype เดิมอย่างไร เหตุใด mobile gateway จึงกลายเป็นแกนหลักของการส่งข้อมูล และ AI pipeline ถูกแยกเป็นชั้นต่าง ๆ เพื่อควบคุม latency, context, hallucination และการส่งงานผ่าน MCP ได้อย่างไร

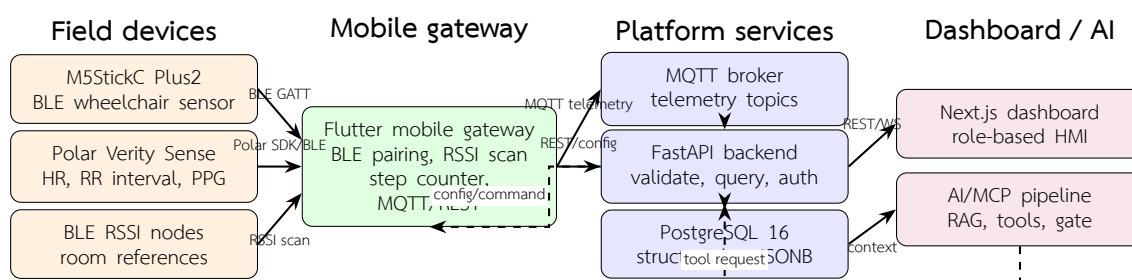
บทที่ 3

วิธีการออกแบบและพัฒนาระบบ

3.1 ภาพรวมของระบบ

ระบบต้นแบบ WheelSense ถูกออกแบบเป็นระบบดูแลและติดตามสถานะผู้สูงอายุและผู้สูงอายุในสภาพแวดล้อมภายในอาคาร โดยรวมข้อมูลจากเซ็นเซอร์รถเข็น อุปกรณ์สวมใส่ สมาร์ทโฟนที่ทำหน้าที่เป็น mobile gateway เซิร์ฟเวอร์ส่วนกลาง แดชบอร์ดตามบทบาทผู้ใช้ และระบบผู้ช่วย AI/MCP ให้อยู่ในโครงสร้างเดียวกัน จุดสำคัญของการออกแบบคือการลดภาระของผู้ดูแลที่ต้องตรวจข้อมูลจากหลายแหล่งด้วยตนเอง และทำให้ข้อมูลตำแหน่ง การเคลื่อนไหว สัญญาณชีพ และเหตุการณ์ที่เกี่ยวข้องถูกส่งเข้าสู่ backend เดียวกันก่อนแสดงผลหรือใช้ประกอบการวิเคราะห์

แนวทางของระบบไม่ได้เริ่มจากการนำไปใช้งานจริงในสถานดูแลผู้สูงอายุทันที แต่ปรับรูปแบบจากการทดสอบภายในพื้นที่ทดลองของโครงการ เมื่อพบข้อจำกัดของการติดตั้งโหนด การเชื่อมต่อ Wi-Fi การรับค่า RSSI และความล่าช้าของการเรียกใช้ AI จึงปรับสถาปัตยกรรมให้สมาร์ทโฟนเป็น gateway หลักแทนการให้อุปกรณ์ภาคสนามส่งข้อมูลขึ้นระบบโดยตรงทั้งหมด สถาปัตยกรรมที่ใช้ในเล่มนี้จึงแยกภาระงานออกเป็น 4 ส่วน คือ field devices, mobile gateway, platform services และ dashboard/AI ดังรูปที่ 3.1



รูปที่ 3.1: สถาปัตยกรรมโดยรวมของระบบ WheelSense จากอุปกรณ์ภาคสนามไปยัง mobile gateway, platform services, dashboard และ AI

ข้อมูลในระบบเริ่มจาก M5StickC Plus2 ที่ติดตั้งบนรถเข็นและทำหน้าที่เป็น BLE sensor สำหรับอ่านข้อมูล IMU และแบตเตอรี่ อุปกรณ์ Polar Verity Sense ส่งข้อมูลสัญญาณชีพผ่าน Polar SDK และสมาร์ทโฟนอ่าน RSSI จาก BLE node สำหรับใช้ประกอบการระบุตำแหน่ง

ภายในอาคาร ข้อมูลเหล่านี้ถูกรวมที่แอป Flutter gateway ก่อนส่งเข้าสู่ MQTT broker หรือ REST API ตามชนิดข้อมูล จากนั้น FastAPI จะตรวจสอบรูปแบบข้อมูล บันทึกลง PostgreSQL และเปิดให้ dashboard หรือ AI pipeline ดึงไปใช้งานต่อ

3.2 การเชื่อมต่ออุปกรณ์ภาคสนามและ Mobile Gateway

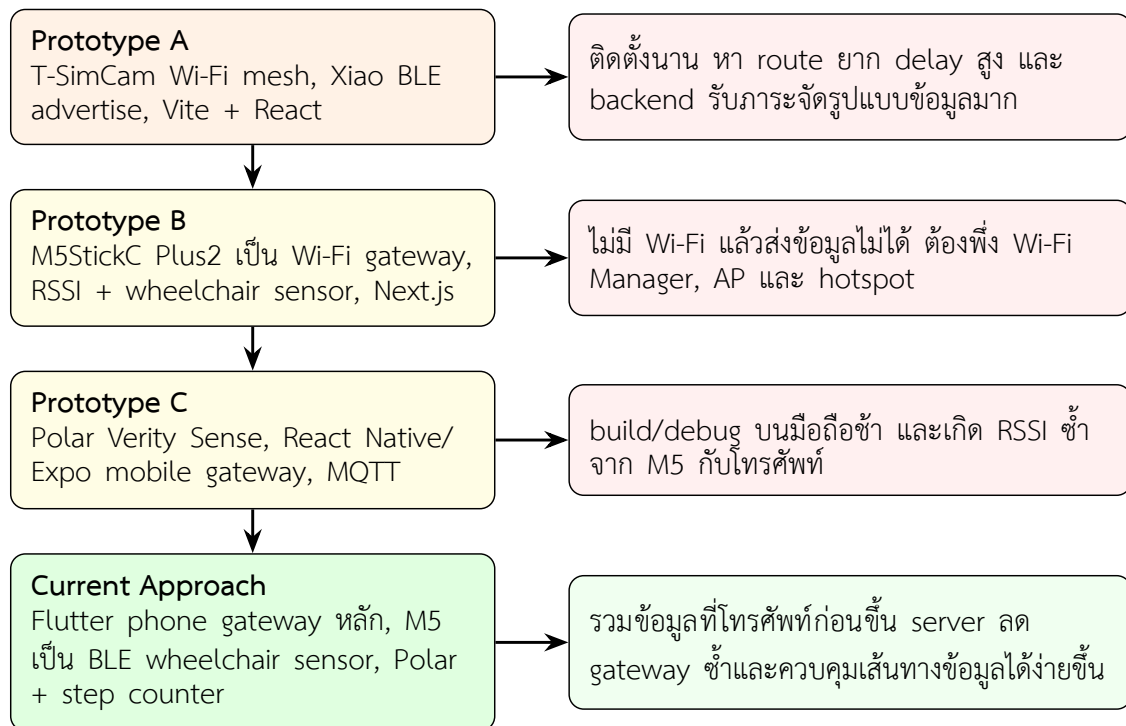
การออกแบบส่วนเชื่อมต่ออุปกรณ์ภาคสนามผ่านการทดลองหลายรูปแบบเพื่อหาจุดสมดุลระหว่างความง่ายในการติดตั้ง ความต่อเนื่องของข้อมูล และภาระของ backend ในระยะแรกของแนวคิด ผู้พัฒนาเคยใช้โครงสร้าง Wi-Fi mesh โดยให้ Node T-SimCam รับข้อมูลจาก Xiao nRF52840 ที่โฆษณาสัญญาณ BLE แล้วส่งต่อข้อมูลเป็นทอด ๆ ไปยัง gateway node ที่มีอินเทอร์เน็ต ขณะเดียวกันฝั่งเว็บเริ่มจาก Vite ร่วมกับ React เพราะตอบสนองเร็วและพัฒนา UI ได้ง่าย แต่เมื่อระบบต้องรับ telemetry หลายชนิดและมี workflow เพิ่มขึ้น backend ต้องรับภาระจัดรูปแบบข้อมูลจำนวนมากเกินไป ข้อจำกัดที่พบในระยะนี้คือการติดตั้ง mesh ใช้เวลานาน การหา route ในพื้นที่จริงทำได้ยากเมื่อสภาพแวดล้อมเปลี่ยน และ delay สูงเกินกว่าจะใช้ติดตามสถานะผู้ใช้แบบต่อเนื่องได้ดี

แนวทางต่อมาคือย้ายภาระ gateway บางส่วนมาไว้ที่ M5StickC Plus2 โดยให้อุปกรณ์เชื่อมต่อ Wi-Fi และส่งค่า RSSI จาก node รวมถึงข้อมูลเซ็นเซอร์ของรถเข็นขึ้น server เอง เมื่อไม่มี Wi-Fi จึงเพิ่ม Wi-Fi Manager ให้ M5StickC เปิดเป็น access point เพื่อให้โทรศัพท์เข้าไปตั้งค่าและใช้ hotspot ได้ ส่วน node ที่ต้องส่งภาพหรือข้อมูลจากอาคารยังใช้เครือข่ายของพื้นที่และสื่อสารผ่าน MQTT/HTTP เพื่อส่งสถานะ คำสั่งควบคุม และภาพจากกล้องเข้าสู่ server ในช่วงนี้ frontend ถูกย้ายไปใช้ Next.js เพื่อรองรับ routing, server-side data และโครงสร้าง dashboard ที่ซับซ้อนขึ้น วิธีนี้ลดการพึ่งพา mesh ลง แต่ยังมีข้อจำกัดสำคัญคือเมื่อเครือข่าย Wi-Fi ไม่พร้อมใช้งาน การส่งข้อมูลจะหยุดชะงัก

เมื่อเพิ่ม Polar Verity Sense เข้ามาในระบบ ข้อมูล heart rate, RR interval และ PPG ต้องอ่านผ่าน Polar SDK ทำให้โทรศัพท์เริ่มมีบทบาทสำคัญมากขึ้น มีการพัฒนา mobile app ต้นแบบด้วย React Native และ Expo ซึ่งสามารถเชื่อมต่อ sensor และส่งข้อมูลผ่าน MQTT ได้ แต่การ build และ debug บนอุปกรณ์จริงใช้เวลานาน อีกทั้ง M5StickC Plus2 ยังทำหน้าที่ทั้งรับ RSSI จาก node และส่งข้อมูลผ่าน Wi-Fi ทำให้ตำแหน่งของผู้ใช้มีข้อมูลซ้ำจากสองแหล่ง คือ RSSI ที่

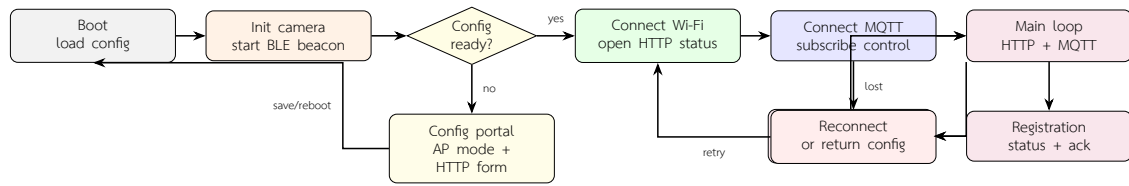
M5StickC รับได้และ RSSI ที่โทรศัพท์รับได้ การจัดการข้อมูลจึงเริ่มซับซ้อนและเสี่ยงต่อการตีความตำแหน่งผิด

จากข้อจำกัดดังกล่าว ระบบที่ใช้ในต้นแบบปัจจุบันจึงให้สมาร์ทโฟนเป็น mobile gateway หลักเพียงจุดเดียว โดย M5StickC Plus2 ลดบทบาทเหลือเป็น BLE wheelchair sensor ที่ส่งข้อมูลไปยังโทรศัพท์โดยตรง Polar Verity Sense เชื่อมต่อกับโทรศัพท์ผ่าน Polar SDK และโทรศัพท์ยังอ่านจำนวนก้าวจาก step counter เพื่อรองรับผู้ใช้งานที่ไม่ได้อยู่บนรถเข็นตลอดเวลา การจัดวางนี้ทำให้ข้อมูลภาคสนามถูกรวมและจัดรูปแบบก่อนขึ้น server ลดปัญหาการมีข้อมูล RSSI ซ้ำจากหลายแหล่ง และช่วยให้ผู้ดูแลตรวจสอบสถานะการเชื่อมต่อจากหน้า gateway ได้ง่ายขึ้น



รูปที่ 3.2: ลำดับการปรับแนวทางการเชื่อมต่ออุปกรณ์ภาคสนามจากการทดลองภายในพื้นที่ทดสอบ

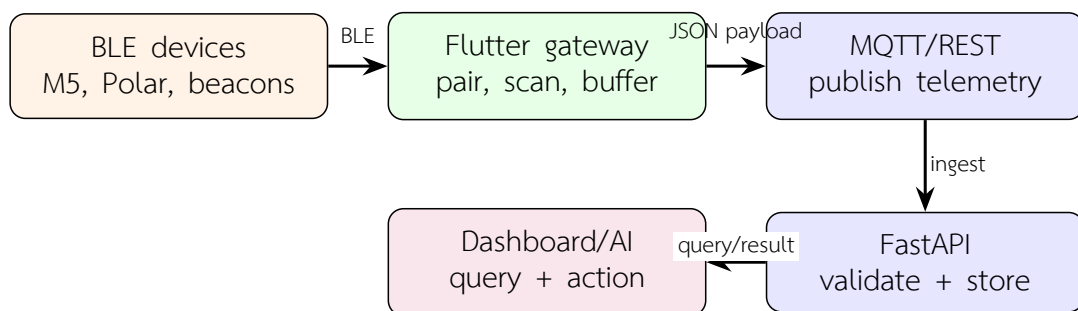
ในต้นแบบ Node T-SimCam ถูกใช้เป็นตัวอย่างของ camera/BLE node ภาคสนาม โค้ดเฟิร์มแวร์ใน `firmware/Node_Tsimcam` เริ่มจากโหลด configuration จากหน่วยความจำ ตั้งค่า camera และเริ่ม BLE beacon ด้วยชื่อ node ที่กำหนดไว้ หากยังไม่ได้ตั้งค่าหรือผู้ใช้กดปุ่ม boot ระหว่างเริ่มต้น อุปกรณ์จะเข้าสู่ config portal ผ่าน Wi-Fi access point เพื่อรับค่า device id, node id, Wi-Fi และ MQTT server แต่ถ้ามี configuration แล้วจะเชื่อมต่อ Wi-Fi เปิดหน้า HTTP status/config และเชื่อมต่อ MQTT broker จากนั้น loop หลักจะรับคำสั่งผ่าน MQTT/HTTP ส่ง registration, status, ack และภาพถ่ายหรือ frame ตามคำสั่งที่ได้รับ



รูปที่ 3.3: วงรอบเฟิร์มแวร์ Node T-SimCam สำหรับ config portal, HTTP status, MQTT status/ack/control และการส่งภาพแบบ capture/stream

ข้อจำกัดที่พบจากแนวทางนี้คือการเชื่อมต่อ Wi-Fi มีช่วงเวลารอที่ทำให้การเริ่มต้นช้าได้ และการ stream ภาพผ่าน HTTP client ต่อเนื่องอาจกินรอบการทำงานของ node มากกว่าการส่ง telemetry ขนาดเล็ก จึงเหมาะเป็นองค์ประกอบเสริมของต้นแบบมากกว่าการเป็น gateway หลักของระบบสุดท้าย การตัดสินใจย้าย gateway หลักไปไว้ที่โทรศัพท์จึงลดทั้งความซับซ้อนของ route และลดปัญหาความล่าช้าจาก node ที่ต้องทำงานหลายหน้าที่พร้อมกัน

จากลำดับดังกล่าว สถาปัตยกรรมปัจจุบันจึงย้ายจุดรวมข้อมูลมาอยู่ที่โทรศัพท์เป็นหลัก การส่งข้อมูลไม่ให้ไหลจากหลาย gateway พร้อมกันช่วยลดความซับซ้อนของ RSSI และทำให้ backend รับข้อมูลจากแหล่งเดียวที่จัดรูปแบบแล้ว ก่อนจะส่งต่อเข้าสู่ MQTT, REST API, ฐานข้อมูล และ dashboard ดังแสดงในรูปที่ 3.4



รูปที่ 3.4: ขั้นตอนการเชื่อมต่อรวมระหว่างอุปกรณ์ภาคสนาม mobile gateway และบริการส่วนกลาง

หากมีคำสั่ง config หรือ command ย้อนกลับจากฝั่ง dashboard หรือ AI ระบบจะไม่ส่งคำสั่งลงอุปกรณ์โดยตรงจาก LLM แต่ให้ backend และ mobile gateway ตรวจสอบเงื่อนไขก่อน แล้วจึงเขียนค่าไปยัง BLE characteristic หรือ config ที่เกี่ยวข้อง วิธีนี้ทำให้เส้นทางคำสั่งแยกจากเส้นทาง telemetry หลักและลดความเสี่ยงจากคำสั่งที่เกิดจากบรีบทไม่ครบถ้วน

3.3 M5StickC Plus2 BLE Wheelchair Sensor

M5StickC Plus2 ถูกใช้เป็นเซ็นเซอร์ประจำรถเข็นเพื่อตรวจจับการเคลื่อนไหวของตัวรถ ไม่ได้ทำหน้าที่เป็น gateway หลักในสถาปัตยกรรมปัจจุบัน เฟิร์มแวร์ถูกออกแบบให้ทำงานเป็น BLE peripheral พร้อม custom GATT service เพื่อให้แอป Flutter บนโทรศัพท์เชื่อมต่อ อ่านค่า และรับ notification ได้อย่างต่อเนื่อง ข้อมูลที่ส่งออกประกอบด้วยค่า IMU สถานะแบตเตอรี่ metadata ของอุปกรณ์ และค่าที่ประมวลผลแล้ว เช่น ความเร็ว ระยะทาง และความเร่งในช่วงเวลาปัจจุบัน

เมื่อเริ่มทำงาน เฟิร์มแวร์จะโหลด configuration ที่บันทึกไว้ เช่น ชื่ออุปกรณ์ ค่า room หรือค่าจับคู่ จากนั้นเริ่ม BLE advertising เพื่อประกาศ service UUID และรอ mobile gateway เชื่อมต่อ หลังเชื่อมต่อแล้วอุปกรณ์จะอ่านค่า IMU และแบตเตอรี่ตามรอบเวลา ประมวลผล gyro offset เพื่อลด bias ของแกนหมุน ใช้ exponential moving average (EMA) เพื่อลดสัญญาณรบกวน และใช้ deadband เพื่อตัดค่าการเคลื่อนไหวขนาดเล็กที่อาจเกิดจาก noise ก่อนคำนวณความเร็ว ระยะทาง และความเร่งของรถเข็น

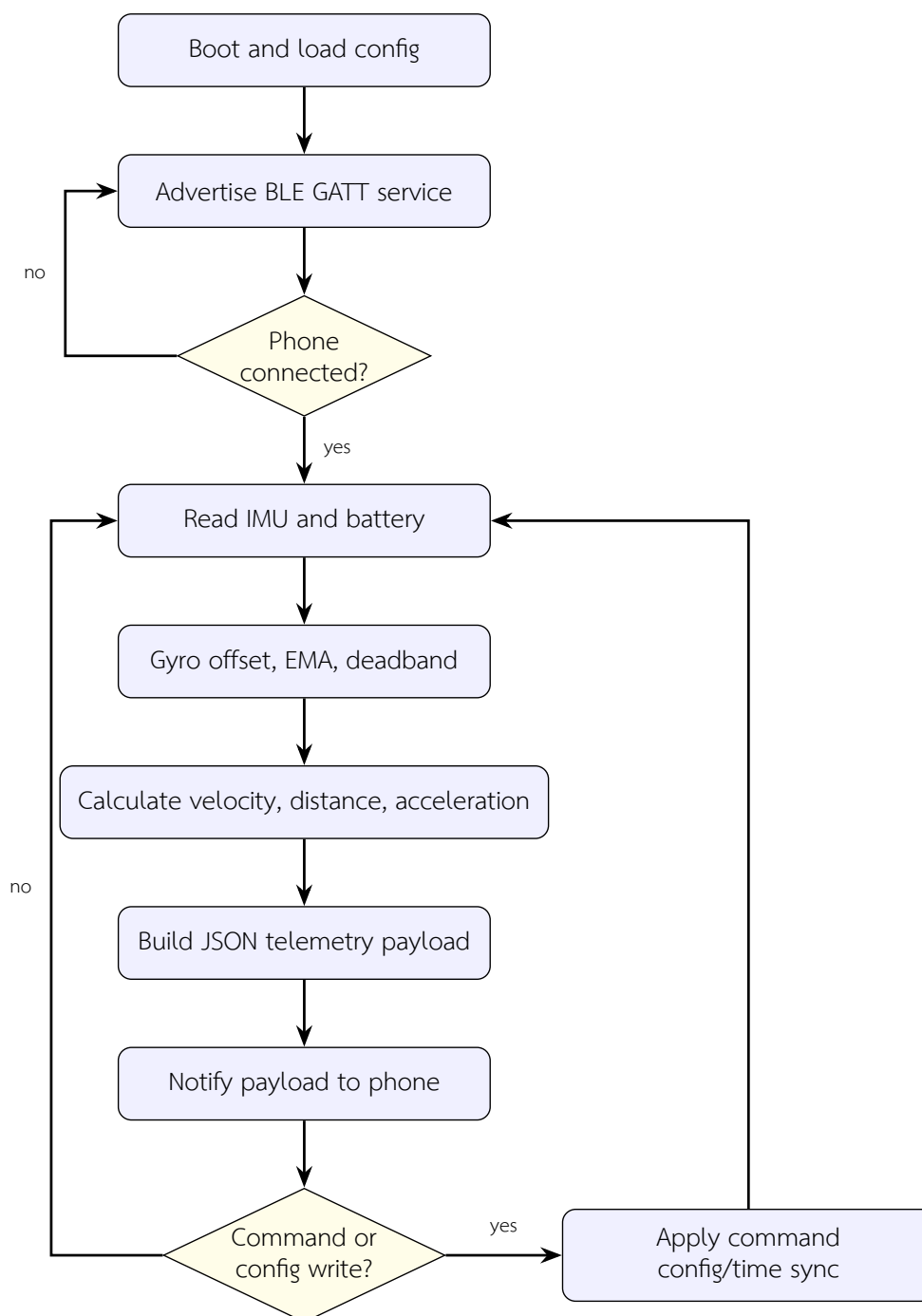


รูปที่ 3.5: ตัวอย่างตำแหน่งติดตั้ง M5StickC Plus2 บนรถเข็นสำหรับอ่านค่าการเคลื่อนไหวและส่งข้อมูลผ่าน BLE ไปยังโทรศัพท์

ตารางที่ 3.1: หน้าที่หลักของเฟิร์มแวร์ M5StickC Plus2 ในระบบปัจจุบัน

ส่วนการทำงาน	รายละเอียดการออกแบบ
Configuration	โหลดชื่ออุปกรณ์ สถานะจับคู่ ค่า room และค่าที่จำเป็นต่อการเชื่อมต่อ BLE
BLE GATT service	advertise service และเปิด characteristic สำหรับ telemetry, command, config และ time sync
Sensor sampling	อ่านค่า acceleration, angular velocity และ battery จากฮาร์ดแวร์ภายใน M5StickC Plus2
Signal processing	ชดเชย gyro offset ใช้ EMA และ deadband เพื่อลด noise ก่อนคำนวณ motion
Telemetry notify	ส่ง JSON payload ผ่าน BLE notification ไปยังโทรศัพท์ที่จับคู่ไว้
Command receiver	รับคำสั่ง start, stop, identify, pair, calibrate, reset distance, config และ time sync จากโทรศัพท์

โครงสร้างข้อมูลที่ส่งไปยังโทรศัพท์ถูกกำหนดให้เป็น JSON เพื่อให้ mobile gateway สามารถ normalize และส่งต่อเข้าระบบได้โดยไม่ต้องผูกกับรูปแบบ binary ของเฟิร์มแวร์โดยตรง ตัวอย่างกลุ่มข้อมูลหลัก ได้แก่ timestamp, device identifier, ค่า IMU, motion summary, battery level และสถานะการเชื่อมต่อ การกำหนดข้อมูลเป็น schema ลักษณะนี้ช่วยให้ backend ตรวจสอบและจัดเก็บข้อมูลใน PostgreSQL/JSONB ได้ง่ายขึ้น



รูปที่ 3.6: แผนผังวงจรการทำงานของเฟิร์มแวร์ M5StickC Plus2 แบบ BLE wheelchair sensor

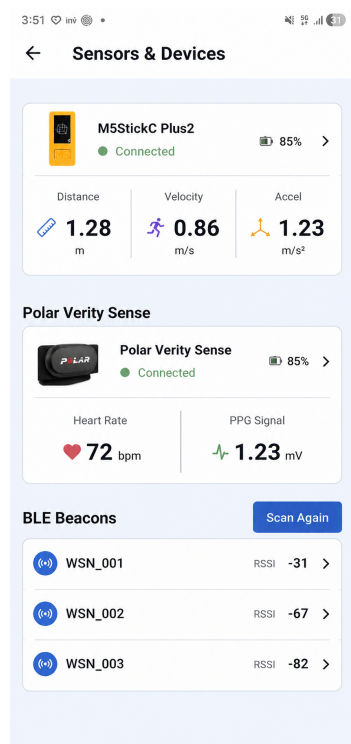
3.4 Polar Verity Sense และ Flutter Mobile Gateway

อุปกรณ์ Polar Verity Sense ถูกเพิ่มเข้ามาเพื่อให้ระบบมีข้อมูลสัญญาณชีพประกอบกับข้อมูลการเคลื่อนไหวของรถเข็น การอ่านข้อมูลจาก Polar ไม่ได้ใช้เพียง BLE GATT มาตรฐานทั่วไปเท่านั้น แต่ใช้ Polar SDK เพื่ออ่านข้อมูลที่ละเอียดขึ้น เช่น heart rate, RR interval และ PPG stream จากนั้นแอป Flutter จะรวมข้อมูลของ Polar เข้ากับข้อมูลจาก M5StickC Plus2 และข้อมูล step counter ของโทรศัพท์

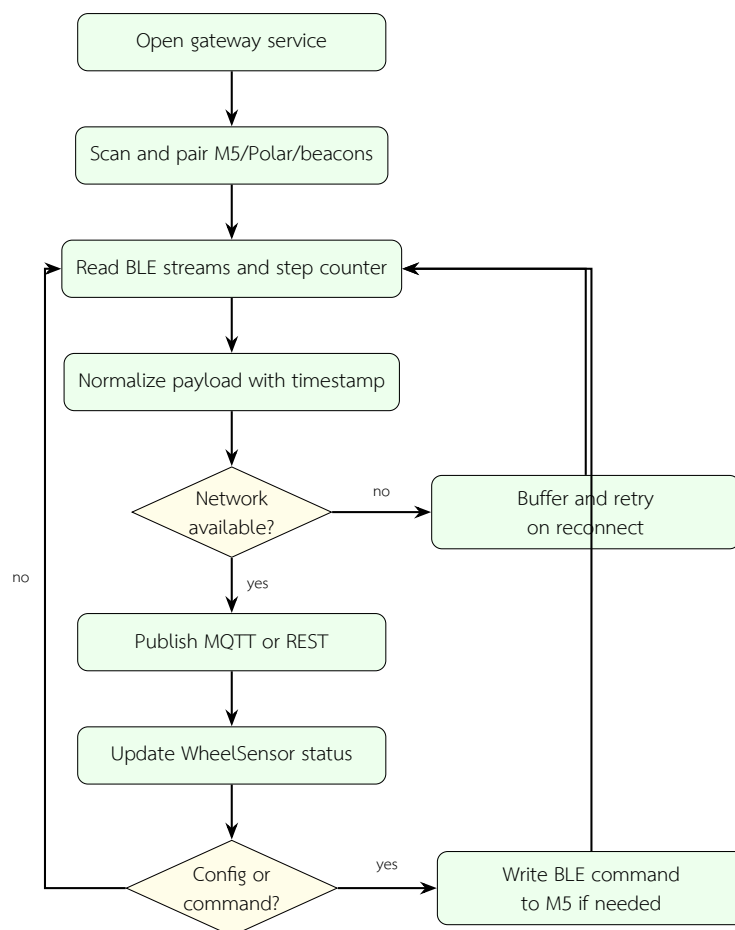


รูปที่ 3.7: การใช้งาน Polar Verity Sense ร่วมกับสมาร์ทโฟนที่ทำหน้าที่เป็น mobile gateway

แอป Flutter ทำหน้าที่เป็น gateway หลักในระบบ เพราะเป็นจุดที่มีทั้ง BLE สำหรับเชื่อมต่ออุปกรณ์ภาคสนามและอินเทอร์เน็ตผ่าน Wi-Fi หรือเครือข่ายเซลลูลาร์ หน้าการทำงานที่เกี่ยวข้องกับการติดตั้งภาคสนามคือหน้า /WheelSensor ซึ่งใช้ตรวจสอบสถานะการเชื่อมต่อ BLE, Polar, M5StickC Plus2, MQTT และการส่งข้อมูลขึ้น server จุดนี้ช่วยให้ผู้ดูแลหรือผู้พัฒนาสามารถแยกปัญหาได้ว่าข้อมูลหายจากระดับ pairing, sensor stream, network publish หรือ backend ingest



รูปที่ 3.8: หน้าจอ /WheelSensor สำหรับตรวจสอบสถานะ BLE, Polar, M5StickC Plus2 และการส่งข้อมูลขึ้น server



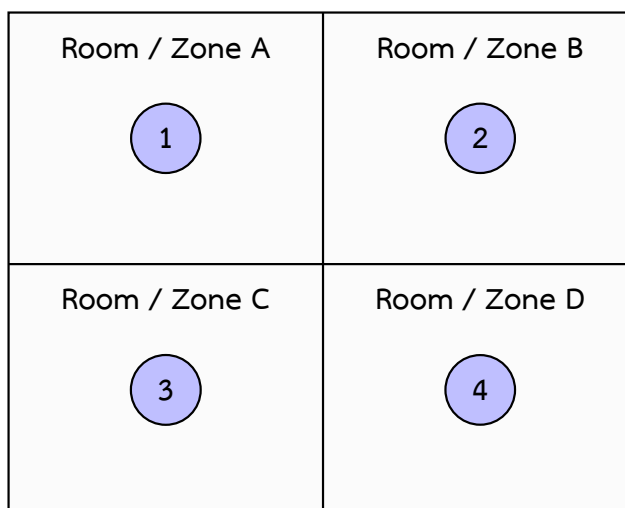
รูปที่ 3.9: วงรอบการทำงานของ Flutter mobile gateway สำหรับรวมข้อมูล Polar, M5StickC Plus2, RSSI และ step counter

ตารางที่ 3.2: กลุ่มข้อมูลที่ mobile gateway รวมก่อนส่งเข้าสู่ระบบ

แหล่งข้อมูล	ข้อมูลที่รับเข้า gateway	การส่งต่อเข้าสู่ระบบ
M5StickC Plus2	IMU, motion summary, battery, device metadata	รวมเป็น telemetry ของรถเข็น และ publish ผ่าน MQTT/REST
Polar Verity Sense	HR, RR interval, PPG และสถานะแบตเตอรี่ตามที่ SDK รองรับ	normalize เป็นข้อมูลสัญญาณชีพของผู้ใช้และส่งพร้อม timestamp
Phone sensor	step counter และสถานะเครื่องข่ายของเครื่อง	ใช้ประกอบบริบทผู้ใช้ที่ไม่ได้อยู่บนรถเข็นตลอดเวลา
BLE RSSI nodes	รายการ beacon ที่พบและค่า RSSI	ส่งเป็นข้อมูลตำแหน่งให้ backend ประเมินห้องหรือโซน

3.5 Indoor Localization

การระบุตำแหน่งภายในอาคารของระบบใช้ค่า RSSI จาก BLE node ที่ติดตั้งตามพื้นที่อ้างอิง โดย mobile gateway สแกนสัญญาณจาก node รอบตัวและส่งรายการค่า RSSI พร้อม timestamp ไปยัง server สำหรับประเมินห้องหรือโซนของผู้ใช้ การออกแบบในบทนี้ไม่ลงรายละเอียดสมการซ้ำจากบทที่ 2 แต่เน้นกระบวนการนำข้อมูล RSSI เข้าระบบและการเลือกกลยุทธ์ที่ backend รองรับ



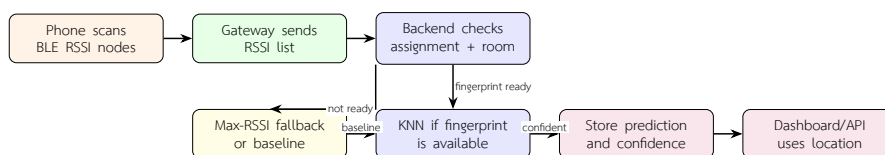
เส้นแบ่งแทนฉากกั้นหรือขอบเขตพื้นที่ทดลอง
หมายเลข 1-4 คือ BLE node ที่วางบริเวณกึ่งกลางของแต่ละ zone

รูปที่ 3.10: แผนผังพื้นที่ทดลอง 4 zone พร้อมตำแหน่ง BLE node กึ่งกลางแต่ละพื้นที่สำหรับเก็บค่า RSSI



รูปที่ 3.11: ตัวอย่างตำแหน่ง BLE node ในพื้นที่ทดลองสำหรับเก็บค่า RSSI ภายในอาคาร

ในเชิง implementation ระบบรองรับทั้งการเทียบกับ fingerprint ด้วย KNN และการใช้ max-RSSI เป็น baseline หรือ fallback เมื่อข้อมูล fingerprint ยังไม่พร้อมหรือความมั่นใจต่ำ การใช้ KNN ต้องมีชุดข้อมูลอ้างอิงของแต่ละตำแหน่ง เช่น room, beacon alias, ค่า RSSI และข้อมูลผู้ใช้หรืออุปกรณ์ที่จับคู่ไว้ ส่วน max-RSSI เลือกตำแหน่งจาก beacon ที่มีสัญญาณแรงที่สุดในรอบปัจจุบัน จึงใช้งานง่ายกว่าแต่ไวต่อการสะท้อนและการเปลี่ยนแปลงของสภาพแวดล้อม



รูปที่ 3.12: Flowchart การส่งข้อมูล RSSI และการประเมินตำแหน่งด้วย KNN หรือ max-RSSI fallback

การเขียนข้อมูลตำแหน่งกลับเข้าสู่ระบบทำเป็นข้อมูลที่ตรวจสอบย้อนกลับได้ โดยเก็บทั้ง raw RSSI reading และผล room prediction แยกกัน เพื่อให้สามารถวิเคราะห์ภายหลังได้ว่าความคลาดเคลื่อนเกิดจากสัญญาณต้นทาง การตั้งค่า fingerprint หรือการเลือกกลยุทธ์ของ localization ในช่วงเวลานั้น การแยก raw data กับ prediction ยังช่วยให้ Chapter 4 สามารถประเมินความแม่นยำของ KNN และ fallback ได้อย่างเป็นระบบ

3.6 Server, API และ Database

ส่วน server ของ WheelSense ประกอบด้วย MQTT broker, FastAPI backend, PostgreSQL 16 และบริการที่เกี่ยวข้องกับ dashboard/AI โดยออกแบบให้รองรับข้อมูลทั้งแบบ structured และ semi-structured ข้อมูล structured เช่น users, patients, devices,

workspaces, rooms และ assignments ใช้สำหรับกำหนดความสัมพันธ์หลักของระบบ ส่วนข้อมูล semi-structured เช่น telemetry payload, AI trace, device configuration และ event metadata เก็บใน JSONB เพื่อรองรับ payload จากอุปกรณ์ที่อาจเปลี่ยนรายละเอียดระหว่างการทดลอง

MQTT broker รับ telemetry จาก mobile gateway และ topic ที่เกี่ยวกับอุปกรณ์ ภาคสนาม ส่วน FastAPI รับทั้งข้อมูลจาก gateway, dashboard และ AI/MCP pipeline ผ่าน REST endpoint หรือ internal service การออกแบบ backend ใช้ async I/O เพื่อให้รับข้อมูลจากหลายอุปกรณ์พร้อมกันได้โดยไม่บล็อกการทำงานของ API หลัก เมื่อข้อมูลเข้ามา server จะทำขั้นตอน ingest, validate, store, query และส่งต่อไปยัง dashboard หรือ AI ตามลำดับ

ตารางที่ 3.3: บริการหลักของ platform services ในระบบ WheelSense

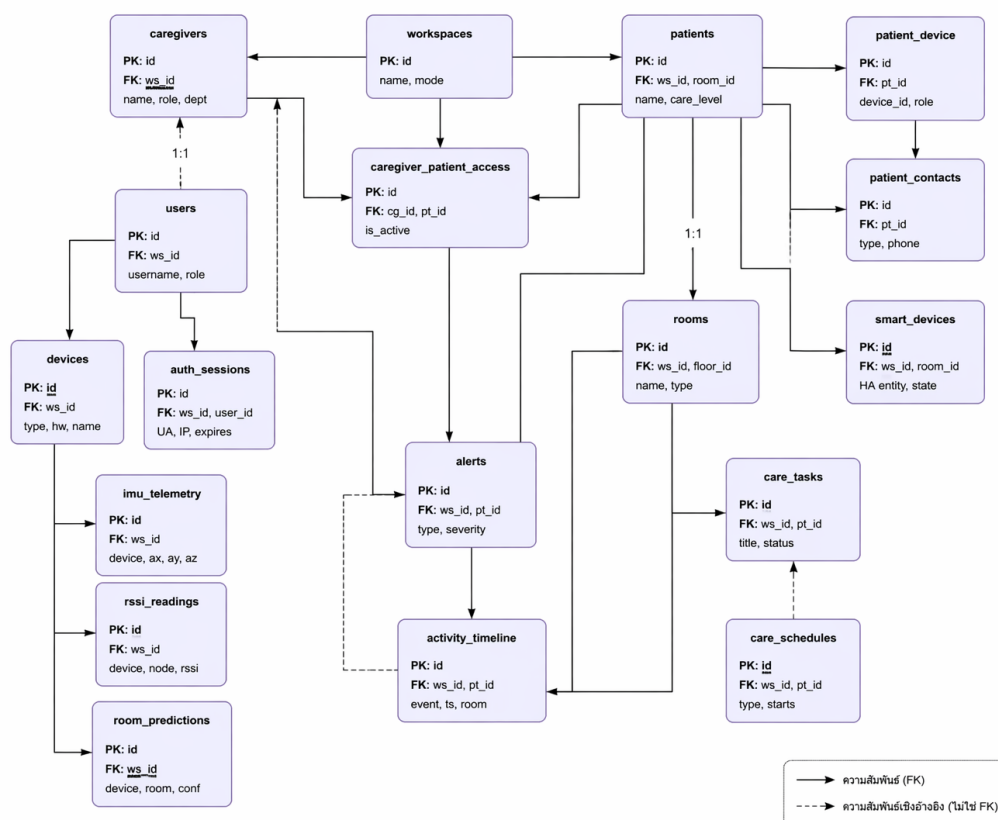
บริการ	เทคโนโลยี	หน้าที่ในระบบ
MQTT broker	Eclipse Mosquitto	รับ telemetry จาก mobile gateway และใช้ส่ง config หรือสถานะบางส่วนกลับไปยังอุปกรณ์
Backend API	FastAPI	ตรวจสอบข้อมูล จัดการสิทธิ์ ประมวลผล request จาก dashboard/gateway และเปิด API ให้ AI ใช้ข้อมูล
Database	PostgreSQL 16 + JSONB	เก็บ schema หลักของระบบ telemetry, RSSI, room prediction, AI trace และข้อมูลตั้งค่า
Frontend	Next.js	แสดงผล dashboard ตามบทบาทผู้ใช้และเรียกข้อมูลจาก backend ผ่าน API
AI/MCP runtime	Agent runtime + MCP server	จัดเตรียมบริบท ใช้ LLM แบบมีข้อจำกัด และควบคุมการเรียก tool ผ่าน execution gate



รูปที่ 3.13: ลำดับการประมวลผลข้อมูลฝั่ง server ตั้งแต่ ingest, validate, store, query ไปจนถึง dashboard และ AI

schema หลักของระบบสามารถแบ่งได้เป็น 5 กลุ่ม ได้แก่ กลุ่ม workspace และสิทธิ์ผู้ใช้ กลุ่ม patient และ assignment กลุ่ม device และ room binding กลุ่ม telemetry/

localization และกลุ่ม AI/runtime log ตารางเช่น users, patients, devices, workspaces, rooms, imu_telemetry, rssi_readings, room_predictions, mobile_device_telemetry, pipeline_events และ behavioral_states ช่วยให้ข้อมูลจากหลายส่วนถูกอ้างอิงกลับไปยังผู้ใช้ อุปกรณ์ และพื้นที่ได้อย่างเป็นระบบ



แผนภาพความสัมพันธ์ของเอนทิตี (ER Diagram) เบื้องต้นในฐานข้อมูล

รูปที่ 3.14: แผนภาพ schema ฐานข้อมูลที่ใช้เชื่อม workspaces, users, patients, devices และข้อมูล telemetry

ตารางที่ 3.4: ตัวอย่างกลุ่มข้อมูลที่จัดเก็บใน PostgreSQL และ JSONB

กลุ่มข้อมูล	ตารางหรือ entity ตัวอย่าง	เหตุผลในการออกแบบ
User/workspace	users, workspaces, roles	แยกขอบเขตองค์กรและสิทธิ์การเข้าถึงข้อมูลของผู้ใช้แต่ละบทบาท
Patient/device	patients, devices, assignments	เชื่อมผู้พักอาศัยกับอุปกรณ์ รถเข็น และห้องที่เกี่ยวข้อง
Telemetry	imu_telemetry, mobile_device_telemetry	เก็บข้อมูลจาก M5, Polar, phone และ gateway พร้อม timestamp
Localization	rss_i_readings, room_predictions, training data	แยก raw RSSI ออกจากผลทำนาย เพื่อประเมินย้อนหลังได้
AI runtime	pipeline_events, behavioral_states	เก็บ trace ของแต่ละ layer และ behavioral context สำหรับตรวจสอบการทำงานของ AI

3.6.1 ระบบควบคุมการเข้าถึงและความปลอดภัยของเซสชัน

การรักษาความปลอดภัยของข้อมูลในส่วนบริการส่วนกลางใช้สถาปัตยกรรมสิทธิ์การเข้าถึงแบบกำหนดสิทธิ์ตามบทบาทและบริบทของ workspace (Workspace-Context Access Control) โดยผู้ใช้งานที่ผ่านการยืนยันตัวตนจะได้รับ JSON Web Token (JWT) สำหรับระบุ `user_id`, `workspace_id`, `role` และ `session id` ของการล็อกอิน ข้อมูล token ฝั่งเว็บถูกส่งผ่าน HTTP-only cookie ชื่อ `ws_token` เพื่อลดความเสี่ยงจากการอ่าน token ด้วยสคริปต์ฝั่งไคลเอนต์ และให้ backend สามารถตรวจสอบสิทธิ์ทุกครั้งก่อนอ่านหรือแก้ไขข้อมูลผู้พักอาศัย อุปกรณ์ ห้อง task และ telemetry

นอกจากการตรวจ JWT ตามปกติ ระบบยังเก็บสถานะเซสชันไว้ในตาราง `auth_sessions` เพื่อให้สามารถถอดถอนสิทธิ์ย้อนหลังได้ หากผู้ดูแลระบบต้องการยุติ session ที่ยังใช้งานอยู่ backend จะทำเครื่องหมาย session นั้นเป็น `revoked` หรือ `expired` แล้วปฏิเสธคำขอถัดไปที่อ้างอิง session เดิม วิธีนี้ทำให้การออกจากระบบ การเปลี่ยนรหัสผ่าน หรือการพบพฤติกรรมผิดปกติไม่ต้องรอให้ JWT หมดอายุเอง

ส่วน MCP ใช้ token แยกจาก session หลักและเก็บ metadata ไว้ในตาราง `mcp_tokens` โดย token แต่ละรายการผูกกับ `auth_session_id`, `workspace`, `role` และขอบเขตสิทธิ์ที่อนุญาตให้ใช้กับเครื่องมือ MCP ได้ การถอดถอน token รายตัวทำผ่าน endpoint `DELETE /`

`api/mcp/token/{token_id}` และการถอดถอน token ทั้งหมดของผู้ใช้ผ่าน `POST /api/mcp/tokens/revoke-all` เมื่อ token ถูก revoke แล้ว execution gate จะไม่อนุญาตให้เรียก tool แม้ payload ของ LLM จะมีรูปแบบถูกต้องก็ตาม กลไกนี้จึงเป็นชั้นป้องกันร่วมระหว่าง authentication, role scope, workspace scope และ MCP tool policy

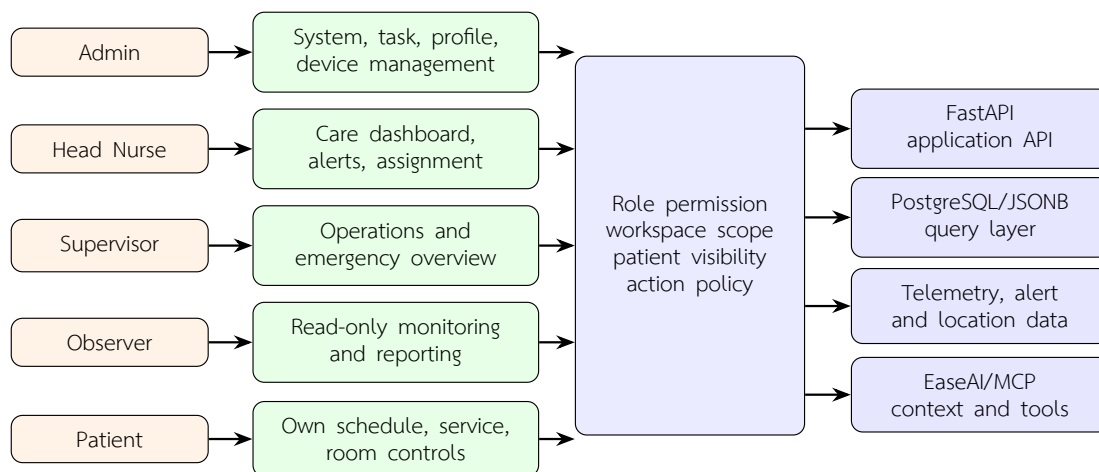
3.7 Dashboard และ HMI ตามบทบาทผู้ใช้

Dashboard ของ WheelSense พัฒนาด้วย Next.js เพื่อเป็นหน้าจอหลักสำหรับผู้ดูแลและผู้ใช้งานตามบทบาท การออกแบบ HMI ไม่ได้ให้ผู้ใช้ทุกกลุ่มเห็นข้อมูลเท่ากัน แต่กำหนด permission และมุมมองตามหน้าที่จริงในระบบ เพื่อให้ผู้ใช้แต่ละกลุ่มเห็นเฉพาะข้อมูลที่จำเป็นต่อการตัดสินใจหรือติดตามสถานะ ขณะที่ mobile app ทำหน้าที่เป็น gateway ภาคนามสำหรับ pairing sensor และส่งข้อมูล ไม่ใช่ dashboard หลักของทุกบทบาท

ตารางที่ 3.5: บทบาทผู้ใช้ หน้าจอหลัก และหน้าที่ที่รองรับบน dashboard

บทบาท	หน้าจอหลักที่ใช้	งานที่ทำได้ในระบบ
Admin	Dashboard overview, task operations, facility, floorplans, devices, personnel, alerts, messages, support, settings, EaseAI	จัดการ workspace, ผู้ใช้, อุปกรณ์, ห้อง, floorplan, task, profile ของผู้พักอาศัย และ staff รวมถึงตรวจสอบสถานะของ platform และเรียกใช้ AI assistant ภายใต้อิทธิพลของผู้ดูแลระบบ
Head Nurse	Dashboard overview, alerts, patient profile, task operations, devices, floorplans, personnel, staff, messages, support	ติดตามผู้พักอาศัยหลายคน ตรวจสอบ alert จัดลำดับความสำคัญ มอบหมายงาน และดูข้อมูลสัญญาณชีพ/ตำแหน่งในระดับที่เกี่ยวข้องกับการดูแล
Supervisor	Dashboard overview, emergency, task operations, floorplans, personnel, prescriptions, patient profile, messages, support	ตรวจภาพรวมงานและเหตุการณ์ในพื้นที่ ติดตาม emergency/task ที่ต้องส่งต่อ และตรวจสอบข้อมูลผู้พักอาศัยหรือบุคลากรในขอบเขตการกำกับดูแล
Observer	Dashboard overview, alerts, floorplans, prescriptions, task operations, patient profile, personnel, messages, support	เฝ้าดูสถานะและเหตุการณ์แบบจำกัดสิทธิ์ ใช้ข้อมูลเพื่อสังเกตการณ์หรือรายงานต่อผู้ดูแล โดยไม่แก้ไขข้อมูลสำคัญของระบบ
Patient	Personal dashboard, schedule, pharmacy, room controls, services, messages, support, settings	ดูข้อมูลส่วนตัวของตนเอง เช่น care plan, schedule, medication/service request, room control และช่องทางขอความช่วยเหลือหรือสื่อสารกับผู้ดูแล

ตารางบทบาทข้างต้นถูกแปลงเป็น role flow เพื่อให้เห็นว่าแต่ละกลุ่มผู้ใช้ไม่ได้เข้าถึงข้อมูลชุดเดียวกันทั้งหมด แต่ทุกคำขอจะผ่าน permission และ visibility layer ก่อนเรียกข้อมูลจาก FastAPI หรือ telemetry/AI context รูปที่ 3.15 แสดงลำดับการไหลของหน้าจอจากบทบาทผู้ใช้ไปยัง service หลักของระบบ โดยไม่รวม mobile gateway เพราะ mobile gateway อยู่ในเส้นทางรับส่งข้อมูล sensor ที่อธิบายไว้ก่อนหน้านี้



รูปที่ 3.15: HMI role flow ของระบบ WheelSense แสดงการผ่านสิทธิ์ บริบท workspace และ service ที่ dashboard เรียกใช้งาน

การใช้งานที่ซ้ำกันหลายบทบาท เช่น task operations, patient profile, staff profile, messages และ support ถูกควบคุมด้วย permission เดียวกันในระดับ backend แต่แสดงข้อมูลและ action ต่างกันตามบทบาท Admin เห็นภาพรวมมากที่สุดและใช้ตรวจสอบความครบถ้วนของ workspace ได้ จึงสามารถจัดการผู้ใช้ อุปกรณ์ ห้อง floorplan task profile การตั้งค่าระบบ และช่องทาง EaseAI ได้ครบ ส่วนบทบาทอื่นจะใช้ชุดหน้าจอบางส่วนร่วมกัน แต่ถูกลดข้อมูลหรือปิด action ที่ไม่เกี่ยวข้องกับหน้าที่ของตนเอง

บทบาท Head Nurse ใช้ dashboard เพื่อดูแลผู้ป่วยพักอาศัยหลายคนในเวลาเดียวกัน หน้าจอหลักจึงเน้น alert, patient profile, task operations, device status และ floorplan เพื่อให้สามารถแยกได้ว่าเหตุการณ์มาจากสภาพของผู้พักอาศัย ตำแหน่งในอาคาร หรืออุปกรณ์ที่ขาดการเชื่อมต่อ เมื่อผู้ใช้ยืนยัน alert หรือมอบหมายงาน ระบบจะบันทึก action พร้อมผู้กระทำและเวลาเพื่อให้ตรวจสอบย้อนหลังได้ในเวอร์ถัดไป

บทบาท Supervisor ใช้ตรวจภาพรวมเชิงปฏิบัติการ เช่น งานที่ค้าง เหตุการณ์ฉุกเฉิน บุคลากรที่เกี่ยวข้อง และรายการที่ต้องส่งต่อระหว่างทีม จึงเห็นข้อมูลมากกว่า Observer แต่ยังไม่ควรแก้ไขค่าคอนฟิกหลักของระบบหรือสิทธิ์ผู้ใช้ระดับ workspace ขณะที่ Observer ถูกออกแบบเป็นมุมมองอ่านอย่างเดียวสำหรับเฝ้าดูสถานะและจัดทำรายงาน จึงมีสิทธิ์เข้าถึงข้อมูลสถานะและเหตุการณ์ แต่ไม่สามารถสร้างคำสั่งที่เปลี่ยนสถานะสำคัญของระบบได้

บทบาท Patient ถูกจำกัดให้เห็นเฉพาะข้อมูลของตนเอง เช่น care plan, schedule, medication, room control, service request และช่องทางติดต่อผู้ดูแล ไม่แสดงรายชื่อผู้พักอาศัย

คนอื่นหรือข้อมูลบุคลากรที่ไม่เกี่ยวข้อง การออกแบบนี้ช่วยลดความซับซ้อนของหน้าจอสำหรับผู้ใช้งานบุคคลและลดความเสี่ยงจากการเปิดเผยข้อมูลข้ามบุคคล

EaseAI เป็น assistant ที่เปิดจาก dashboard เพื่อช่วยค้นข้อมูล สรุปบริบท หรือเตรียม action candidate ตามสิทธิ์ของบทบาทผู้ใช้ ระบบจะส่ง role, workspace และบริบทที่จำเป็นเข้าสู่ AI pipeline ในหัวข้อ 3.8 หากคำตอบเป็นการสรุปข้อมูลจะตอบกลับในหน้าต่างสนทนา แต่ถ้าเกี่ยวข้องกับการสั่งงานจริงจะต้องผ่าน execution gate และเงื่อนไขการยืนยันก่อนเสมอ ภาพหน้าจอจริงของแต่ละบทบาทและตัวอย่าง EaseAI ถูกนำไปแสดงในบทที่ 4 เพื่อใช้ประกอบผลการประเมิน HMI และ SUS Score แทนการวางซ้ำในบทสถาปัตยกรรม

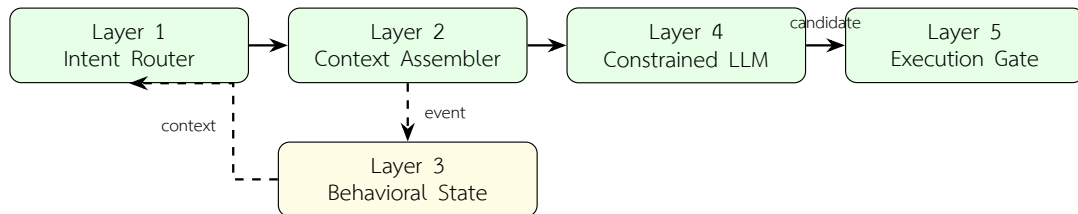
3.8 ระบบ AI และ MCP Tool Pipeline

ส่วน AI ของระบบถูกออกแบบใหม่จากปัญหาของแนวทางเดิมที่ให้ LLM และ MCP agent จัดการงานส่วนใหญ่ในเส้นทางเดียว เมื่อระบบมี tool หลายกลุ่ม เช่น การค้นข้อมูลผู้พักอาศัย การอ่านข้อมูล sensor การสรุปสถานะ และการเตรียม action ให้ผู้ดูแลตรวจสอบ prompt ต้องบรรจุคำอธิบาย tool, schema, policy และบริบทจำนวนมาก ส่งผลให้เกิด token overhead สูง และ latency เพิ่มขึ้นมาก จากบันทึกการพัฒนาระบบเดิมพบว่าโมเดลขนาด 7B บางรอบใช้เวลาประมาณ 30–45 นาที หรือมากกว่านั้นเมื่อต้อง reasoning กับ tool หลายขั้นตอน แม้ภายหลังจะลดขนาดโมเดลและปรับ prompt แล้ว เวลาตอบสนองยังไม่เหมาะกับ dashboard ที่ผู้ดูแลต้องรอผลในรอบการทำงานสั้นกว่าเดิม

หลังการทดสอบรอบแรก ระบบถูกปรับเป็นแนวทางที่แยก logic เชิงสัญลักษณ์ออกจาก การสังเคราะห์ภาษาของ LLM โดยให้ deterministic rule, schema, policy และ tool contract ทำงานร่วมกับ LLM แทนการปล่อยให้โมเดลตัดสินใจทุกขั้นตอน แนวทางนี้เป็นการออกแบบแบบ neuro-symbolic-inspired กล่าวคือใช้ LLM สำหรับความเข้าใจเชิงภาษาและการเรียบเรียงคำตอบ ส่วนเงื่อนไขเชิงระบบและการสั่งงานจริงยังถูกควบคุมด้วยกฎและ schema ที่ตรวจสอบได้ จึงไม่กล่าวอ้างว่าเป็นระบบ neuro-symbolic เต็มรูปแบบ แต่ใช้หลักการดังกล่าวเพื่อลด hallucination และลดคำสั่งที่อยู่นอกขอบเขต

pipeline หลักของระบบแบ่งเป็น 5 layer ได้แก่ Layer 1 Intent Router, Layer 2 Context Assembler, Layer 3 Behavioral State, Layer 4 Constrained LLM และ Layer 5

Execution Gate อย่างไรก็ตาม Layer 3 ไม่ได้อยู่ใน execute path หลักแบบ synchronous เสมอไป เส้นทางหลักของคำสั่งคือ Layer 1 → Layer 2 → Layer 4 → Layer 5 ส่วน Layer 3 ทำหน้าที่เตรียมและเก็บ behavioral context จากข้อมูลผู้ใช้หรือเหตุการณ์ เพื่อให้ระบบนำบริบทนี้กลับมาใช้ในคำถามหรือการประเมินรอบถัดไปได้

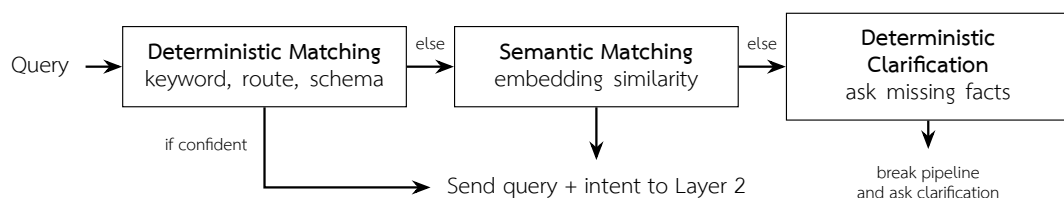


รูปที่ 3.16: โครงสร้าง AI pipeline 5 layer โดย Layer 3 ทำหน้าที่เตรียม behavioral context นอกเส้นทาง execute หลัก

Layer 1 Intent Router ทำหน้าที่รับคำสั่งจากผู้ใช้แล้วจัดกลุ่มเจตนาเบื้องต้นก่อนให้ LLM ทำงาน ระบบเริ่มจาก deterministic matching เช่น keyword, route, role และ schema ที่คาดหวัง หาก confidence เพียงพอจะส่ง query พร้อม intent ไปยัง Layer 2 ได้ทันที หากยังไม่ชัดเจนจึงใช้ semantic matching ร่วมกับ Sentence-BERT เพื่อคำนวณเวกเตอร์ฝังตัว (embedding vector) ของคำถามเปรียบเทียบกับกลุ่มคำถามอ้างอิง โดยคำนวณความใกล้เคียงด้วยสมการความคล้ายคลึงโคไซน์ (Cosine Similarity) ดังสมการที่ (3.1)

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \|v\|} \quad (3.1)$$

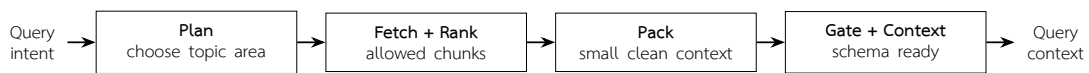
เมื่อ u และ v คือเวกเตอร์ฝังตัวของคำถามผู้ใช้และคำถามต้นแบบตามลำดับ หากความคล้ายคลึงมีค่าเกินเกณฑ์ที่กำหนดจะส่งข้อมูลไปยัง Layer 2 และหากยังขาดข้อมูลสำคัญ เช่น ไม่ระบุผู้พักอาศัย ห้อง หรือ action ที่ต้องการ ระบบจะหยุด pipeline แล้วถามกลับแทนการเดาคำสั่ง



รูปที่ 3.17: Layer 1 Intent Router สำหรับจับเจตนาแบบ deterministic ก่อนใช้ semantic matching และ clarification

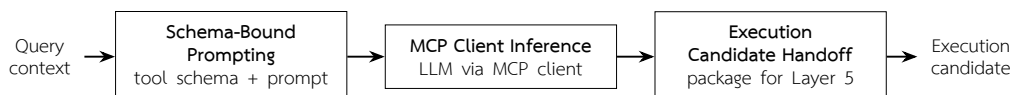
Layer 2 Context Assembler รับ intent จาก Layer 1 แล้วเลือกบริบทที่เกี่ยวข้อง

กับคำสั่ง เช่น ข้อมูลผู้พักอาศัย สถานะ sensor ล่าสุด ประวัติ telemetry สรุปเชิงพฤติกรรม หรือ เอกสารใน RAG ขั้นตอนนี้ไม่ได้ดึงข้อมูลทั้งหมดเข้า prompt แต่เริ่มจากการวางแผนว่าคำสั่งต้องใช้หัวข้อใด จากนั้น fetch และ rank เฉพาะ chunk ที่อนุญาตตาม role และ policy ก่อนแพ็กข้อมูลในรูปแบบแพลตฟอร์มที่สะอาดและมีขนาดควบคุมได้ ข้อมูลที่ประกอบขึ้นจะอยู่ภายใต้ data contract ชื่อ `ValidatedContextPackage` ซึ่งประกอบด้วย `correlation_id`, `actor`, `intent`, `required_facts`, `system_state_snapshot` และ `policy_tags` เพื่อให้แต่ละคำตอบตรวจสอบย้อนกลับได้ว่าใครเป็นผู้ถาม อยู่ใน workspace ใด ตั้งใจเรียกงานประเภทใด และใช้บริบทระบบชุดใดก่อนส่งเข้า Layer 4



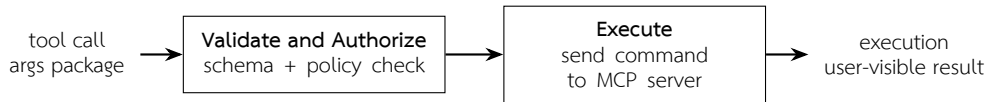
รูปที่ 3.18: Layer 2 Context Assembler สำหรับดึงและจัดแพ็กบริบทก่อนส่งเข้า LLM

บริบทที่ถูกส่งต่อจะประกอบด้วยข้อมูลที่จำเป็นต่อคำตอบหรือการสั่งงานเท่านั้น เช่น actor facts ของผู้ใช้, patient id ที่ระบุในคำถาม, required facts ที่ยังขาด, ค่า telemetry ล่าสุด ที่ถูกสรุปเป็น `system_state_snapshot`, สิทธิ์ของผู้ใช้ และข้อความอ้างอิงที่ถูกดึงจาก retrieval logic วิธีนี้ทำให้ Layer 4 ไม่ต้องรับ prompt ขนาดใหญ่ทุกครั้ง และลดโอกาสที่ LLM จะเห็นข้อมูลที่ไม่เกี่ยวข้องจนสรุปผิด



รูปที่ 3.19: Layer 4 Constrained LLM สำหรับสังเคราะห์คำตอบหรือ execution candidate ภายใต้ schema ที่กำหนด

Layer 4 Constrained LLM เป็นขั้นที่ใช้ LLM (เช่น gemma4:e2b หรือ โมเดลในระดับเดียวกัน) สังเคราะห์คำตอบภายใต้ prompt และ schema ที่จำกัดไว้ ระบบใช้ schema-bound prompting เพื่อระบุรูปแบบผลลัพธ์ที่ต้องการ เช่น ต้องตอบเป็นข้อความสรุป ต้องคืนค่า action candidate หรือไม่ควรเรียก tool ใดในสถานการณ์นั้น หากคำสั่งเกี่ยวข้องกับ MCP ระบบจะให้ LLM สร้าง execution candidate ที่มี argument ตาม schema เท่านั้น ยังไม่ให้ดำเนินการกับระบบจริงในขั้นนี้



รูปที่ 3.20: Layer 5 Execution Gate สำหรับตรวจสอบ schema, policy และยืนยันการสั่งงานผ่าน MCP tool

Layer 5 Execution Gate เป็นชั้นป้องกันก่อนส่งคำสั่งไปยัง MCP server หรือ backend tool โดยตรวจสอบ schema, policy, role, allowlist และเงื่อนไขด้านความปลอดภัยของ action อีกครั้ง หาก argument ไม่ครบ สิทธิ์ไม่ตรง หรือ action เสี่ยงเกินขอบเขต ระบบจะไม่ execute และส่งผลกลับเป็นคำถามหรือคำอธิบายให้ผู้ดูแลตรวจสอบ วิธีนี้ทำให้ LLM ไม่สามารถสั่งงานจริงได้โดยลำพัง แม้จะสร้างข้อความที่ดูเหมือนคำสั่งถูกต้อง โดยขั้นตอนการสั่งงานจริงทั้งหมดจะทำงานเป็นสัญญาณระหว่างส่วนงานแบบ 3 ขั้นตอน (3-stage chat action flow) คือ **Propose (นำเสนอ) → Confirm (ยืนยันจากผู้ใช้) → Execute (ดำเนินการจริงบนระบบ)** เสมอ

Layer 3 Behavioral State ทำงานเป็นกระบวนการเตรียมบริบทจากข้อมูล biometrics, movement, resident id และ timestamp ที่ระบบรับมาอย่างต่อเนื่อง ข้อมูลรายวันถูกบีบให้เป็นสัญญาณที่อ่านได้ เช่น ค่าเฉลี่ย ส่วนเบี่ยงเบนมาตรฐาน rolling window แนวโน้มจาก linear regression และจำนวนเหตุการณ์ผิดปกติ จากนั้นระบบใช้ rule หรือ model เช่น XGBoost ในระดับต้นแบบเพื่อจัดกลุ่มความเร่งด่วนและแกนปัญหาที่ควรเฝ้าดู ผลลัพธ์ถูกเก็บเป็น state snapshot เชิงพฤติกรรมหรือ **BehavioralStateSnapshot** ในความหมายของเล่มนี้ และถูกบันทึกเป็น payload ภายใต้อายัด `system_state_snapshot` หรือเหตุการณ์ใน `pipeline_events` เพื่อให้ Layer 2 ดึงกลับไปประกอบคำตอบรอบถัดไป โครงสร้างนี้ไม่ใช่ผลวินิจฉัยทางการแพทย์หรือการตัดสินใจแทนผู้ดูแล แต่เป็นบริบทช่วยจัดลำดับสิ่งที่ควรตรวจสอบ โดยการทำงานของแต่ละเลเยอร์ถูกบันทึกลงใน `pipeline_events` เพื่อใช้ประเมินความหน่วงรายขั้นตอนภายใต้อายัดสภาพแวดล้อมระบบที่ระบุแพทช์ `EASEAI_PIPELINE_V2=1`



รูปที่ 3.21: Layer 3 Behavioral State สำหรับเตรียมบริบทพฤติกรรมจากข้อมูล sensor และเหตุการณ์ของผู้ใช้

ส่วน Edge AI ที่เกี่ยวข้องกับ Layer 3 ยังอยู่ในระดับ proof of concept โดยแนวคิด

ด้าน motion anomaly ใช้ข้อมูล IMU เช่น gyro magnitude, acceleration magnitude และ rolling standard deviation เพื่อสังเกตการเคลื่อนไหวที่ต่างจากรูปแบบปกติ ส่วนแนวคิดด้าน health anomaly ใช้ HR, RR interval และ PPG จาก Polar Verity Sense เป็นบริบทประกอบ หากพบเหตุการณ์ที่เข้าข่ายผิดปกติ ระบบจะสร้าง event หรือ context ให้ผู้ดูแลตรวจสอบผ่าน dashboard หรือใช้ประกอบการตอบของ AI ต่อไป ไม่ได้ใช้เป็นผลตัดสินใจทางคลินิก



รูปที่ 3.22: Context Window Management สำหรับจำกัดบริบทให้มีขนาดเหมาะสมและตรวจสอบย้อนกลับได้

Context Window Management เป็นส่วนสนับสนุนที่ช่วยให้การสนทนาและผลลัพธ์จาก tool ไม่โดนเจ็ทของขอบเขตของโมเดล แต่ยังย้อนกลับได้ว่าแต่ละคำตอบใช้ข้อมูลใด ระบบจึง tag payload ตาม session และ I/O turn เก็บ embedding signal สำหรับใช้ rank รวมกับความใหม่ของข้อมูล และมี policy จำกัดจำนวน active session หรือ bundle ที่เก็บใน context เมื่อข้อมูลเก่าเกินจำเป็นจะถูกลดความสำคัญหรือถอดออกจาก prompt แต่ยังคงบันทึกในฐานข้อมูลหรือ log เพื่อการตรวจสอบภายหลัง

MCP ทำหน้าที่เป็นสัญญาณระหว่าง AI runtime กับระบบจริง โดยฝั่ง agent runtime เป็นผู้ประกอบเจตนา บริบท และ schema ก่อนส่ง execution candidate ไปยัง execution gate ส่วน MCP server เปิด resources, prompts และ tools ที่ผ่านการควบคุมสิทธิ์แล้ว การแยกเช่นนี้ช่วยลดโอกาสที่ LLM จะเรียกใช้คำสั่งนอกขอบเขต เพราะ tool ที่รันจริงต้องผ่านทั้ง schema validation, allowlist ตามบทบาท และ execution gate ก่อนเสมอ

3.8.1 สถาปัตยกรรมเซิร์ฟเวอร์ MCP และการจำกัดขอบเขตสิทธิ์การใช้งาน

เพื่อรองรับการขยายขอบเขตการทำงานร่วมกับบริการภายนอก ระบบออกแบบให้ MCP Server ทำงานเป็นบริการแยกส่วนบน FastMCP และถูก mount เข้ากับ FastAPI ภายใต้ path หลัก /mcp โดยรองรับระบบขนส่งข้อมูลสองรูปแบบ (Dual Transport Layer) คือ Server-Sent Events (SSE) ที่ /mcp/sse สำหรับ client ที่ต้องการ session แบบต่อเนื่อง และ Streamable HTTP ที่ /mcp/mcp สำหรับการเรียกใช้ MCP ผ่าน HTTP request/response ตัว registry ของ MCP ใน snapshot ที่ใช้เขียนเล่มมี registered tools หลายกลุ่มรวม 112 tools, 6 prompts และ 4

resources อย่างไรก็ตามการประเมินในเล่มนี้จำกัดขอบเขตเฉพาะ core/evaluated tools จำนวน 28 รายการที่เกี่ยวข้องกับผู้พักอาศัย อุปกรณ์ telemetry dashboard และ action candidate เพื่อลดการปะปนกับเครื่องมือช่วยพัฒนาที่ไม่ได้อยู่ในเส้นทางทดลองหลัก

ความปลอดภัยของ MCP Server ถูกควบคุมผ่านกลไกการจำกัดขอบเขตของโทเค็น (OAuth-based Scope Narrowing) โดยทุกครั้งที่ AI Agent ร้องขอการรันเครื่องมือ ระบบควบคุมสิทธิ์ของ MCP Gateway จะทำการคำนวณขอบเขตสิทธิ์ที่มีผลบังคับใช้จริง (effective scopes) ตามความสัมพันธ์ระหว่างบทบาทของผู้ใช้ (allowed scopes) และขอบเขตการเข้าถึงที่โทเค็นนั้น ๆ ร้องขอ (requested scopes) ดังแสดงในสมการที่ (3.2)

$$S_{\text{effective}} = S_{\text{allowed}} \cap S_{\text{requested}} \quad (3.2)$$

เมื่อ S_{allowed} คือ scope ที่ role และ workspace อนุญาตให้ใช้ และ $S_{\text{requested}}$ คือ scope ที่ token หรือเครื่องมือร้องขอ หากเครื่องมือหรือคำสั่งใดต้องการสิทธิ์ที่อยู่นอกเหนือจาก $S_{\text{effective}}$ ระบบรักษาความปลอดภัยระดับ MCP Gateway จะสกัดกั้นคำขอนั้นทันทีโดยไม่ส่งคำขอไปยังเครื่องมือจริง กลไกนี้ทำงานร่วมกับ session revocation ในหัวข้อ 3.6.1 และ execution gate ของ Layer 5 เพื่อป้องกันไม่ให้ LLM หรือ client เรียกใช้คำสั่งที่อยู่นอกขอบเขตของผู้ใช้

3.9 การทดสอบโครงงาน

การทดสอบโครงงานถูกกำหนดให้สอดคล้องกับโครงสร้างระบบที่ออกแบบไว้ในบทนี้ โดยแยกการตรวจสอบออกเป็นกลุ่มตามส่วนประกอบหลัก ได้แก่ firmware, mobile gateway, backend, dashboard และ AI/MCP pipeline จุดประสงค์ของการทดสอบในขั้นนี้คือการตรวจสอบว่าระบบย่อยแต่ละส่วนทำงานตามหน้าที่ที่ออกแบบไว้ก่อนนำผลจากการทดลองจริงไปวิเคราะห์ในบทที่ 4

ตารางที่ 3.6: กลยุทธ์การทดสอบโครงการและความเชื่อมโยงกับผลการประเมินในบทที่ 4

กลุ่มการทดสอบ	เครื่องมือและสิ่งที่ตรวจสอบ	ผลที่เชื่อมโยงในบทที่ 4
Backend/API และ EaseAI	pytest ตรวจสอบ FastAPI endpoint, policy, contract และ pipeline event ของ agent runtime	ความเสถียรของ server, IRR, cosine similarity และ latency ราย layer
Frontend unit	Jest ตรวจสอบ navigation, hook และ utility ที่เกี่ยวข้องกับ dashboard role flow	ความชัดเจนของ HMI และ task completion ของแต่ละบทบาท
Dashboard E2E	Playwright ตรวจสอบ login, role visibility, dashboard navigation และ EaseAI interaction	ผลประเมิน Dashboard/HMI และ SUS Score
Mobile gateway	flutter test ตรวจสอบ gateway runtime, sensor telemetry model และ widget พื้นฐานของ Flutter app	ผลทดสอบ BLE gateway, MQTT/REST publish และ dashboard update
Firmware	PlatformIO ตรวจสอบการ build firmware ในโฟลเดอร์ M5StickCPlus2_BLEGateway และการตั้งค่า board/library	ผลทดสอบ IMU, velocity, distance และ telemetry completeness
CI/CD	GitHub Actions ใน ci.yml และ security.yml ตรวจสอบ backend, frontend, E2E และ security gate	สรุปเสถียรภาพระบบและข้อจำกัดก่อนนำไปใช้จริง

1. **Backend/API และ EaseAI pipeline:** ใช้ **pytest** ตรวจสอบเส้นทาง API, data contract, การตรวจสอบสิทธิ์, การบันทึก **pipeline_events** และการประกอบ **ValidatedContextPackage** ชุดทดสอบนี้จำลอง actor facts เช่น role, user id, workspace id และ patient id เพื่อดูว่า policy และ scope ถูกบังคับใช้ก่อนส่งข้อมูลไปยัง LLM ผลการทดสอบเชื่อมกับหัวข้อ 4.4, 4.6 และ 4.8
2. **Frontend unit และ dashboard behavior:** ใช้ **Jest** ตรวจสอบ logic ของ Next.js เช่น navigation, role route, utility และการแสดงผลพื้นฐานของ dashboard เพื่อดูว่าหน้าจอเปลี่ยนตามสิทธิ์ผู้ใช้ได้ถูกต้องก่อนนำไปประเมินกับ workflow ของผู้ใช้ในหัวข้อ 4.5

3. **End-to-End HMI:** ใช้ Playwright จำลองการใช้งานผ่านเบราว์เซอร์ ตั้งแต่ login, การเปิด dashboard ตามบทบาท Admin, Head Nurse, Supervisor, Observer และ Patient, การตรวจ alert card และการเรียก EaseAI ในสถานการณ์ที่กำหนด ผลส่วนนี้ใช้ประกอบการประเมิน task completion และ SUS Score
4. **Mobile gateway:** ใช้ flutter test กับ ชุดทดสอบ เช่น gateway_runtime_service_test.dart, sensor_telemetry_test.dart และ widget_test.dart เพื่อตรวจ model ของ telemetry, state ของ gateway runtime และองค์ประกอบ UI พื้นฐานของแอป Flutter ผลการทดสอบเชื่อมกับการส่งข้อมูลจาก M5StickC Plus2, Polar Verity Sense และ phone gateway ในหัวข้อ 4.4
5. **Firmware และอุปกรณ์ภาคสนาม:** ใช้ PlatformIO ตรวจสอบว่า firmware ของ M5StickCPlus2_BLEGateway build ได้ตาม board และ library ที่กำหนด ก่อนนำไปทดสอบกับอุปกรณ์จริงในเรื่อง IMU, battery, BLE notification, velocity และ distance การทดสอบ build เป็นการตรวจสอบความถูกต้องของการตั้งค่า firmware ไม่ใช่การยืนยันค่าความแม่นยำของ sensor โดยตรง
6. **CI/CD และ security gate:** ใช้ GitHub Actions ตรวจสอบ backend test, frontend build, E2E test และ dependency/security gate ที่กำหนดไว้ ผลส่วนนี้ใช้เป็นหลักฐานประกอบว่าระบบย่อยผ่านการตรวจขั้นพื้นฐานก่อนรวบรวมผลทดลอง แต่ยังไม่ใช้การประเมินความทนทานระยะยาวของระบบทั้งหมด

จากขั้นตอนดังกล่าว การทดสอบโครงการจึงทำหน้าที่เป็นหลักฐานสนับสนุนด้านวิศวกรรมก่อนเข้าสู่บทผลการทดลอง โดยผลที่นำเสนอในบทที่ 4 ยังต้องอ่านร่วมกับข้อจำกัดของข้อมูลทดสอบ สภาพแวดล้อม และจำนวนผู้เข้าร่วมที่ใช้ในการประเมิน

3.10 แนวทางการประเมินผลระบบ

การประเมินผลระบบถูกออกแบบให้เชื่อมกับองค์ประกอบหลักของสถาปัตยกรรมที่อธิบายในบทนี้ โดยไม่สรุปผลเกินกว่าข้อมูลทดลองที่มี ในบทที่ 4 แต่ละหัวข้อจึงระบุสภาพแวดล้อม

การทดสอบ วิธีทดสอบ ผลที่ได้ และการวิเคราะห์ในรูปแบบย่อหน้าต่อเนื่อง เพื่อให้ผู้อ่านแยกได้ว่า ปัญหาหรือข้อจำกัดเกิดจาก sensor, gateway, server, dashboard หรือ AI pipeline

ตารางที่ 3.7: แนวทางการประเมินผลที่ใช้ต่อเนื่องไปยังบทผลการทดลอง

ส่วนที่ประเมิน	สิ่งที่ทดสอบ	ตัวชี้วัดหรือผลที่ต้องรายงาน
Sensor และอุปกรณ์ภาคสนาม	M5StickC Plus2 IMU, battery, Polar HR/RR/PPG, BLE connection	ความต่อเนื่องของข้อมูล ความถูกต้องเชิงสังเกต และความเสถียรของการเชื่อมต่อ
Indoor localization	การเก็บ fingerprint, KNN, max-RSSI fallback และการส่งผล prediction	accuracy, error หรือความคลาดเคลื่อนรายจุด พร้อมวิเคราะห์ RSSI fluctuation
Mobile gateway และ server	BLE pairing, MQTT/REST publish, reconnect/offline handling, backend ingest	อัตราการส่งข้อมูลสำเร็จ latency และการแสดงผลบน dashboard
Dashboard/HMI	workflow ของ Admin, Head Nurse, Supervisor, Observer และ Patient	task completion, ข้อสังเกตจากการใช้งาน และ SUS Score สำหรับ usability
AI/MCP pipeline	Intent routing, context retrieval, constrained LLM, execution gate และ trace ต่อ layer	IRR, cosine similarity, latency ต่อ layer และความปลอดภัยของ tool execution

การเพิ่ม SUS Score ในการประเมินช่วยให้ส่วน HMI ไม่ได้ประเมินจากภาพหน้าจอหรือความเห็นทั่วไปเพียงอย่างเดียว แต่มีคะแนนมาตรฐานด้าน usability ประกอบกับ task scenario ของผู้ใช้ ขณะที่การประเมิน AI ใช้ทั้งความเห็นของมนุษย์ต่อคำตอบ ความใกล้เคียงเชิงข้อความ และ latency ราย layer เพื่อแสดงให้เห็นว่าการแยก logic ออกจาก LLM ช่วยให้ระบบตรวจสอบและปรับปรุงได้เป็นส่วน ๆ

บทที่ 4

ผลการทดลองและการประเมินผล

4.1 ภาพรวมการทดลอง

การประเมินผลระบบต้นแบบ WheelSense ในบทนี้จัดทำเพื่อดูว่าองค์ประกอบหลักที่พัฒนาไว้สามารถทำงานร่วมกันได้ในระดับต้นแบบหรือไม่ โดยแบ่งการทดสอบออกเป็น ส่วนที่สอดคล้องกับสถาปัตยกรรมในบทที่ 3 ได้แก่ อุปกรณ์ sensor และอุปกรณ์ภาคสนาม การระบุตำแหน่งภายในอาคารด้วย RSSI/KNN การทำงานของ mobile gateway และ server การใช้งาน dashboard/HMI พร้อมการประเมิน SUS และการทดสอบ AI/MCP pipeline ในด้านความสอดคล้องของการเลือกเครื่องมือ ความสม่ำเสมอของคำตอบ และเวลาแฝงของแต่ละ layer

การทดลองทั้งหมดเป็นการทดสอบในพื้นที่ทดลองและสภาพแวดล้อมของโครงการ ไม่ใช่การนำระบบไปใช้งานจริงในสถานดูแลผู้สูงอายุแบบต่อเนื่อง ดังนั้นผลที่รายงานจึงเป็นผลเพื่อยืนยันความเป็นไปได้ของระบบต้นแบบและเพื่อระบุข้อจำกัดที่ต้องพัฒนาต่อ ไม่ใช่การรับรองความพร้อมใช้งานทางการแพทย์หรือการใช้งานจริงในทุกสภาพแวดล้อม ตารางที่ 4.1 แสดงขอบเขตการทดลองที่ใช้ในบทนี้

จากตารางที่ 4.1 การรายงานผลจะไม่แยกเป็นคำอธิบายเชิงทฤษฎีซ้ำกับบทที่ 2 หรือ บทที่ 3 แต่จะเน้นว่าทดสอบในเงื่อนไขใด ใช้วิธีใด ผลที่ได้เป็นอย่างไร และผลนั้นสะท้อนข้อจำกัดของระบบส่วนใด การจัดรูปแบบนี้ยึดแนวทางเดียวกับรายงานรุ่นพี่ที่อธิบายเงื่อนไขการทดลองก่อน แล้วจึงอ้างตารางหรือรูปเพื่อสรุปผลที่วัดได้

ตารางที่ 4.1: ภาพรวมขอบเขตการทดลองของระบบ WheelSense

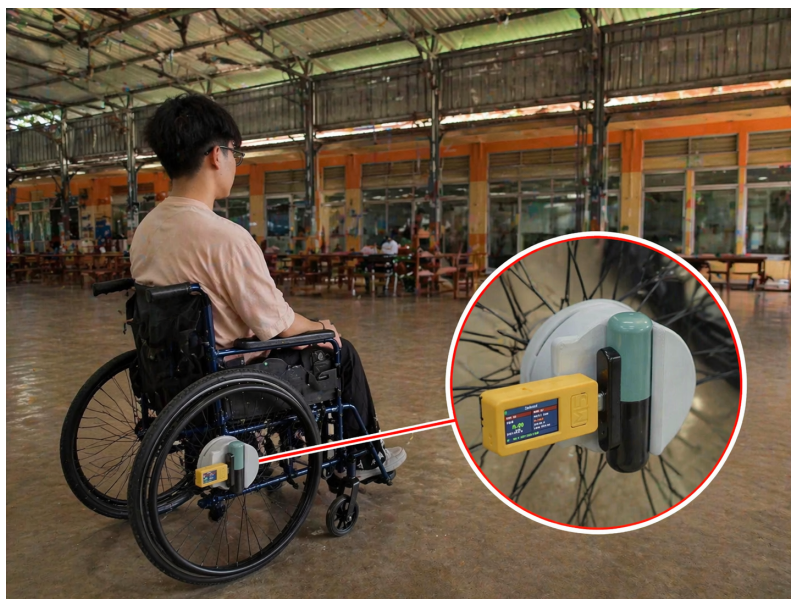
ส่วนที่ทดสอบ	ประเด็นที่ประเมิน	หลักฐานที่ใช้ประกอบผล
Sensor และอุปกรณ์ภาคสนาม	ความถี่ IMU, ความครบถ้วนของ telemetry, การจำแนก motion state, การเชื่อมต่อ Polar	ตารางสรุป sensor, ตาราง motion state, ภาพอุปกรณ์ และผล mobile gateway
Indoor localization	ผลของ RSSI fingerprint, KNN และ max-RSSI fallback เมื่อสัญญาณผันผวน	ตารางผล RSSI/KNN ในภาคผนวก และการวิเคราะห์ข้อจำกัดของสัญญาณ
Mobile gateway และ server	BLE connection, MQTT/REST publish, reconnect, database latency และ dashboard update	ตาราง gateway, ตาราง server/database latency และภาพ runtime
Dashboard/HMI และ SUS	การทำงานตามบทบาทผู้ใช้ task completion และ usability score ที่มีหลักฐานประกอบ	ตาราง task result และ usability summary โดยระบุสถานะของ SUS score อย่างระมัดระวัง
AI/MCP pipeline	IRR, cosine similarity และ latency ของ Layer 1, 2, 4 และ 5	ชุดทดสอบ AI 36 กรณี และการวัด latency 30 รอบ

4.2 การทดสอบ Sensor และอุปกรณ์ภาคสนาม

การทดสอบส่วนนี้ตรวจสอบการทำงานของ M5StickC Plus2 ที่ติดตั้งกับรถเข็นเพื่ออ่านค่า IMU และ battery ร่วมกับ Polar Verity Sense ที่อ่านข้อมูลสัญญาณชีพผ่าน mobile gateway การทดสอบทำในพื้นที่ทดลองโดยกำหนดลักษณะการเคลื่อนไหวของรถเข็นเป็น 4 กลุ่ม ได้แก่ หยุดนิ่ง เคลื่อนที่ปกติ เลี้ยวแรง และเหตุการณ์ลักษณะกระแทก เพื่อดูว่าข้อมูล sensor มีความต่อเนื่องเพียงพอและสามารถใช้จำแนกสถานะเบื้องต้นได้หรือไม่

ขั้นตอนการทดลองเริ่มจากการให้ M5StickC Plus2 อ่านค่า IMU ด้วยรอบเวลาที่กำหนด จากนั้นส่งข้อมูล telemetry ผ่าน mobile gateway ไปยัง broker และ backend โดยทดสอบ 10 รอบ รอบละ 60 วินาที เป้าหมายการอ่านข้อมูลอยู่ที่ 20 Hz คิดเป็นข้อมูลที่คาดหวังประมาณ 12,000 packet และรับได้ 11,808 packet ส่วนข้อมูลการเคลื่อนไหวถูกแบ่งเป็นหน้าต่างข้อมูล 170 window เพื่อเปรียบเทียบกับ การสังเกตระหว่างทดลอง ผลสรุปเชิงระบบแสดงในตารางที่ 4.2

เมื่อพิจารณาข้อมูล motion state การจำแนกสถานะหยุดนิ่งและการเคลื่อนที่ปกติให้ผลดีกว่าสถานะเลี้ยวแรงและเหตุการณ์ลักษณะกระแทก เนื่องจากสองสถานะหลังเกิดในช่วงเวลาสั้น และขึ้นกับแรงที่ผู้ทดสอบกระทำต่อรถเข็น ตารางที่ 4.3 แสดงจำนวนหน้าต่างข้อมูลและผลที่ตรวจจับ



รูปที่ 4.1: การติดตั้ง M5StickC Plus2 บนรถเข็นสำหรับทดสอบข้อมูล IMU และ telemetry

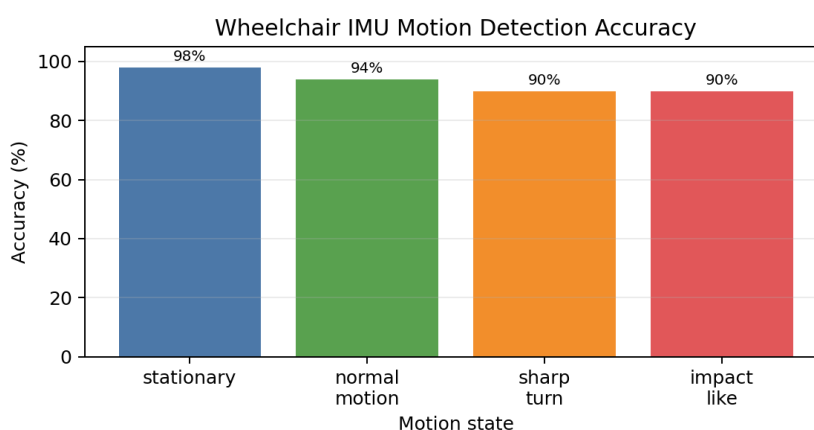
ตารางที่ 4.2: ผลสรุปการทดสอบอุปกรณ์ภาคสนาม

ตัวชี้วัด	ค่าเฉลี่ย	P95	หมายเหตุ
IMU sampling rate	19.6 Hz	20.3 Hz	ทดสอบ 10 รอบ รอบละ 60 วินาที ไกล่เคียงเป้า หมาย 20 Hz
telemetry completeness	98.4%	99.1%	รับได้ 11,808 จาก 12,000 packet ที่คาด หวัง
device/gateway to broker latency	41 ms	156 ms	ทดสอบภายใต้เครือข่าย ปกติของพื้นที่ทดลอง
Polar connection success	94.4%	–	สำเร็จ 34 จาก 36 ครั้ง พบการ re-pairing บาง รอบ

ได้

ตารางที่ 4.3: ผลการตรวจจับสถานะการเคลื่อนไหวจากหน้าต่างข้อมูล IMU

สถานะ	จำนวน window	ตรวจจับได้	Accuracy	หมายเหตุ
stationary	50	49	0.98	baseline ของการ สั่นต่ำ
normal motion	50	47	0.94	รถเข็นเคลื่อนที่ปกติ
sharp turn	40	36	0.90	gyro magnitude สูงขึ้นในช่วงสั้น
impact-like	30	27	0.90	ตรวจเทียบกับการ สั่นกระทันหัน ทดลอง



รูปที่ 4.2: กราฟสรุป accuracy ของการตรวจจับสถานะการเคลื่อนไหวจาก IMU

ผลการทดสอบแสดงว่า M5StickC Plus2 สามารถอ่านและส่งข้อมูล IMU ได้ใกล้เคียงค่าที่ออกแบบไว้ ขณะที่ telemetry completeness อยู่ในระดับที่เพียงพอสำหรับระบบต้นแบบ อย่างไรก็ตามการจำแนกเหตุการณ์ที่เกิดขึ้นเร็ว เช่น การเลี้ยวแรงหรือการกระแทก ยังต้องอาศัยข้อมูลระยะยาวและการทวนสอบกับเหตุการณ์จริงมากขึ้นก่อนใช้เป็นตัวชี้วัดความเสี่ยง

4.2.1 ผลการประเมินความแม่นยำในการวัดระยะทาง ความเร็ว และความเร่ง

การประเมินความแม่นยำของข้อมูลการเคลื่อนไหวดำเนินการในพื้นที่ทดสอบภายในอาคาร โดยติดตั้ง M5StickC Plus2 บนล้อรถเข็นและกำหนดเส้นทางตรงระยะอ้างอิง 10.00 เมตร ผู้ทดสอบจำนวน 2 คนเข็นรถผ่านเส้นทางเดียวกันคนละ 10 รอบ รวม 20 รอบการทดสอบ จากนั้นเปรียบเทียบระยะทางและความเร็วที่ระบบคำนวณได้กับระยะอ้างอิงและเวลาที่จับด้วยวิธี manual

timing calibration ผลการทดสอบสรุปแสดงในตารางที่ 4.4

ตารางที่ 4.4: ผลการทดสอบความแม่นยำในการวัดระยะทางและความเร็วของเซ็นเซอร์รถเข็นในอาคาร (จำนวน 20 รอบการทดสอบ)

ผู้ทดสอบ	ระยะทางจริง (m)	ระยะทางที่วัดได้ (m)	Error (%)	ความเร็วเฉลี่ยจริง (m/s)	ความเร็วที่วัดได้ (m/s)	Error (%)
คนที่ 1	10.00	9.85 ± 0.12	1.50	1.20	1.17 ± 0.05	2.50
คนที่ 2	10.00	9.78 ± 0.14	2.20	1.35	1.31 ± 0.07	2.96
ค่าเฉลี่ย	10.00	9.82 ± 0.13	1.85	1.28	1.24 ± 0.06	2.73

จากตารางที่ 4.4 พบว่าการคำนวณระยะทางมีค่าคลาดเคลื่อนเฉลี่ยร้อยละ 1.85 และการวัดความเร็วมีค่าคลาดเคลื่อนเฉลี่ยร้อยละ 2.73 ภายใต้เงื่อนไขการทดสอบในอาคารบนเส้นทางตรงระยะสั้น ผลดังกล่าวเพียงพอสำหรับใช้เป็นตัวชี้วัดเชิงพฤติกรรมระดับต้นแบบ เช่น การเคลื่อนที่ต่อเนื่อง ระยะที่เคลื่อนในช่วงเวลาหนึ่ง และการเปลี่ยนแปลงความเร็วโดยประมาณ แต่ยังไม่ควรตีความเป็นค่ามาตรฐานที่ใช้แทนเครื่องมือวัดมาตรฐาน

ในส่วนของการวัดความเร่ง สัญญาณ acceleration และ angular velocity ถูกใช้เป็น feature สำหรับแยกจังหวะหยุดนิ่ง เคลื่อนที่ เลี้ยวแรง และเหตุการณ์กระแทก ผลเชิงความเร่งจึงรายงานร่วมกับผล motion-window ในตารางที่ 4.5 แทนการสร้างตัวเลขใหม่ที่ไม่มีข้อมูลดิบรองรับ โดยใช้จำนวน window และผลตรวจจับจากการทดสอบ motion state ชุดเดียวกับตารางที่ 4.3

ผลในตารางที่ 4.5 แสดงว่า EMA, gyro offset calibration และ deadband threshold ที่อธิบายในบทที่ 3 ช่วยลดสัญญาณรบกวนช่วงหยุดนิ่งและทำให้ feature ของการเคลื่อนที่มีความต่อเนื่องมากขึ้น อย่างไรก็ตามเหตุการณ์ที่เกิดขึ้นเร็ว เช่น เลี้ยวแรงหรือกระแทก ยังมีความคลาดเคลื่อนสูงกว่าสถานะหยุดนิ่งและเคลื่อนที่ปกติ จึงควรใช้เป็นสัญญาณช่วยเตือนเบื้องต้นร่วมกับตำแหน่ง เวลา และข้อมูลผู้ดูแล ไม่ใช่ผลตัดสินเหตุการณ์อันตรายโดยลำพัง

ตารางที่ 4.5: ผลการตรวจ feature ความเร่งและ gyro จาก IMU สำหรับจำแนกสถานะการเคลื่อนไหว

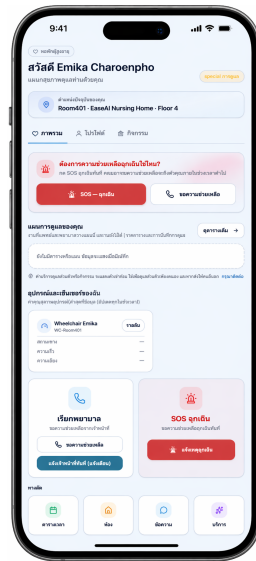
สถานะทดสอบ	จำนวน window	feature ที่ใช้ตรวจ	ผลตรวจจับ	ข้อสังเกต
หยุดนิ่ง	50	acceleration magnitude ต่ำร่วมกับ deadband	49/50	สะท้อน baseline noise ต่ำหลังกรองสัญญาณ
เคลื่อนที่ปกติ	50	แนวโน้ม velocity และ acceleration ต่อเนื่อง	47/50	เหมาะกับการตรวจการเคลื่อนที่ระยะสั้น
เลี้ยวแรง	40	gyro magnitude และการเปลี่ยนทิศทางเร็ว	36/40	พลาดได้เมื่อเหตุการณ์สั้นหรือ RSSI/IMU fluctuation สูง
กระแทกคล้ายชน	30	acceleration magnitude peak และ jerk โดยประมาณ	27/30	ใช้เป็น alert candidate ได้ แต่ยังต้องยืนยันด้วยข้อมูลบริบท

4.3 การทดสอบ Indoor Localization ด้วย KNN

การทดสอบการระบุตำแหน่งภายในอาคารใช้ข้อมูล RSSI จาก BLE node ในพื้นที่ทดลอง โดย gateway สแกนสัญญาณและส่งรายการ RSSI พร้อม timestamp เข้าสู่ backend เพื่อประเมินตำแหน่งด้วย fingerprint และ KNN ในกรณีที่ fingerprint ยังไม่พร้อมหรือค่าความมั่นใจต่ำ ระบบสามารถใช้ max-RSSI เป็น baseline หรือ fallback ได้ หัวข้อนี้จึงอธิบายสภาพพื้นที่ทดลอง วิธีจัดชุดข้อมูล ผลที่ได้ และข้อสังเกตจากความผันผวนของ RSSI ต่อเนื่องในรูปแบบเดียวกับการทดสอบส่วนอื่นของบทนี้

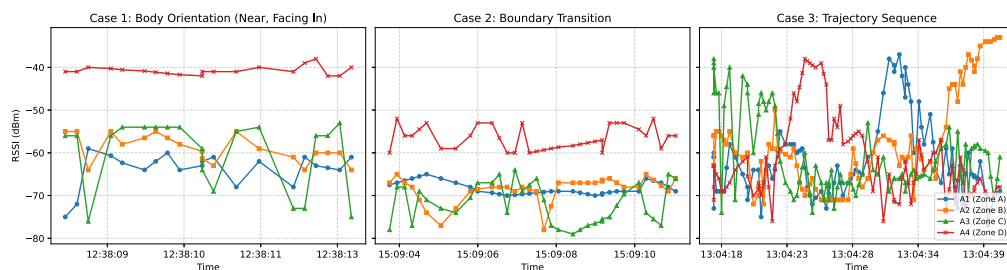
พื้นที่ทดลองจากชุดข้อมูล RSSI แบ่งห้องหนึ่งออกเป็น 4 zone ขนาดประมาณ 4×4 เมตรต่อ zone โดยใช้ฉากผ้าเป็นขอบเขตระหว่างพื้นที่ และติดตั้ง ESP32-S3 BLE anchor จำนวน 4 ตัวไว้บริเวณกึ่งกลางของแต่ละ zone ที่ระดับความสูงประมาณ 0.7 เมตร อุปกรณ์ tag คือ M5StickC Plus2 ที่โฆษณา BLE ด้วยอัตราประมาณ 5 Hz หรือคาบ 200 ms และอยู่ที่ระดับความสูงประมาณ 0.7 เมตรเช่นกัน ข้อมูลหนึ่งตัวอย่างประกอบด้วย timestamp และค่า RSSI จาก anchor ทั้ง 4 ตัวในรูปแบบ $(timestamp, r_1, r_2, r_3, r_4)$

ชุดทดลองแบ่งเป็น 3 กรณี กรณีละ 48 episode และใช้การแบ่งข้อมูลแบบเดียวกัน คือ 13 episode สำหรับ train และ 35 episode สำหรับ test โดย Case 1 เป็นการตรวจ body



รูปที่ 4.3: ตัวอย่างหน้าจอ mobile gateway ที่ใช้ตรวจสอบสถานะตำแหน่งและข้อมูลจาก BLE node

orientation แบบ facing-in/facing-out, Case 2 เป็น boundary transition ระหว่างในและนอก zone และ Case 3 เป็น multi-zone trajectory ที่ต้องทำนายลำดับการเคลื่อนที่ผ่าน 4 zone วิธี KNN ใช้ $k = 5$ และ Euclidean distance สำหรับ Case 1 และ Case 2 จะทำนายจาก RSSI 4 มิติราย sample แล้วรวมผลเป็นระดับ episode ด้วยค่าเฉลี่ยของผลทำนาย ส่วน Case 3 แบ่ง episode เป็น 4 segment ต่อเนื่องและใช้ feature mean/std ของ RSSI จาก anchor ทั้ง 4 ตัวในแต่ละ segment เพื่อทำนายลำดับ zone



รูปที่ 4.4: ตัวอย่างสัญญาณ RSSI ต่อเนื่องจากชุดทดลอง Case 1, Case 2 และ Case 3

ผลการทดสอบ KNN จากชุดข้อมูลที่มีหลักฐานประกอบแสดงในตารางที่ 4.6 โดยรายละเอียด raw data และ summary อยู่ในภาคผนวก ค และแนบเอกสารเรื่อง *Comparative Performance Evaluation of BLE-Based Indoor Motion Tracking Using Machine Learning and Large Language Models* ไว้ในภาคผนวก ญ

จากตารางที่ 4.6 Case 1 และ Case 3 ให้ค่าความถูกต้องเท่ากันที่ 0.8857 โดย Case 3

ตารางที่ 4.6: ผลการทดสอบ RSSI/KNN จากชุดข้อมูลที่มีหลักฐานประกอบ

Case	เงื่อนไขทดลอง	Accuracy	Precision	Recall	F1
Case 1	Body orientation, facing-in/ facing-out	0.8857	0.8421	0.9412	0.8889
Case 2	Boundary transition, in/out zone	0.6857	0.6875	0.6471	0.6667
Case 3	Multi-zone trajectory, exact sequence	0.8857	0.8857	0.8857	0.8857

ตารางที่ 4.7: ผลเปรียบเทียบ KNN กับ max-RSSI baseline

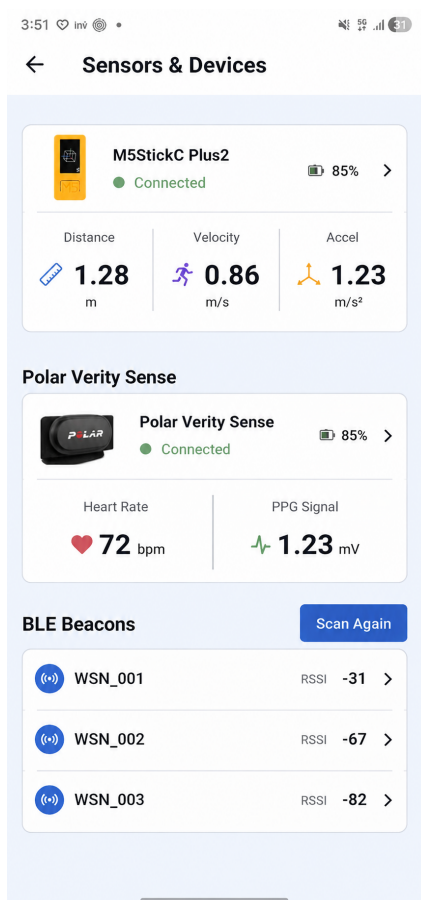
Case	เงื่อนไขทดลอง	KNN	Max-RSSI	ส่วนต่าง
Case 1	Body orientation, facing-in/ facing-out	0.8857	0.8000	+0.0857
Case 2	Boundary transition, in/out zone	0.6857	0.6286	+0.0571
Case 3	Multi-zone trajectory, exact sequence	0.8857	0.7714	+0.1143

ยังมีค่า per-position accuracy สูงกว่าค่า exact sequence เพราะการทำนายลำดับหนึ่ง episode ต้องถูกครบทุกตำแหน่งพร้อมกันจึงนับว่าถูก ส่วน Case 2 ลดลงเหลือ 0.6857 เนื่องจากตำแหน่งบริเวณ boundary มี RSSI จากหลาย anchor ใกล้เคียงกันและได้รับผลจาก body shadowing หรือ multipath พร้อมกันหลายช่องสัญญาณ KNN จึงสับสนได้ง่ายเมื่อ distribution ของ RSSI ซ้อนทับกัน ผลนี้สะท้อนว่า BLE RSSI ใช้ได้ในระดับ zone สำหรับ prototype แต่ยังคงเก็บ fingerprint ในตำแหน่งจริงให้มากขึ้น ปรับจำนวน anchor หรือใช้ fallback เช่น max-RSSI/ temporal smoothing เมื่อค่า confidence ต่ำ

4.4 การทดสอบ Mobile Gateway และ Server

ส่วน mobile gateway ถูกทดสอบเพื่อดูการรวมข้อมูลจาก M5StickC Plus2, Polar Verity Sense และข้อมูลจากโทรศัพท์ก่อนส่งเข้าสู่ platform ผ่าน MQTT หรือ REST API การทดสอบเน้นเวลาการเชื่อมต่อ BLE, latency จาก smartphone ไปถึง broker, อัตราความสำเร็จของ MQTT publish, ความครบถ้วนของ vitals telemetry และการทำงานเมื่อเกิด reconnect หรือ retry queue

ผลการทดสอบในตารางที่ 4.8 แสดงว่าเวลาการเชื่อมต่อ BLE GATT อยู่ในระดับไม่กี่วินาทีจากการทดสอบ 36 ครั้ง และการส่งข้อมูลจาก smartphone ไปถึง broker มีค่า p50 เท่ากับ



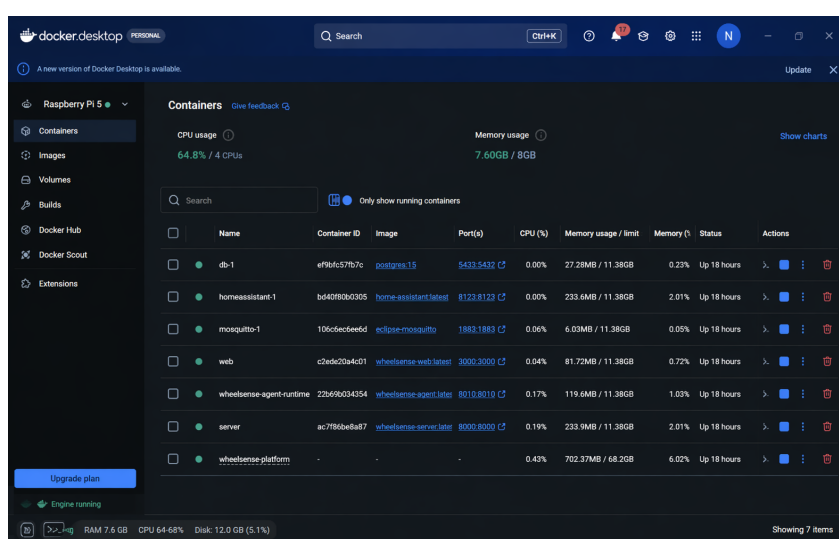
รูปที่ 4.5: หน้าจอ mobile gateway สำหรับตรวจสอบการเชื่อมต่อ Polar Verity Sense และสถานะส่งข้อมูล

78 ms ส่วน MQTT publish ทดสอบ 1,200 ครั้งและสำเร็จ 1,183 ครั้ง คิดเป็น 98.6% ซึ่งเพียงพอสำหรับการทดสอบระบบต้นแบบ แต่ยังพบผลกระทบจากการ re-pairing และการสูญหายของ PPG block บางช่วง

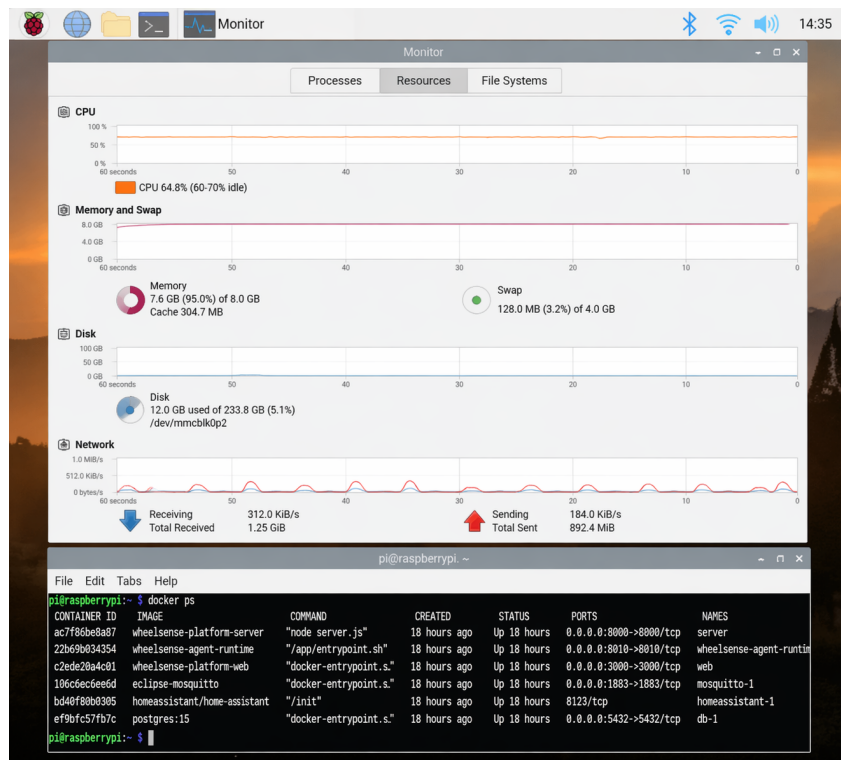
ตารางที่ 4.8: ผลการทดสอบ mobile gateway และ latency ของ server path

ตัวชี้วัด	P50/ค่าเฉลี่ย	P95	หมายเหตุ
BLE GATT connection time	3.2 s	5.8 s	ทดสอบการจับคู่ 36 ครั้งกับ M5StickC Plus2 และ Polar
smartphone to broker latency	78 ms	215 ms	วัดจาก gateway ไปถึง broker
MQTT connection success	98.6%	–	สำเร็จ 1,183 จาก 1,200 publish มี retry/reconnect บางช่วง
vitals completeness	96.2%	98.9%	รับได้ 2,309 จาก 2,400 block จาก Polar ผ่าน mobile gateway
heart rate resting	72.4 bpm	–	ค่าที่ใช้เป็น baseline ระหว่างทดสอบ
heart rate active	95.6 bpm	–	ค่าขณะเคลื่อนไหว/ทำกิจกรรม
PPG block loss	2.1%	–	บางช่วงหายจากการเคลื่อนไหวหรือ BLE reconnect

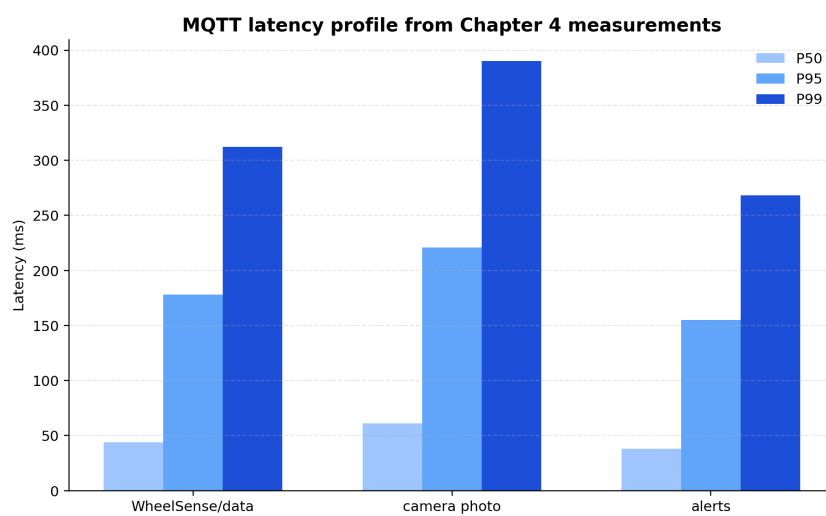
สำหรับฝั่ง server การทดสอบตรวจสอบเส้นทาง ingest, validate, store และ query ผ่าน FastAPI, MQTT broker และ PostgreSQL/JSONB บน Raspberry Pi 5 โดยดูทั้งค่า latency ราย endpoint และการอ่านข้อมูลที่ dashboard หรือ AI ต้องใช้ซ้ำ



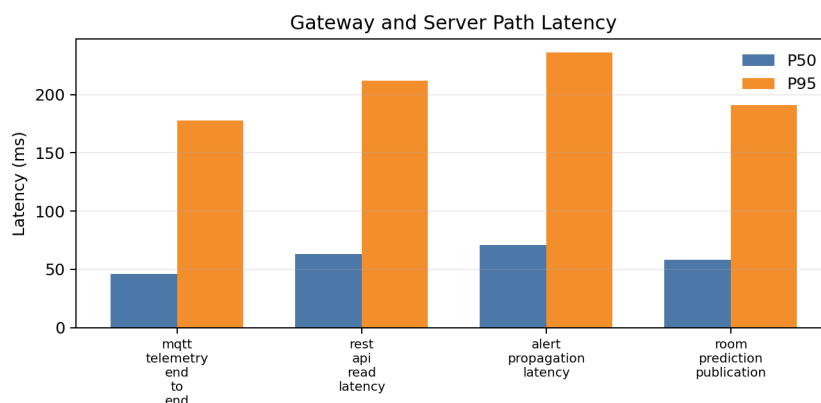
รูปที่ 4.6: ตัวอย่างการตรวจสอบบริการบน Raspberry Pi 5 และ Docker runtime ระหว่างการทดสอบ server



รูปที่ 4.7: หน้าจอ monitor ของ Raspberry Pi 5 ระหว่างรัน service และ container ของระบบ



รูปที่ 4.8: กราฟสรุปการกระจายของ latency ในเส้นทาง MQTT ระหว่างการทดสอบ



รูปที่ 4.9: กราฟสรุปค่า P50 และ P95 ของ server path จากการทดสอบ ingest, API, alert และ room prediction

ตารางที่ 4.9: ผลการทดสอบ latency ของ server path

Server path	P50	P95	P99	หมายเหตุ
MQTT telemetry end-to-end	46 ms	178 ms	304 ms	รับ telemetry จาก gateway
REST API read latency	63 ms	212 ms	341 ms	อ่านข้อมูลให้ dashboard
Alert propagation latency	71 ms	236 ms	388 ms	สร้างและส่งต่อ alert
Room prediction publication	58 ms	191 ms	325 ms	เผยแพร่ผลตำแหน่ง

ตารางที่ 4.10: ผลการทดสอบ latency ของ query ฐานข้อมูล

Query path	P50	P95	หมายเหตุ
recent telemetry JSONB	3 ms	43 ms	อ่านค่าสถานะล่าสุดของผู้ใช้
alert list by patient state	14 ms	50 ms	อ่านรายการ alert สำหรับ dashboard
recent sensor event	12 ms	35 ms	อ่านเหตุการณ์ล่าสุดจากอุปกรณ์
audit trail read	24 ms	51 ms	อ่านประวัติการทำงาน

ผลจาก mobile gateway และ server สะท้อนว่าสถาปัตยกรรมที่ให้โทรศัพท์เป็น gateway หลักช่วยลดปัญหาข้อมูลซ้ำจากหลายแหล่งและทำให้สามารถตรวจสอบสถานะการเชื่อมต่อได้ชัดเจน ขณะที่ backend มี latency ระดับเพียงพอสำหรับ dashboard ต้นแบบ อย่างไรก็ตาม BLE และ PPG stream ยังไวต่อการเคลื่อนไหวของผู้ใช้และการ reconnect จึงต้องเพิ่ม offline buffer และ diagnostic ที่ละเอียดขึ้นก่อนใช้งานระยะยาว

4.5 การประเมิน Dashboard/HMI และ SUS Score

การประเมิน dashboard/HMI ใช้ task scenario ตามบทบาทผู้ใช้ เช่น Admin, Head Nurse, Supervisor, Observer และ Patient โดยมีผู้เข้าร่วมทดสอบ 15 คน และดูว่าผู้ใช้สามารถเข้าถึงข้อมูลผู้พักอาศัย สถานะอุปกรณ์ alert และหน้าที่เกี่ยวข้องกับ mobile gateway ได้ครบหรือไม่ ผลที่รายงานเป็นข้อมูลระดับต้นแบบและใช้เพื่อประเมินทิศทาง usability ของระบบ

ตารางที่ 4.11: ผลสรุป usability ของ dashboard/HMI จากหลักฐานปัจจุบัน

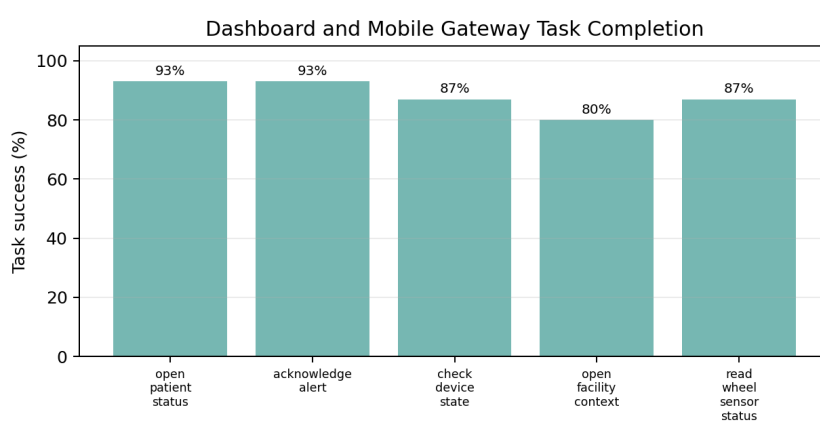
ตัวชี้วัด	ผลเฉลี่ย	การตีความ
task completion	0.93	ผู้เข้าร่วม 15 คนทำภารกิจหลักสำเร็จในระดับสูง
average satisfaction	4.31/5	ความพึงพอใจต่อภาพรวมอยู่ในระดับดีสำหรับต้นแบบ
alert readability	4.42/5	ข้อความและการจัดวาง alert อ่านได้ค่อนข้างชัดเจน
role navigation clarity	4.18/5	การแยกบทบาทช่วยลดความสับสน แต่ยังต้องเพิ่ม workflow บางส่วน
SUS mean / SD	76.5 / 8.7	สะท้อน usability ระดับใช้งานได้ดีในระบบต้นแบบ

เพื่อให้เห็นงานย่อยในแต่ละบทบาท ตารางที่ 4.12 สรุป task ที่ใช้ทดสอบ dashboard และ mobile gateway ผลแสดงว่าหน้าที่ดูผู้พักอาศัยและอ่าน alert ทำได้ดีกว่างานที่เกี่ยวกับการแก้ไขอุปกรณ์หรือการตรวจ queue การส่งข้อมูล ซึ่งเป็นส่วนที่ควรพัฒนาต่อใน workflow ของผู้ดูแล

ผล usability แสดงว่า dashboard ช่วยให้ผู้ใช้ดูแลมองเห็นข้อมูลผู้พักอาศัยและสถานะอุปกรณ์ได้รวมศูนย์มากขึ้น โดยคะแนน SUS เฉลี่ย 76.5 อยู่ในระดับที่ใช้สนับสนุนความเหมาะสมของแนวทาง HMI สำหรับระบบต้นแบบ อย่างไรก็ตามงานที่เกี่ยวกับบริบทพื้นที่และการตรวจอุปกรณ์ยังใช้เวลามากกว่างานอ่านสถานะทั่วไป จึงควรปรับ navigation และตัวกรองข้อมูลในรอบพัฒนาถัดไป

ตารางที่ 4.12: ผลการทำภารกิจบน dashboard และ mobile gateway

ภารกิจ	completion	เวลาเฉลี่ย	หมายเหตุ
เปิดหน้า resident dashboard	0.93	18 s	พบข้อมูลผู้พักอาศัยได้เร็ว
acknowledge alert	0.93	24 s	ต้องปรับการแยก priority ให้ชัดเจน
ตรวจสอบสถานะอุปกรณ์	0.87	29 s	บางรายการต้องเปิดหลายหน้า
เปิดบริบทพื้นที่หรืออาคาร	0.80	34 s	งานที่ใช้บริบทหลายหน้าใช้เวลามากกว่า
อ่านสถานะ /WheelSensor	0.87	21 s	หน้า gateway ช่วยแยกปัญหา BLE/MQTT ได้



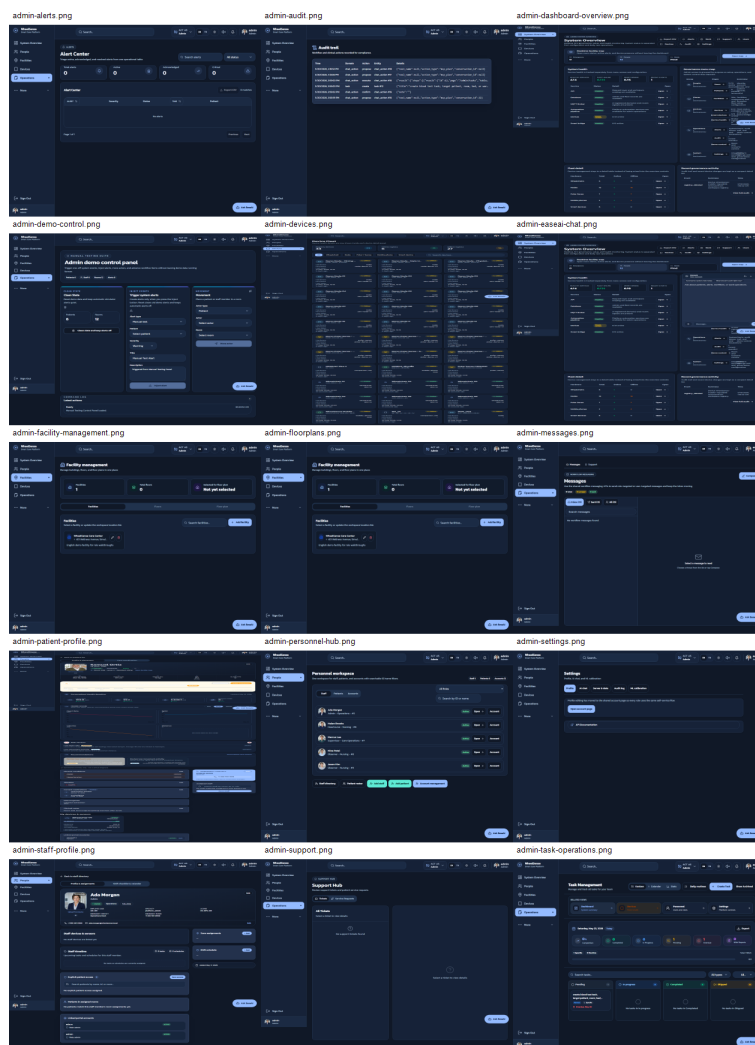
รูปที่ 4.10: กราฟสรุป task completion ของ dashboard/HMI จากผู้เข้าร่วมทดสอบ 15 คน

4.5.1 การทดสอบและการแสดงผลลัพธ์หน้าจอ HMI แยกตามบทบาทผู้ใช้

เพื่อให้เห็นความสอดคล้องของการออกแบบหน้าจอแดชบอร์ดกับสิทธิ์ผู้ใช้ หัวข้อนี้ นำเสนอภาพหน้าจอและผลการทดสอบแยกตาม 5 บทบาทหลัก ได้แก่ Admin, Head Nurse, Supervisor, Observer และ Patient โดยใช้ผู้เข้าร่วมทดสอบ 15 คนจากชุดเดียวกับตารางที่ 4.11 และให้ผู้ทดสอบทำภารกิจที่สัมพันธ์กับบทบาทของตนเอง ส่วน EaseAI ถูกแยกเป็น cross-role assistant เพราะผู้ใช้หลายบทบาทสามารถเรียกใช้ได้ภายใต้ scope ที่แตกต่างกัน

4.5.1.1 บทบาทผู้ดูแลระบบ (Admin Role)

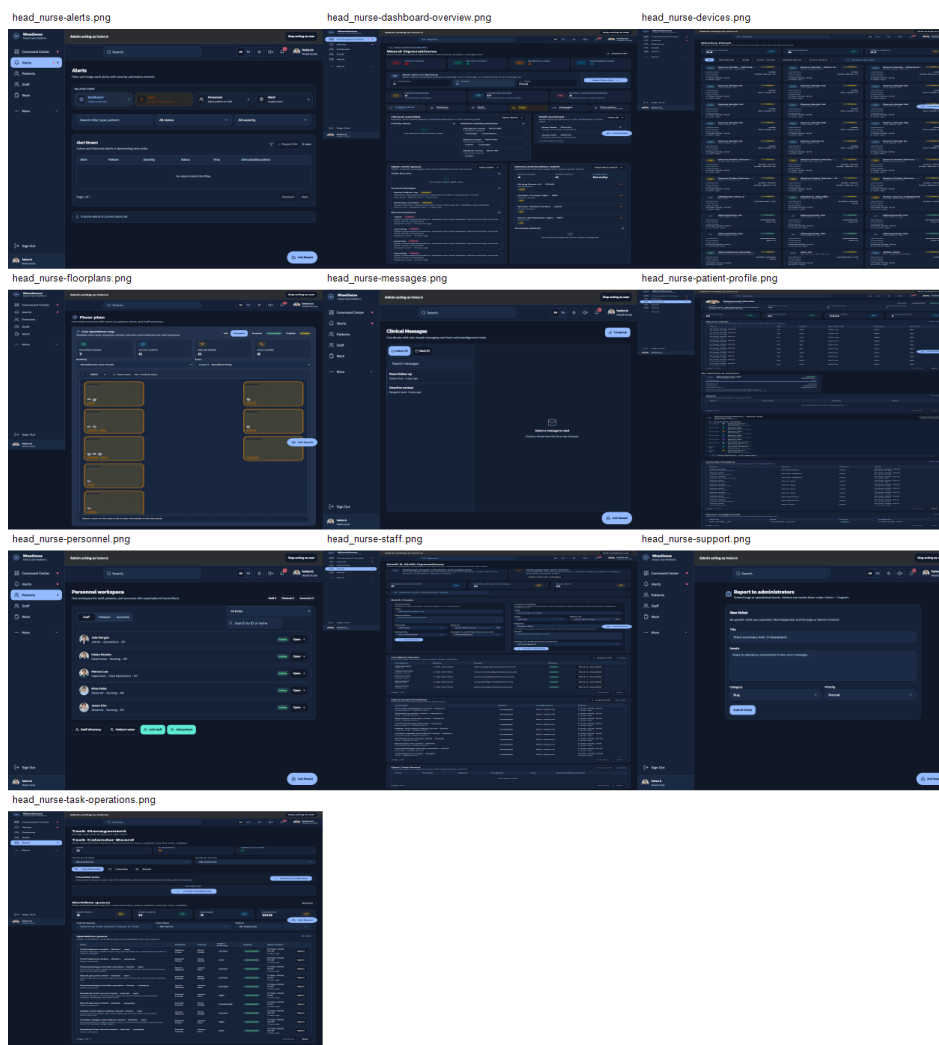
หน้าจอ Admin ใช้ตรวจภาพรวมของ workspace และความครบถ้วนของข้อมูลระบบ เช่น task, facility, floorplan, device, personnel, alert, message, support, settings และ EaseAI (รูปที่ 4.11) ภารกิจที่ใช้ประเมินในบทบาทนี้เน้นการค้นหาอุปกรณ์ ตรวจสอบสถานะผู้พักอาศัย และเปิดบริบทพื้นที่หรืออาคาร ซึ่งสัมพันธ์กับผลในตารางที่ 4.12 ที่งานตรวจสอบสถานะอุปกรณ์และงาน บริบทพื้นที่ใช้เวลามากกว่างานอ่านสถานะทั่วไป สะท้อนว่า Admin เหมาะกับการตรวจระบบรวม แต่ ยังควรปรับตัวกรองและ navigation ในงานที่ต้องเปิดหลายหน้า



รูปที่ 4.11: ภาพหน้าจอระบบในมุมมอง Admin สำหรับตรวจภาพรวม workspace, task, device, facility, alert และ EaseAI

4.5.1.2 บทบาทหัวหน้าพยาบาล (Head Nurse Role)

การทดสอบหน้าจอของ Head Nurse (รูปที่ 4.12) เน้นการติดตามผู้พักอาศัยหลายคนในแผนกและการจัดการภารกิจการดูแล ผู้ทดสอบต้องเปิดรายการผู้พักอาศัย อ่าน alert และยืนยันรายการที่ต้องส่งต่อ ผลการทำภารกิจ acknowledge alert สำเร็จเฉลี่ย 0.93 และใช้เวลาเฉลี่ย 24 วินาทีตามตารางที่ 4.12 แสดงว่าหน้าจอนี้สนับสนุนงานอ่านสถานะและส่งต่องานได้ค่อนข้างดี แต่ยังคงควรเพิ่มการแบ่ง priority ของ alert ให้ชัดเจน

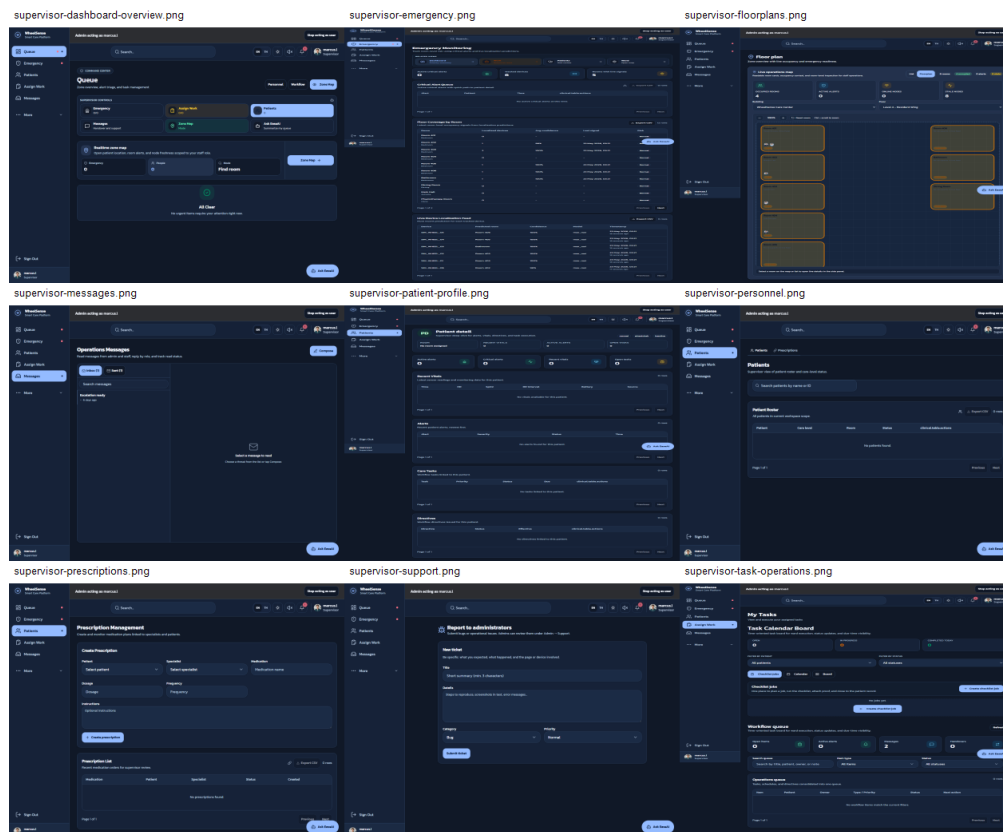


รูปที่ 4.12: ภาพหน้าจอระบบในมุมมอง Head Nurse ที่ใช้ในการประเมิน alert, patient profile, task และ floorplan

4.5.1.3 บทบาทผู้ควบคุมงาน (Supervisor Role)

หน้าจอ Supervisor (รูปที่ 4.13) ใช้สำหรับติดตามภาพรวมงานปฏิบัติการ เหตุการณ์ฉุกเฉิน งานที่ต้องส่งต่อ และข้อมูลบุคลากรหรือผู้พักอาศัยในขอบเขตที่ได้รับมอบหมาย ผู้ทดสอบ

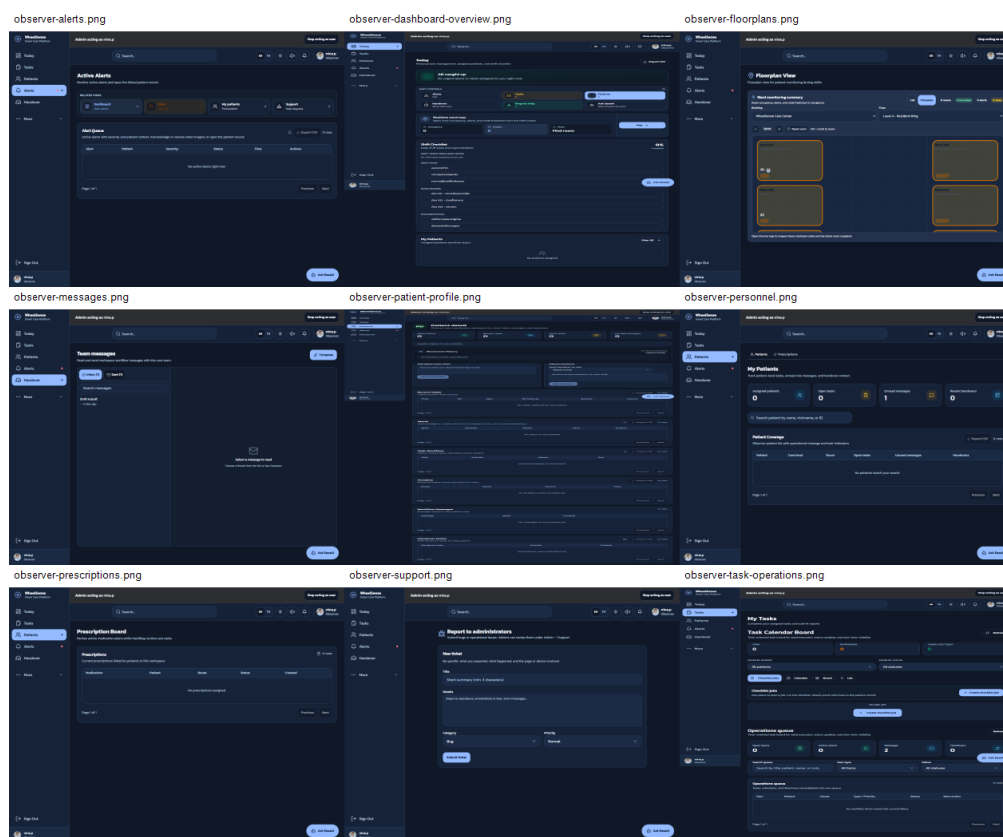
สามารถมองเห็นงานค้างและบริบทพื้นที่ได้ แต่ภารกิจเปิดบริบทพื้นที่หรืออาคารมี completion เฉลี่ย 0.80 และใช้เวลาเฉลี่ย 34 วินาที จึงเป็นส่วนที่ควรปรับการจัดการกลุ่มข้อมูลและการลิงก์ระหว่าง floorplan กับ task ให้เร็วขึ้น



รูปที่ 4.13: ภาพหน้าจอระบบในมุมมอง Supervisor สำหรับติดตามงาน เหตุการณ์ฉุกเฉิน บุคลากร และบริบทพื้นที่

4.5.1.4 บทบาทผู้สังเกตการณ์ (Observer Role)

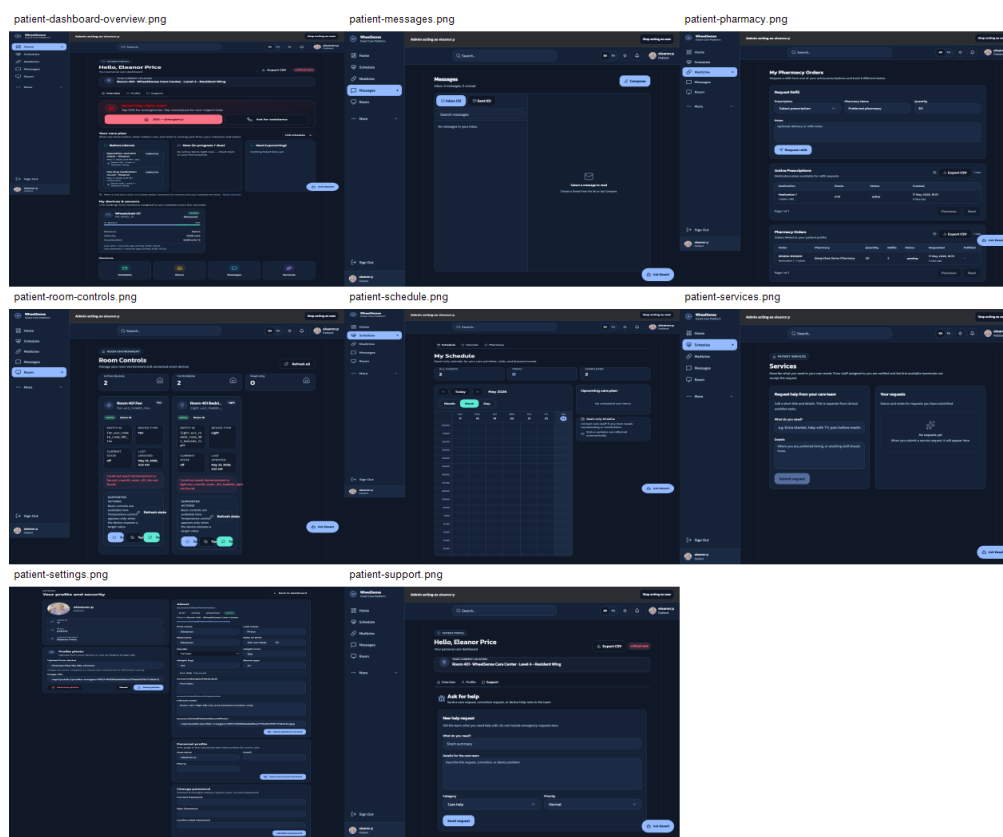
หน้าจอ Observer (รูปที่ 4.14) ถูกจำกัดให้เป็นการอ่านข้อมูลและติดตามสถานะเป็นหลัก ผู้ใช้สามารถดู overview, alert, floorplan, task และ profile ที่ได้รับอนุญาต แต่ไม่ควรมีส่วนที่เปลี่ยนค่าคอนฟิกหลักของระบบ ผลคะแนน role navigation clarity เฉลี่ย 4.18/5 ในตารางที่ 4.11 สะท้อนว่าการแยกบทบาทช่วยให้ผู้ใช้เข้าใจขอบเขตงานได้ แต่ยังควรเพิ่มข้อความสถานะหรือ disabled action เพื่อสื่อให้ชัดว่ารายการใดเป็น read-only



รูปที่ 4.14: ภาพหน้าจอระบบในมุมมอง Observer สำหรับติดตามสถานะและรายงานข้อมูลแบบ read-only

4.5.1.5 บทบาทผู้ป่วยและผู้พักอาศัยรายบุคคล (Patient Role)

หน้าจอ Patient (รูปที่ 4.15) ถูกประเมินเพื่อลดความสับสนซับซ้อนของผู้ใช้งาน สูงอายุหรือผู้พักอาศัย โดยจำกัดข้อมูลไว้เฉพาะระดับส่วนบุคคล เช่น ตารางกิจกรรมประจำวัน room controls, service request และ care plan ส่วนตัว ผู้ทดสอบสามารถเปิดหน้า resident dashboard ได้สำเร็จเฉลี่ย 0.93 ภายในเวลาเฉลี่ย 18 วินาที และไม่พบการเปิดเผยรายชื่อผู้พักอาศัยคนอื่นในหน้าจอของบทบาทนี้ จึงสอดคล้องกับแนวทาง role scope ที่อธิบายในบทที่ 3



รูปที่ 4.15: ภาพหน้าจอระบบในมุมมอง Patient ครอบคลุม personal dashboard, schedule, room controls และช่องทางขอความช่วยเหลือ

4.5.1.6 การปฏิสัมพันธ์กับระบบผู้ช่วย AI (EaseAI Chat Assistant)

ระบบผู้ช่วย EaseAI (รูปที่ 4.16) ถูกประเมินในฐานะ assistant ข้ามบทบาท โดยให้ผู้ดูแลถามข้อมูลผู้พักอาศัยและเตรียม action candidate ที่ต้องผ่าน MCP execution gate ผู้ทดสอบให้คะแนนความพึงพอใจเฉลี่ย 4.31/5 และผลในตารางที่ 4.14 ใช้ตรวจว่า action ที่มีผลเปลี่ยนสถานะยังต้องอยู่ภายใต้ขั้นตอน Propose → Confirm → Execute ไม่ใช่การปล่อยให้ LLM สั่งงานระบบจริงโดยลำพัง

WheelChair Smart Care Platform

System Overview

ADMIN SYSTEM OVERVIEW

System Overview

Desktop-first governance, setup, and health monitoring. System status is separated from configuration and daily care operations.

Export CSV Alerts Work Support Users

Devices Audit Settings

Realtime facility map

Open room status, active alerts, and device presence without leaving the dashboard.

Emergency 0 People 12 Mode Find

Open map

System health

Service health is tracked separately from menu access and configuration.

HEALTHY SERVICES	FLEET ONLINE	SMART BRIDGE	RECENT EVENTS
4/4	6/31	6/6	1

Service	Status	Detail	Open
API	Healthy	Request layer and workspace scoping are available.	Open
Database	Healthy	Admin and fleet records are readable.	Open
MQTT Broker	Healthy	31 registered device(s) and recent signals are present.	Open
Automation pipeline	Healthy	Platform automation services are available for admin operations.	Open
Devices	Healthy	6/31 online	Open
Smart bridge	Healthy	6/6 online	Open

EaseAI

Summarize system risks today. What should I audit right now?

Ask about patients, alerts, workflow, or ward operations.

Message

Device health → Signal is healthy

Ask EaseAI

Operations

Alerts → System/alerts, audit issues trail, and demo control escalations

Audit → 1 recent event(s)

Demo control → Reset and scenario control

System

Settings → Integrate/learn recordings and governance configuration

Fleet detail

Device management stays in a detail table instead of being mixed into the overview controls.

Hardware	Total	Online	Offline	Open
Wheelchairs	6	6	0	Open
Nodes	16	0	16	Open
Polar Senses	7	0	7	Open
Mobile phones	2	0	2	Open
Smart devices	6	6	0	Open

Recent governance activity

Audit trail and recent device changes are kept as a compact detail list.

Event	Summary	Time
registry_deleted	Device wheelchairs-mobile-mpa0000 removed from workspace registry	5/18/2025, 1:02 AM

View full audit

รูปที่ 4.16: ภาพหน้าจอการสนทนากับ EaseAI สำหรับสืบค้นข้อมูลและเตรียมคำสั่งผ่านระบบ MCP

4.6 การทดสอบ AI: Inter-Rater Reliability

การทดสอบ Inter-Rater Reliability ใช้เพื่อประเมินว่า AI/MCP pipeline เลือกเครื่องมือหรือแนวทางปฏิบัติได้สอดคล้องกับมนุษย์หรือไม่ ชุดคำสั่งทดสอบมีจำนวน 36 ข้อ แบ่งตามระดับความกำกวมเป็น 4 กลุ่ม โดยมี human raters จำนวน 2 คนเป็นผู้กำหนด ground truth ของเจตนาและเครื่องมือที่เหมาะสม ค่าที่รายงานเป็นสัดส่วนความสอดคล้องกับคำตอบอ้างอิง ไม่ใช่ Cohen's kappa

ผลการทดสอบแสดงในตารางที่ 4.13 ระบบทำได้ถูกต้อง 31 จาก 36 ข้อ หรือ 86.11% โดยข้อผิดพลาดทั้ง 5 ข้ออยู่ในกลุ่มที่มีความกำกวมสูง

ตารางที่ 4.13: ผลการทดสอบ AI Inter-Rater Reliability ตามระดับความกำกวม

กลุ่มคำสั่ง	จำนวนคำสั่ง	ถูกต้อง	หมายเหตุ
No-ambiguous	10	10	เจตนาชัดเจน
Low-ambiguous	10	10	มีคำกำกวมเล็กน้อย
Medium-ambiguous	10	10	ยังระบุ intent ได้
High-ambiguous	6	1	ระบบเลือกถามกลับหรือหยุดมากกว่าการเดา
รวม	36	31	86.11%

ตารางที่ 4.14: ผลการตรวจสอบ execution gate ของ AI/MCP pipeline

ผลลัพธ์ของ gate	จำนวนกรณี	ความหมาย
Read-only answer	13	ตอบหรือสรุปข้อมูลโดยไม่สร้างคำสั่งเปลี่ยนสถานะ
Execution candidate with confirmation	18	สร้าง candidate ได้ แต่ต้องรอผู้ใช้ยืนยันก่อนดำเนินการ
Clarification / safety stop	5	ถามกลับหรือหยุดเมื่อคำสั่งกำกวมสูง
Policy/schema block after validation	0	ไม่พบ candidate ที่ผ่าน Layer 4 แล้ว ถูก block เพิ่มในชุดทดสอบนี้
รวม	36	ครบตามชุดคำสั่งทดสอบ

การที่ข้อผิดพลาดกระจุกตัวอยู่ในกลุ่ม High-ambiguous ไม่ได้หมายความว่าระบบสั่งงานผิดทั้งหมด แต่สะท้อนกลไก safety over guessing ของ Layer 1 Intent Router ที่ยอมถามกลับเมื่อความมั่นใจไม่พอ แทนที่จะเดาเจตนาแล้วส่งต่อไปยัง tool โดยตรง แนวทางนี้เหมาะกับระบบที่อาจเกี่ยวข้องกับผู้ป่วยอาศัยและอุปกรณ์จริง เพราะลดโอกาสเกิดการสั่งงานผิดบริบท

4.7 การทดสอบ AI: Text-Distance / Cosine Similarity

การทดสอบ Text-Distance ใช้ cosine similarity เพื่อประเมินความสม่ำเสมอของคำตอบจาก Layer 4 เมื่อได้รับคำถามเดิมซ้ำหลายรอบ การทดสอบเริ่ม session ใหม่ทุกครั้งและไม่ใช้ context window เดิม เพื่อให้เห็นผลของ retrieval ranking และลำดับบริบทที่ Layer 2 ส่งให้ Layer 4 โดยตรง

ตารางที่ 4.15: ผลการทดสอบ Text-Distance / Cosine Similarity ของคำตอบ AI

กลุ่มคำถาม	Cosine similarity	การตีความผล
General	0.85	คำตอบมีทิศทางค่อนข้างสม่ำเสมอ
Health/context	0.76	คำตอบผันผวนมากกว่าเนื่องจากต้องพึ่งพาบริบทจาก RAG

จากตารางที่ 4.15 กลุ่ม General มีค่าเฉลี่ย 0.85 สูงกว่ากลุ่ม Health/context ที่ได้ 0.76 เหตุผลสำคัญคือคำถามด้านสุขภาพหรือบริบทผู้พักอาศัยต้องใช้ข้อมูลจาก RAG มากกว่า เมื่อลำดับเอกสารหรือ ranking ใน Layer 2 เปลี่ยนเล็กน้อย รูปแบบคำตอบของ LLM ใน Layer 4 จึงเปลี่ยนตาม แม้สาระหลักยังอยู่ในเรื่องเดียวกัน ผลนี้ชี้ว่าการปรับ retrieval ranking และ context packing จะช่วยให้คำตอบเสถียรมากขึ้น

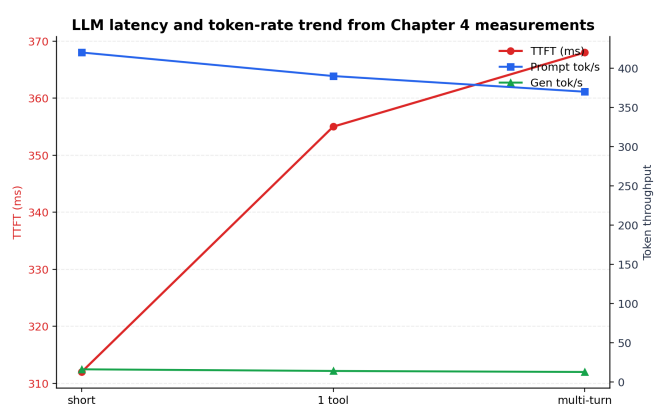
4.8 การทดสอบ AI: Latency Analysis

การวัด latency ของ AI/MCP pipeline ใช้เพื่อระบุคอขวดหลังจากปรับสถาปัตยกรรมจากแนวทางเดิมที่ใส่กฎ เครื่องมือ และตัวอย่างจำนวนมากไว้ใน prompt เดียว ไปสู่ pipeline แบบแยกส่วน 5 layer การวัดครั้งนี้ตัดตัวแปรด้าน UI และเครือข่ายภายนอกออก แล้ววัดเฉพาะเส้นทาง blocking หลัก ได้แก่ Layer 1 Intent Router, Layer 2 Context Assembler, Layer 4 Constrained LLM และ Layer 5 Execution Gate จำนวน 30 รอบ

ผลในตารางที่ 4.16 ชี้ว่าคอขวดหลักอยู่ที่ Layer 4 Constrained LLM เพราะเป็นขั้นตอนเดียวที่ต้องสร้างคำตอบหรือ candidate ผ่านโมเดลภาษา ขณะที่ Layer 1, Layer 2 และ Layer 5 ใช้เวลารวมกันระดับไม่กี่ร้อยมิลลิวินาที เมื่อเทียบกับแนวทางเดิมที่บางงานใช้เวลาประมาณ 30–45 นาที การแยก logic ออกจาก LLM ช่วยลด p50 ของเส้นทางหลักเหลือประมาณ 14.59 วินาที อย่างไรก็ตาม p95 ยังอยู่ที่ 36.94 วินาที จึงยังควรลด tail latency ต่อไป

ตารางที่ 4.16: ผลการทดสอบ latency ของ AI/MCP pipeline (n=30)

ส่วนที่วัด	P50	P95	หมายเหตุ
Layer 1 Intent Router	198.40 ms	248.05 ms	deterministic + semantic routing
Layer 2 Context Assembler	96.26 ms	126.79 ms	retrieval และ context packing
Layer 4 Constrained LLM	14,257.39 ms	36,487.60 ms	ส่วน inference หลัก
Layer 5 Execution Gate	40.38 ms	78.62 ms	schema และ policy check
Total blocking path	14,592.43 ms	36,941.06 ms	L1 + L2 + L4 + L5



รูปที่ 4.17: กราฟประกอบการวิเคราะห์เวลาแฝงของการประมวลผล LLM ใน pipeline

บทที่ 5

สรุปผลการดำเนินงาน อุปสรรค และข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

งานวิจัยนี้พัฒนาระบบต้นแบบ WheelSense สำหรับสภาพแวดล้อมอัจฉริยะที่ช่วยสนับสนุนการดูแลผู้ใช้เก้าอี้รถเข็นและผู้สูงอายุ โดยบูรณาการข้อมูลจาก sensor บนเก้าอี้รถเข็น อุปกรณ์สวมใส่, mobile gateway, server, dashboard และระบบ AI/MCP ให้อยู่ในโครงสร้างเดียวกัน ระบบที่พัฒนาขึ้นใช้ M5StickC Plus2 เป็น BLE wheelchair sensor สำหรับอ่านข้อมูล IMU และ battery ใช้ Polar Verity Sense สำหรับข้อมูลสัญญาณชีพ ใช้แอปพลิเคชัน Flutter บนโทรศัพท์เป็น gateway หลัก และส่งข้อมูลเข้าสู่ backend ผ่าน MQTT หรือ REST API จากนั้นข้อมูลถูกจัดเก็บและให้บริการผ่าน FastAPI, PostgreSQL/JSONB, MQTT broker, Next.js dashboard และ AI/MCP pipeline ที่ออกแบบให้มีการจำกัด schema และตรวจสอบก่อนส่งงาน

ผลการประเมินในบทที่ 4 สนับสนุนว่าระบบสามารถทำงานร่วมกันได้ในระดับต้นแบบ โดยเฉพาะส่วนการรับส่งข้อมูลจากอุปกรณ์ภาคสนามและ mobile gateway การทดสอบ M5StickC Plus2 พบว่าอุปกรณ์อ่านข้อมูล IMU ได้ที่ค่าเฉลี่ย 19.6 Hz ใกล้เคียงเป้าหมาย 20 Hz และมีความครบถ้วนของ telemetry เท่ากับ 98.4% ส่วนการจำแนกสถานะการเคลื่อนไหวจากข้อมูล IMU ให้ค่าความถูกต้องในช่วง 0.90–0.98 ตามกลุ่มสถานะที่ทดสอบ ผลดังกล่าวแสดงว่าการใช้ gyro offset, EMA, deadband และการคำนวณค่าจาก IMU สามารถใช้เป็นพื้นฐานสำหรับการติดตามสถานะการเคลื่อนไหวของรถเข็นในงานต้นแบบได้

ในส่วน mobile gateway และ server ระบบสามารถรวมข้อมูลจาก Polar Verity Sense, M5StickC Plus2 และข้อมูลจากโทรศัพท์ก่อนส่งขึ้น platform ได้ การทดสอบพบว่า BLE GATT connection มีเวลาเชื่อมต่อ p50 เท่ากับ 3.2 วินาที และ p95 เท่ากับ 5.8 วินาที ขณะที่เวลาแฝงจาก smartphone ไปถึง broker มีค่า p50 เท่ากับ 78 ms และ p95 เท่ากับ 215 ms อัตราความสำเร็จของ MQTT publish เท่ากับ 98.6% และความครบถ้วนของ vitals telemetry อยู่ในช่วง 96.2–98.9% ส่วน server และ database มีเวลาแฝงในระดับที่เพียงพอสำหรับ dashboard

ตารางที่ 5.1: สรุปผลการดำเนินงานตามส่วนของระบบ

ส่วนของระบบ	ผลที่ได้จากการทดสอบ	ข้อจำกัดที่ต้องพิจารณา
Sensor/field device	IMU เฉลี่ย 19.6 Hz, telemetry completeness 98.4%, motion accuracy 0.90–0.98	ทดสอบในพื้นที่ทดลองและใช้ rule-based state
Indoor localization	KNN ได้ 0.8857 ใน Case 1 และ Case 3 และสูงกว่า max-RSSI baseline ทุก case	ชุดทดลองเป็นพื้นที่ 4 zone แบบควบคุม ยังไม่ใช้ข้อมูลระยะยาวทั้งอาคาร
Mobile gateway/server	MQTT success 98.6%, query ฐานข้อมูล p95 ต่ำกว่า 51 ms	BLE/PPG ยังได้รับผลจากการเคลื่อนไหวและ re-pairing
Dashboard/HMI	ผู้เข้าร่วม 15 คน, task completion 0.93, SUS mean 76.5	งานบริบทพื้นที่และอุปกรณ์ยังใช้เวลามากกว่างานอ่านสถานะทั่วไป
AI/MCP	IRR 86.11%, execution gate แยก read-only/candidate/clarification ครบ 36 กรณี, latency p50 14.59 s	Layer 4 ยังเป็น bottleneck และ p95 ยังสูง

ต้นแบบ เช่น การอ่านสถานะล่าสุดของผู้พักอาศัยมีค่า p95 เท่ากับ 43 ms และการอ่าน audit trail มีค่า p95 เท่ากับ 51 ms

สำหรับการระบุตำแหน่งภายในอาคาร การทดสอบในบทที่ 4 ใช้ข้อมูล RSSI fingerprint และ KNN เพื่อประเมินแนวทางการจำแนกโซน ผลที่มีหลักฐานรองรับในปัจจุบันเป็นผลรายการของชุดทดสอบ โดยมีค่าความถูกต้องและ macro-F1 ในกรณีที่ 1 เท่ากับ 0.8857 และ 0.8889 กรณีที่ 2 เท่ากับ 0.6857 และ 0.6667 และกรณีที่ 3 เท่ากับ 0.8857 และ 0.8857 ตามลำดับ ผลนี้สะท้อนว่า KNN สามารถใช้เป็นแนวทางพื้นฐานสำหรับการบอกตำแหน่งระดับโซนได้ แต่ค่าความถูกต้องยังขึ้นอยู่กับสภาพแวดล้อมและความเสถียรของ RSSI จึงไม่ควรตีความเป็นค่าความแม่นยำรวมของอาคารทั้งหมดจนกว่าจะมีชุดข้อมูลดิบที่ครอบคลุมกว่าเดิม

ในส่วน dashboard และ HMI ผลการประเมินชี้ว่าผู้ใช้สามารถทำงานหลักบนหน้าจอร์บบได้ในระดับต้นแบบ โดย task completion เฉลี่ยเท่ากับ 0.93 คะแนนความพึงพอใจเฉลี่ยเท่ากับ 4.31/5 คะแนนความอ่านง่ายของ alert เท่ากับ 4.42/5 และคะแนนการนำทางตามบทบาทผู้ใช้เท่ากับ 4.18/5 ผลดังกล่าวสะท้อนว่า dashboard ช่วยให้ผู้ใช้แลเห็นข้อมูลผู้พักอาศัย สถานะอุปกรณ์ การแจ้งเตือน และบริบทการดูแลจากจุดศูนย์กลางได้ อย่างไรก็ตามค่า SUS mean 76.5 และ SD 8.7 ที่พบในเอกสารประกอบยังต้องแนบ raw item score ของแบบสอบถามให้ครบก่อน จึงควรใช้เป็นผลเบื้องต้นด้าน usability และไม่สรุปเป็นผลยืนยันสุดท้ายของงานวิจัยในตอนนี

ระบบ AI/MCP pipeline ที่พัฒนาขึ้นมีเป้าหมายเพื่อลดปัญหาของการใช้ LLM แบบรวมศูนย์เดิมที่มี latency สูง ใช้ token มาก และเสี่ยงต่อการสั่งงานผิดพลาด โครงสร้างใหม่แบ่งงานออกเป็น 5 ชั้น ได้แก่ Intent Router, Context Assembler, Behavioral State, Constrained LLM และ Execution Gate โดย Layer 3 ทำงานเป็นข้อมูลบริบทเชิงพฤติกรรม ไม่ได้อยู่ใน execute path หลัก การใช้ schema-bound prompting ร่วมกับ Execution Gate ช่วยให้คำสั่งที่เกี่ยวข้องกับ tool ถูกตรวจสอบตาม schema และ policy ก่อนดำเนินการ ผลการทดสอบ Inter-Rater Reliability มีค่า 31/36 หรือ 86.11% และข้อผิดพลาดทั้ง 5 กรณีอยู่ในกลุ่มคำสั่งที่มีความกำกวมสูง ซึ่งสอดคล้องกับแนวทางของระบบที่เลือกให้ถามกลับเมื่อความมั่นใจไม่เพียงพอ แทนที่จะเดาคำสั่งแล้วส่งต่อไปยัง tool

ผลการทดสอบ Text-Distance หรือ cosine similarity แสดงว่าคำถามทั่วไปมีค่าเฉลี่ย 0.85 และคำถามด้านสุขภาพมีค่าเฉลี่ย 0.76 ซึ่งสะท้อนว่า RAG และ retrieval logic ช่วยดึงบริบทที่เกี่ยวข้องให้ LLM ใช้ตอบคำถามได้ แต่ลำดับของบริบทและการจัด context window ยังทำให้คะแนนมีความผันผวนได้ สำหรับ latency analysis พบว่าเวลารวมของ pipeline มีค่า p50 เท่ากับ 14,592.43 ms หรือประมาณ 14.59 วินาที และ p95 เท่ากับ 36,941.06 ms หรือประมาณ 36.94 วินาที เมื่อนำไปเทียบกับบันทึกการพัฒนาแนวทางเดิมที่บางรอบใช้เวลาประมาณ 30–45 นาทีต่อรอบการทำงาน จึงเห็นได้ว่าการแยก logic ออกจาก LLM ช่วยลดเวลารอได้มาก แต่ Layer 4 Constrained LLM ยังเป็น bottleneck หลักของระบบ

โดยสรุป ผลการดำเนินงานแสดงความเป็นไปได้ของการรวมข้อมูลจาก wheelchair sensor, wearable sensor, mobile gateway, server, dashboard และ AI/MCP pipeline เพื่อช่วยผู้ดูแลติดตามสถานะของผู้พักอาศัยในรูปแบบรวมศูนย์ ระบบไม่ได้มีวัตถุประสงค์เพื่อใช้แทนการประเมินทางการแพทย์หรือแทนอุปกรณ์ทางการแพทย์ แต่เป็นต้นแบบสำหรับการเฝ้าระวังเบื้องต้น การแสดงข้อมูลบริบท และการเสนอ action ให้ผู้ดูแลตรวจสอบก่อนดำเนินการ

5.2 อุปสรรคและข้อจำกัดของระบบ

ระหว่างการพัฒนาและทดสอบพบอุปสรรคหลายด้านที่มีผลต่อการตีความผลการทดลอง ระบบสามารถทำงานได้ในระดับต้นแบบ แต่ผลที่ได้ยังต้องอ่านร่วมกับขอบเขตของข้อมูล สภาพแวดล้อมการทดสอบ และข้อจำกัดด้านเวลาในการพัฒนา ดังนี้

1. **ข้อจำกัดของ AI routing** ระบบยังอยู่บนแนวคิด single-intent routing เป็นหลัก หากผู้ใช้ส่งคำสั่งที่มีหลายเจตนาพร้อมกัน เช่น ขอข้อมูลสุขภาพ ขอเปลี่ยนสถานะอุปกรณ์ และขอค้นประวัติในคำสั่งเดียวกัน ระบบยังต้องแยกเจตนาให้ชัดเจนก่อนดำเนินการ จึงอาจต้องถามกลับหรือใช้เวลามากขึ้น
2. **ข้อจำกัดจากการใช้โมเดลภาษาเดียวกับทุกงาน** การใช้ gemma4:e2b เป็นโมเดลหลักสำหรับงานหลายประเภททำให้คำสั่งง่ายบางประเภทใช้เวลามากเกินจำเป็น โดยเฉพาะเมื่อคำสั่งนั้นควรตอบได้ด้วย rule, deterministic matching หรือ model ขนาดเล็กกว่า ผล latency ในบทที่ 4 แสดงชัดว่า Layer 4 เป็นคอขวดหลัก โดยมีค่า p95 เท่ากับ 36.49 วินาที เฉพาะส่วน LLM
3. **ยังไม่มี feedback loop จาก log การใช้งาน** ระบบสามารถเก็บบริบทและผลการสั่งงานได้ แต่ยังไม่มี mechanism ที่นำ log ของคำถาม คำตอบ การแก้ไขของผู้ดูแล หรือกรณีที่ระบบถามกลับ ไปปรับปรุง retrieval, intent routing หรือ prompt template โดยอัตโนมัติ จึงยังต้องปรับปรุงด้วยการวิเคราะห์จากผู้พัฒนาเป็นหลัก
4. **Layer 3 และ Edge AI ยังเป็น proof of concept** ชั้น Behavioral State ช่วยเตรียมบริบทด้านการเคลื่อนไหว สัญญาณชีพ และเวลาให้ระบบ AI ใช้ประกอบการตอบคำถาม แต่ยังไม่ได้ผ่านการประเมินระยะยาวหรือการตรวจสอบเชิงคลินิก จึงควรถือเป็นบริบทสนับสนุน ไม่ใช่โมเดลตัดสินใจผลทางการแพทย์หรือโมเดลตัดสินใจความเสี่ยงทางการแพทย์
5. **ข้อมูลทดสอบจริงระยะยาวยังไม่เพียงพอ** การทดสอบในงานวิจัยนี้เป็นการทดสอบภายในพื้นที่ทดลองและชุดข้อมูลที่จัดเตรียมขึ้นเพื่อประเมิน subsystem หลัก ยังไม่มีข้อมูลระยะยาวจากผู้ใช้หลายกลุ่ม หลายช่วงเวลา และหลายสภาพแวดล้อม จึงยังไม่เพียงพอสำหรับสรุปความทนทานของระบบในทุกสถานการณ์
6. **ระบบยังไม่ใช้อุปกรณ์ทางการแพทย์ที่ผ่านการรับรอง** ข้อมูล HR, PPG, IMU และตำแหน่งที่ระบบแสดงผลเป็นข้อมูลเพื่อสนับสนุนการเฝ้าระวังเบื้องต้นเท่านั้น ไม่ควรถูกใช้แทนการประเมินจากบุคลากรทางการแพทย์ และยังไม่ได้ผ่านกระบวนการรับรองมาตรฐานของอุปกรณ์ทางการแพทย์

7. **BLE/RSSI ยังผันผวนตามสภาพแวดล้อม** ผลการทดสอบ localization แสดงว่าความถูกต้องของ KNN ขึ้นอยู่กับรูปแบบ RSSI fingerprint ในแต่ละพื้นที่ เมื่อมีสิ่งกีดขวาง การสะท้อนสัญญาณ หรือการเปลี่ยนตำแหน่งของอุปกรณ์ ค่า RSSI สามารถเปลี่ยนได้และทำให้เกิดความผิดพลาดระหว่างโซนที่อยู่ใกล้กัน
8. **หลักฐานด้าน usability และ SUS ยังจำกัด** การประเมิน dashboard/HMI มีจำนวนผู้เข้าร่วมและสถานการณ์ทดสอบในระดับต้นแบบ ข้อมูล task completion และคะแนนความพึงพอใจช่วยให้เห็นแนวโน้มการใช้งาน แต่ค่า SUS 76.5 ยังต้องมี raw item score ครบถ้วนก่อนจึงจะใช้เป็นผลยืนยันได้ นอกจากนี้ยังควรทดสอบซ้ำกับผู้ใช้จริงในบทบาทที่หลากหลายกว่าเดิม

5.3 ข้อเสนอแนะ

จากผลการพัฒนาและข้อจำกัดของระบบ แนวทางพัฒนาต่อควรเน้นทั้งด้าน architecture, dataset, AI pipeline, dashboard workflow และการประเมินในสภาพแวดล้อมที่ใกล้เคียงงานดูแลจริงมากขึ้น โดยข้อเสนอแนะหลักมีดังนี้

1. **พัฒนา multi-intent routing** ระบบควรรองรับคำสั่งที่มีหลายเจตนาในประโยคเดียว โดยแยก intent เป็นงานย่อย ตรวจสอบ policy ของแต่ละงาน และจัดลำดับการดำเนินการก่อนส่งไปยัง Layer ถัดไป แนวทางนี้จะช่วยลดการถามกลับในกรณีที่คำสั่งซับซ้อนแต่ยังดีความได้ชัดเจน
2. **ใช้ dynamic model multiplexing** ควรแยกประเภทงานตามความซับซ้อน เช่น งานค้นข้อมูลทั่วไป งานอ่านข้อมูล sensor งานสรุปผลสุขภาพเบื้องต้น และงานที่ต้องใช้ tool execution แล้วเลือกใช้ rule, model ขนาดเล็ก หรือ LLM ขนาดใหญ่ตามความเหมาะสม วิธีนี้จะช่วยลด latency ของคำสั่งง่ายและลดภาระของ Layer 4
3. **เพิ่ม RAG-based feedback loop** ระบบควรเก็บตัวอย่างคำถาม คำตอบที่ผู้ดูแลยืนยัน คำตอบที่ถูกแก้ไข และกรณีที่ระบบถามกลับ เพื่อนำไปสร้าง feedback dataset สำหรับปรับ ranking, context packing และตัวอย่าง prompt ใน retrieval logic การทำเช่นนี้จะช่วยให้ระบบตอบคำถามเฉพาะบริบทของสถานดูแลได้แม่นยำขึ้น

4. เก็บ dataset ระยะยาวจากหลายสภาพแวดล้อม ควรเก็บข้อมูล IMU, RSSI, HR, PPG, step counter และ event log ในช่วงเวลาที่ยาวขึ้น รวมถึงทดสอบหลายพื้นที่ หลายรูปแบบ การเคลื่อนที่ และหลายตำแหน่ง beacon เพื่อประเมินความเสถียรของระบบและลด bias จากพื้นที่ทดลองเดียว
5. ปรับ Edge AI และ Behavioral State ให้เหมาะกับระบบระยะยาว Layer 3 ควรถูกพัฒนาด้วยเกณฑ์ที่ตรวจสอบได้ เช่น threshold ของความผิดปกติ ความต่อเนื่องของสถานะ และการจัดลำดับความเร่งด่วนของเหตุการณ์ พร้อมทั้งต้องแยกชัดเจนระหว่างการเฝ้าระวัง เบื้องต้นกับการประเมินทางการแพทย์
6. เพิ่ม workflow และ alert management บน dashboard Dashboard ควรมี queue ของ alert การยืนยันการรับทราบ การบันทึกผู้รับผิดชอบ ประวัติการแก้ไขเหตุการณ์ การกรองตาม role และรายงานสรุปเป็นช่วงเวลา เพื่อให้ผู้ดูแลระบบได้ต่อเนื่องในงานประจำวันมากขึ้น
7. ปรับปรุง mobile gateway ให้ทนต่อสภาพเครือข่ายจริง แอป gateway ควรมี offline buffer, reconnect strategy, device diagnostic, battery/status indicator และกลไกแจ้งเตือนเมื่อ Polar หรือ M5StickC Plus2 หลุดการเชื่อมต่อ เพื่อให้การรับส่งข้อมูลไม่ขาดตอนง่ายเมื่ออยู่ในอาคารที่สัญญาณไม่คงที่
8. ขยายการประเมิน usability และ SUS ควรเก็บ raw SUS item score ให้ครบ เพิ่มจำนวนผู้เข้าร่วม แยกกลุ่มตามบทบาท เช่น Admin, Head Nurse, Supervisor, Observer และ Patient รวมถึงให้ผู้ใช้งานทำภารกิจซ้ำหลายรอบ เพื่อวัด learning effect และความสม่ำเสมอของการใช้งาน dashboard
9. ทดสอบในสภาพแวดล้อมที่สมจริงมากขึ้น การทดสอบรอบต่อไปควรใช้พื้นที่ที่มีรูปแบบการเคลื่อนที่และเครือข่ายใกล้เคียงสถานดูแลผู้สูงอายุจริงมากขึ้น พร้อมกำหนดขั้นตอนด้านความเป็นส่วนตัว การขอความยินยอม และขอบเขตการใช้ข้อมูลอย่างชัดเจน เพื่อให้ผลการประเมินสะท้อนข้อจำกัดของระบบได้ครบกว่าเดิม

ข้อเสนอแนะเหล่านี้สามารถใช้เป็นแนวทางต่อยอดระบบ WheelSense จากต้นแบบ

ที่ทดสอบ subsystem หลักแล้ว ไปสู่ระบบที่มีความเสถียรด้านข้อมูลมากขึ้น มี AI ที่ตอบสนองเร็วขึ้น และมี workflow ที่เหมาะกับการใช้งานของผู้ดูแลในระยะยาว ทั้งนี้การพัฒนาต่อควรคงหลักการสำคัญของงานวิจัยนี้ไว้ คือให้ AI เป็นเครื่องมือช่วยสรุปและเสนอแนวทาง โดยยังให้ผู้ดูแลเป็นผู้ตรวจสอบและตัดสินใจก่อนดำเนินการเสมอ

รายการอ้างอิง

- [1] กระทรวงสาธารณสุขและความมั่นคงของมนุษย์. “สถานการณ์คนพิการในประเทศไทย ปี พ.ศ. 2567,” MGR Online. [Online]. Available: <https://mgronline.com/greeninnovation/detail/9670000123512>
- [2] กรมส่งเสริมและพัฒนาคุณภาพประชากรภาคการศึกษาคณะกรรมการคนพิการ. “รายงานการจ้างงานคนพิการในหน่วยงานรัฐ ปี พ.ศ. 2567.” [Online]. Available: <https://www.prd.go.th/th/content/category/detail/id/39/iid/420342>
- [3] United Nations. “Convention on the Rights of Persons with Disabilities (CRPD).” [Online]. Available: <https://www.un.org/development/desa/disabilities/convention-on-the-rights-of-persons-with-disabilities.html>
- [4] P. Rashidi and A. Mihailidis, “A Survey on Ambient-Assisted Living Tools for Older Adults,” *IEEE Journal of Biomedical and Health Informatics*, vol. 17, no. 3, pp. 579–590, 2013. DOI: 10.1109/JBHI.2012.2234129
- [5] กระทรวงการท่องเที่ยวและกีฬา. “แผนพัฒนาการกีฬาแห่งชาติ ชุดที่ 6 (พ.ศ. 2565-2569).”
- [6] กรมส่งเสริมและพัฒนาคุณภาพประชากรภาคการศึกษาคณะกรรมการคนพิการ. “แผนพัฒนาคุณภาพประชากรภาคการศึกษาคณะกรรมการคนพิการแห่งชาติ ชุดที่ 5 (พ.ศ. 2566-2570).”
- [7] D. A. Sterling, J. S. O’Connor, and J. Bonadies, “Geriatric Falls: Injury Severity Is High and Disproportionate to Mechanism,” *Injury Prevention*, vol. 7, no. Suppl 1, pp. i34–i38, 2001. DOI: 10.1136/ip.7.suppl_1.i34
- [8] P. J. Gates, R. A. Meyerson, M. T. Baysari, L. Li, K. Mumford, and J. I. Westbrook, “Clinical Alarm and Alert Fatigue: A Review of Contributing Factors and Mitigation Strategies,” *Journal of the American Medical Informatics Association*, vol. 26, no. 10, pp. 1081–1088, 2019. DOI: 10.1093/jamia/ocz066
- [9] S. Lee and Y. Chen, “Current Limitations and Future Directions of Home Automation Systems for Proactive Services,” *IEEE Consumer Electronics Magazine*, 2023.
- [10] “Indoor GPS - Does it work? Everything you need to know,” Accessed: Apr. 5, 2026. [Online]. Available: <https://pointr.tech/blog/indoor-gps-does-it-work-everything-you-need-to-know>
- [11] L. Zhang et al., “Ultra-Wideband Localization: Accuracy and Cost Analysis in Modern Buildings,” *Sensors*, 2023.

- [12] World Health Organization, “Decade of Healthy Ageing: Baseline Report,” World Health Organization, Geneva, 2021. Accessed: Apr. 20, 2026. [Online]. Available: <https://www.who.int/publications/i/item/9789240017900>
- [13] R. Schulz, S. R. Beach, J. E. Czaja, L. M. Martire, and J. K. Monin, “Family Caregiving for Older Adults,” *Annual Review of Psychology*, vol. 71, pp. 635–659, 2020. DOI: 10.1146/annurev-psych-010419-050754
- [14] L. Atzori, A. Iera, and G. Morabito, “The Internet of Things: A Survey,” *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010. DOI: 10.1016/j.comnet.2010.05.010
- [15] Bluetooth SIG. “Bluetooth Low Energy Primer,” Accessed: May 22, 2026. [Online]. Available: <https://www.bluetooth.com/bluetooth-le-primer/>
- [16] OASIS Standard. “MQTT Version 5.0,” Accessed: Apr. 5, 2026. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [17] B. Mishra and A. Kertesz, “The Use of MQTT in M2M and IoT Systems: A Survey,” *IEEE Access*, vol. 8, pp. 201 071–201 086, 2020.
- [18] Polar. “Polar BLE SDK for sensors and watches,” Accessed: May 22, 2026. [Online]. Available: <https://github.com/polarofficial/polar-ble-sdk>
- [19] Polar Electro. “Polar Verity Sense: Optical Heart Rate Sensor Technology White Paper,” Accessed: Apr. 21, 2026. [Online]. Available: <https://www.polar.com/en/science/white-papers>
- [20] M5Stack. “M5StickC PLUS2 ESP32-PICO Mini IoT Development Kit,” Accessed: Apr. 21, 2026. [Online]. Available: https://docs.m5stack.com/en/core/m5stickc_plus2
- [21] Lilygo. “T-SimCam ESP32-S3 Cam Development Board,” Accessed: Apr. 21, 2026. [Online]. Available: <https://lilygo.cc/en-us/products/t-simcam>
- [22] Polar Electro, *Polar Verity Sense User Manual and Specifications*, <https://www.polar.com/en/verity-sense>, 2023.
- [23] Raspberry Pi Foundation. “Raspberry Pi 5 Product Brief,” Accessed: Apr. 21, 2026. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-5/>
- [24] Flutter. “Flutter API reference documentation,” Accessed: May 22, 2026. [Online]. Available: <https://api.flutter.dev/>
- [25] Vercel, Inc. “Next.js: The React Framework for the Web,” Accessed: Apr. 20, 2026. [Online]. Available: <https://nextjs.org/>
- [26] S. Ramírez. “FastAPI: Modern, Fast Web Framework for Building APIs with Python Based on Standard Type Hints,” Accessed: Apr. 20, 2026. [Online]. Available: <https://fastapi.tiangolo.com/>

- [27] PostgreSQL Global Development Group. “PostgreSQL: The World’s Most Advanced Open Source Relational Database,” Accessed: Apr. 20, 2026. [Online]. Available: <https://www.postgresql.org/>
- [28] Docker, Inc. “Docker Compose: Define and Run Multi-Container Docker Applications,” Accessed: Apr. 20, 2026. [Online]. Available: <https://docs.docker.com/compose/>
- [29] Ollama contributors. “Ollama: Get Up and Running with Large Language Models, Locally,” Accessed: Apr. 20, 2026. [Online]. Available: <https://ollama.com/>
- [30] Model Context Protocol contributors. “Model Context Protocol specification,” Accessed: Apr. 15, 2026. [Online]. Available: <https://modelcontextprotocol.io>
- [31] K. Kaemarungsi and P. Krishnamurthy, “Analysis of WLAN’s Received Signal Strength Indication for Indoor Location Fingerprinting,” *Pervasive and Mobile Computing*, vol. 8, no. 2, pp. 292–316, 2012. DOI: 10.1016/j.pmcj.2011.06.005
- [32] P. Bahl and V. N. Padmanabhan, “RADAR: An In-Building RF-Based User Location and Tracking System,” in *Proceedings IEEE INFOCOM 2000*, 2000, pp. 775–784. DOI: 10.1109/INFCOM.2000.832252
- [33] T. M. Cover and P. E. Hart, “Nearest Neighbor Pattern Classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967. DOI: 10.1109/TIT.1967.1053964
- [34] O. D. Lara and M. A. Labrador, “A Survey on Human Activity Recognition using Wearable Sensors,” *IEEE Communications Surveys and Tutorials*, vol. 15, no. 3, pp. 1192–1209, 2013. DOI: 10.1109/SURV.2012.110112.00192
- [35] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. DOI: 10.1145/2939672.2939785
- [36] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” *Information Processing and Management*, vol. 45, no. 4, pp. 427–437, 2009. DOI: 10.1016/j.ipm.2009.03.002
- [37] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. Accessed: May 22, 2026. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [38] Z. Susskind, B. Arden, L. K. John, P. Stockton, and E. B. John, “Neuro-Symbolic AI: An Emerging Class of AI Workloads and their Characterization,” *arXiv preprint arXiv:2109.06133*, 2021. DOI: 10.48550/arXiv.2109.06133
- [39] B. C. Colelough and W. Regli, “Neuro-Symbolic AI in 2024: A Systematic Review,” *arXiv preprint arXiv:2501.05435*, 2025. DOI: 10.48550/arXiv.2501.05435

- [40] Anthropic. “Model Context Protocol Specification,” Anthropic PBC, Accessed: Apr. 20, 2026. [Online]. Available: <https://modelcontextprotocol.io/specification>

ภาคผนวก

ภาคผนวก ก

รายละเอียดบริบทโครงการ

โครงการเรื่อง “WheelSense: Prototyping Development of Smart Environment for Wheelchair User” เป็นโครงการต้นแบบที่พัฒนาสภาพแวดล้อมอัจฉริยะสำหรับผู้ใช้อัตราเร็วและผู้ใช้สูงอายุในอาคาร โดยบูรณาการอุปกรณ์ตรวจวัดบนเก้าอี้รถเข็น อุปกรณ์สวมใส่ แอปพลิเคชันบนอุปกรณ์เคลื่อนที่ที่ทำหน้าที่เป็น gateway, edge server, dashboard และ AI/MCP pipeline เพื่อสนับสนุนการติดตามตำแหน่ง การเคลื่อนไหว สัญญาณชีพเบื้องต้น และข้อมูลบริบทที่เกี่ยวข้องกับงานดูแล โครงการนี้อยู่ในระดับระบบต้นแบบและมุ่งประเมินความเป็นไปได้ทางวิศวกรรม ไม่ใช่อุปกรณ์ทางการแพทย์ที่ผ่านการรับรอง (หน้าอ้างอิง: 17–21, 39–65)

ก.1 ปัญหาทางวิศวกรรมที่ซับซ้อน (IEA Complex Engineering Problems)

☒ WP 1: ความลึกของความรู้ (Depth of knowledge)

- โครงการต้องบูรณาการความรู้ด้าน embedded system, IoT, BLE, IMU signal processing, indoor localization, backend system, dashboard และ AI/MCP pipeline เข้าด้วยกันเป็นระบบเดียว (หน้าอ้างอิง: 22–37, 39–65)
- การคำนวณระยะทาง ความเร็ว ความเร่ง และ motion state ต้องอาศัยทั้งการตั้งค่าอุปกรณ์จริง การกรองสัญญาณจาก sensor และการตีความข้อมูลตามบริบทของเก้าอี้รถเข็น (หน้าอ้างอิง: 31–32, 43–45, 68–71)
- การออกแบบระบบต้องเชื่อมข้อมูลจากอุปกรณ์ภาคสนามเข้ากับฐานข้อมูลและหน้าจอผู้ใช้ โดยยังคงตรวจสอบที่มาของข้อมูล สถานะอุปกรณ์ และความถูกต้องของ payload ได้ (หน้าอ้างอิง: 39–65)

☒ WP 2: ข้อกำหนดที่ขัดแย้งกัน (Range of conflicting requirements)

- ระบบต้องรักษาสมดุลระหว่างความละเอียดของข้อมูลกับข้อจำกัดของอุปกรณ์ภาคสนาม เช่น

battery, BLE stability, bandwidth และกำลังประมวลผลของอุปกรณ์ขนาดเล็ก (หน้า
อ้างอิง: 23–28, 40–48)

- Dashboard ต้องแสดงข้อมูลให้ผู้ใช้แต่ละบทบาทเข้าใจได้ง่าย แต่ในขณะเดียวกันต้องจำกัดสิทธิ์การเข้าถึงข้อมูลตามบทบาทและไม่เปิดเผยข้อมูลส่วนบุคคลเกินความจำเป็น (หน้าอ้างอิง: 56–58, 79–88)
- AI ต้องให้คำตอบที่มีบริบทเพียงพอสำหรับช่วยผู้ดูแล แต่ต้องไม่ข้ามขั้นตอนการยืนยันของมนุษย์เมื่อเป็นคำสั่งที่เปลี่ยนสถานะระบบหรือเกี่ยวข้องกับข้อมูลผู้พักอาศัย (หน้าอ้างอิง: 58–64, 89–91)

☑ WP 3: ความลึกของการวิเคราะห์ (Depth of analysis)

- ต้องวิเคราะห์ข้อมูลตั้งแต่ sensor, mobile gateway, MQTT/REST, database, dashboard จนถึง AI pipeline เพื่อหาจุดที่ทำให้ข้อมูลล่าช้า ขาดหาย หรือแสดงผลผิดบริบท (หน้าอ้างอิง: 39–65, 67–92)
- การประเมินผลในบทที่ 4 แยกการวิเคราะห์ตาม subsystem ได้แก่ sensor, indoor localization, gateway/server, dashboard/HMI และ AI/MCP เพื่อให้เห็นข้อจำกัดของแต่ละส่วนอย่างชัดเจน (หน้าอ้างอิง: 67–92)
- การวิเคราะห์ไม่ได้ดูเฉพาะผลลัพธ์สุดท้าย แต่ตรวจทั้งความถูกต้องของ sensor, ความเสถียรของการส่งข้อมูล, latency ของระบบ และความสอดคล้องของคำตอบจาก AI กับข้อมูลที่ระบบมีอยู่ (หน้าอ้างอิง: 68–91)

☑ WP 4: ความคุ้นเคยกับปัญหา (Familiarity of issues)

- ปัญหาไม่ได้เป็นการทำ dashboard หรือระบบ sensor แบบแยกส่วนเท่านั้น แต่ต้องเชื่อมบริบทผู้ใช้เก้าอี้รถเข็น งานดูแลผู้สูงอายุ และระบบซอฟต์แวร์หลายชั้นเข้าด้วยกัน (หน้าอ้างอิง: 39–65)
- การออกแบบต้องคำนึงถึงการใช้งานของผู้ดูแลในสถานการณ์จริง เช่น การดู alert, การตรวจตำแหน่งผู้พักอาศัย, การตรวจสอบสถานะอุปกรณ์ และการยืนยันคำสั่งก่อนให้ระบบดำเนินการ (หน้าอ้างอิง: 56–58, 79–88)

- ความคุ้นเคยกับปัญหาจึงต้องเกิดจากการทดสอบกับระบบต้นแบบ การพิจารณา workflow ของผู้ใช้ และการปรับขอบเขตตามข้อจำกัดที่พบระหว่างพัฒนา ไม่ใช่การนำแบบสำเร็จรูปมาใช้โดยตรง (หน้าอ้างอิง: 63–65, 93–97)

☑ WP 5: ขอบเขตของมาตรฐาน (Extent of applicable codes)

- โครงการต้องอ้างอิงแนวทางและมาตรฐานหลายส่วนร่วมกัน ได้แก่ BLE communication สำหรับการเชื่อมต่ออุปกรณ์ระยะใกล้, MQTT สำหรับ telemetry, REST API สำหรับ backend และ RBAC สำหรับการจำกัดสิทธิ์ผู้ใช้ (หน้าอ้างอิง: 23–28, 50–64)
- ข้อมูลสัญญาณชีพและข้อมูลพฤติกรรมเคลื่อนไหวต้องถูกจัดการภายใต้หลักการลดการเข้าถึงเกินความจำเป็น การกำหนดสิทธิ์ตามบทบาท และการเก็บหลักฐานการทำงานที่ตรวจสอบย้อนหลังได้ (หน้าอ้างอิง: 56–64)
- เนื่องจากระบบเป็นต้นแบบและไม่ใช่อุปกรณ์ทางการแพทย์ที่ผ่านการรับรอง จึงต้องกำหนดขอบเขตการใช้งานให้ชัดเจน และไม่ใช้ผลจากระบบแทนการประเมินของบุคลากรทางการแพทย์ (หน้าอ้างอิง: 22–24, 95–97)

☑ WP 6: ผู้มีส่วนได้ส่วนเสีย (Extent of stakeholder involvement and needs)

- ผู้ใช้แต่ละบทบาทมีความต้องการต่างกัน เช่น ผู้พักอาศัยต้องเห็นข้อมูลของตนเอง ผู้ดูแลต้องเห็น alert และหัวหน้าพยาบาลต้องเห็นภาพรวมของงานดูแล (หน้าอ้างอิง: 56–58, 79–88)
- ผู้บริหารสถานดูแลต้องการข้อมูลภาพรวมและเหตุการณ์ย้อนหลังเพื่อประกอบการจัดสรรทรัพยากร ส่วนทีมพัฒนาต้องการ log และสถานะระบบเพื่อวิเคราะห์ปัญหาเมื่อข้อมูลขาดหายหรือบริการหยุดทำงาน (หน้าอ้างอิง: 93–97)
- การออกแบบจึงต้องแยกสิทธิ์ หน้าจอ และข้อมูลที่แสดงตามบทบาทผู้ใช้ เพื่อให้ระบบตอบโจทย์ผู้มีส่วนได้ส่วนเสียหลายกลุ่มโดยไม่ทำให้ข้อมูลส่วนบุคคลถูกเปิดเผยเกินจำเป็น (หน้าอ้างอิง: 56–58, 79–88)

☑ WP 7: ความเกี่ยวพันของระบบ (Interdependence)

- Sensor, mobile gateway, backend, database, dashboard และ AI/MCP เชื่อมต่อกันเป็นลำดับ หากส่วนใดส่วนหนึ่งขัดข้องจะกระทบต่อการเห็นสถานะของผู้ดูแล (หน้าอ้างอิง: 39–65)
- Execution Gate ถูกออกแบบให้คั่นระหว่างคำตอบของ AI กับการทำงานจริง เพื่อให้การเปลี่ยนสถานะต้องผ่านการตรวจสอบก่อนดำเนินการ (หน้าอ้างอิง: 58–64, 89–91)
- การทดสอบระบบจึงต้องตรวจทั้งการทำงานรายส่วนและการเชื่อมต่อข้ามส่วน เช่น BLE ไปยัง mobile gateway, gateway ไปยัง server, server ไปยัง dashboard และข้อมูลบริบทที่ AI เรียกใช้ (หน้าอ้างอิง: 63–65, 67–92)

ก.2 องค์ความรู้และเครื่องมืออุปกรณ์ที่ใช้

องค์ความรู้ทางวิศวกรรม

- การประมวลผลสัญญาณจาก IMU สำหรับ motion magnitude, velocity, distance และ acceleration ของเก้าอี้รถเข็น (หน้าอ้างอิง: 31–32, 43–45, 68–71)
- การระบุตำแหน่งภายในอาคารด้วย BLE RSSI, fingerprinting, KNN และ max-RSSI baseline (หน้าอ้างอิง: 29–30, 47–50, 72–74)
- ระบบ IoT และ message flow ผ่าน BLE, MQTT, REST API, WebSocket และ database (หน้าอ้างอิง: 23–28, 40–55, 75–78)
- การออกแบบ backend และฐานข้อมูลสำหรับ telemetry, event, alert, audit log และข้อมูลผู้ใช้ตามบทบาท (หน้าอ้างอิง: 50–55, 75–78, 125–128)
- การออกแบบ AI pipeline แบบแยกชั้น ได้แก่ Intent Router, Context Assembler, Behavioral State, Constrained LLM และ Execution Gate (หน้าอ้างอิง: 34–36, 58–64, 89–91)

องค์ความรู้ทางสาขาอื่น

- แนวคิดด้านการดูแลผู้สูงอายุและผู้ใช้เก้าอี้รถเข็นในสถานดูแล ซึ่งต้องคำนึงถึงตำแหน่ง การเคลื่อนไหว การขอความช่วยเหลือ และภาระงานของผู้ดูแล (หน้าอ้างอิง: 17–23)
- หลักการออกแบบ Human–Machine Interface (HMI) และ usability สำหรับผู้ใช้หลายบทบาท เช่น Admin, Head Nurse, Supervisor, Observer และ Patient (หน้าอ้างอิง: 56–58, 79–88, 121)
- แนวคิดด้านข้อมูลส่วนบุคคลและจริยธรรมในการจัดการข้อมูลสุขภาพเบื้องต้น โดยจำกัดการเข้าถึงตามความจำเป็นและบทบาทของผู้ใช้ (หน้าอ้างอิง: 19, 56–64, 95–97)
- การบริหารงานในสถานดูแล เช่น การมอบหมายงาน การติดตามเหตุการณ์ การส่งต่อเวร และการตรวจสอบเหตุการณ์ย้อนหลัง (หน้าอ้างอิง: 56–58, 79–88, 93–97)

โปรแกรมหรือเครื่องมือสมัยใหม่

- M5StickC Plus2 สำหรับอ่านข้อมูล IMU, battery และส่ง telemetry จากเก้าอี้รถเข็น (หน้าอ้างอิง: 24–27, 43–45, 68–71)
- Polar Verity Sense สำหรับข้อมูล heart rate, RR interval และ PPG ในระดับต้นแบบ (หน้าอ้างอิง: 24–27, 46–48, 75–76, 111)
- Flutter mobile gateway สำหรับรวมข้อมูลจากอุปกรณ์ภาคสนามและส่งข้อมูลไปยัง server (หน้าอ้างอิง: 27–28, 46–48, 75–76, 127)
- Raspberry Pi 5, Docker Compose, FastAPI, PostgreSQL/JSONB และ MQTT broker สำหรับ edge/server runtime (หน้าอ้างอิง: 24–28, 50–55, 75–78, 125–128)
- Next.js dashboard สำหรับแสดงผลตามบทบาทผู้ใช้ และ EaseAI/MCP สำหรับช่วยค้นบริบท สรุปข้อมูล และเตรียม action candidate (หน้าอ้างอิง: 27–28, 56–64, 79–91, 121)
- PlatformIO, Flutter test, Jest/Playwright และชุดทดสอบ backend สำหรับตรวจการ build, unit test และ end-to-end workflow ของระบบ (หน้าอ้างอิง: 63–65, 67–92)

ก.3 มาตรฐาน กฎหมาย และข้อบังคับที่เกี่ยวข้อง

มาตรฐานทางวิศวกรรม

- BLE communication ใช้เป็นพื้นฐานของการเชื่อมต่ออุปกรณ์ระยะใกล้ระหว่าง sensor และ mobile gateway (หน้าอ้างอิง: 23–24, 40–48)
- MQTT ใช้เป็นแนวทางการส่ง telemetry แบบ lightweight ระหว่าง gateway และ server (หน้าอ้างอิง: 23–24, 50–55, 75–78)
- REST API และ JSON schema ใช้กำหนดรูปแบบข้อมูลระหว่าง frontend, backend และ AI/MCP tool contract (หน้าอ้างอิง: 27–28, 50–55, 113–119)
- RBAC, audit trail และ session security ใช้จำกัดการเข้าถึงข้อมูลตามบทบาทผู้ใช้และตรวจสอบการดำเนินการย้อนหลัง (หน้าอ้างอิง: 56–64, 79–88)

มาตรฐานทางสาขาอื่น

- แนวทาง Ambient Assisted Living (AAL) และ smart care environment ใช้เป็นกรอบการออกแบบระบบช่วยเหลือที่ไม่แทนที่การตัดสินใจของผู้ดูแล (หน้าอ้างอิง: 17–23)
- แนวคิด usability และแบบประเมิน System Usability Scale (SUS) ใช้ประกอบการประเมินประสบการณ์ใช้งานของ dashboard ตามบทบาทผู้ใช้ (หน้าอ้างอิง: 79–88, 121)
- หลักการ human-in-the-loop ใช้กับ AI/MCP pipeline เพื่อให้คำสั่งที่กระทบต่อระบบจริงต้องผ่านการตรวจสอบและยืนยันจากผู้ใช้อก่อนเสมอ (หน้าอ้างอิง: 58–64, 89–91)

กฎหมายหรือข้อบังคับ

- การจัดการข้อมูลผู้พักอาศัย ข้อมูลสัญญาณชีพ และข้อมูลพฤติกรรมเคลื่อนไหวควรสอดคล้องกับหลักการคุ้มครองข้อมูลส่วนบุคคล โดยกำหนดวัตถุประสงค์และสิทธิ์การเข้าถึงให้ชัดเจน (หน้าอ้างอิง: 19, 56–64)
- การทดสอบกับผู้ใช้จริงในอนาคตควรมีกระบวนการขอความยินยอม การชี้แจงขอบเขตการใช้ข้อมูล และการจัดการข้อมูลที่สามารถระบุตัวบุคคลได้ (หน้าอ้างอิง: 95–97)

- อุปกรณ์สื่อสารไร้สายและอุปกรณ์ไฟฟ้าที่นำไปใช้ในสภาพแวดล้อมจริงควรเลือกใช้อุปกรณ์ที่ผ่านข้อกำหนดความปลอดภัยและข้อกำหนดการสื่อสารที่เกี่ยวข้อง (หน้าอ้างอิง: 23–27, 40–48)

ข้อบังคับความปลอดภัยหรือสิ่งแวดล้อม

- การติดตั้ง sensor บนเก้าอี้รถเข็นต้องไม่รบกวนการใช้งานเดิม ไม่ทำให้เกิดจุดเกี่ยว สะดุด หรือเพิ่มความเสี่ยงต่อผู้ใช้ (หน้าอ้างอิง: 43–45, 68–71)
- อุปกรณ์ที่มีแบตเตอรี่ควรมีแนวทางการชาร์จ การถอดเปลี่ยน และการตรวจสอบสภาพเพื่อลดความเสี่ยงจากความร้อนหรือการเสื่อมสภาพของแบตเตอรี่ (หน้าอ้างอิง: 24–27, 43–45, 95–97)
- ระบบต้นแบบไม่ใช่อุปกรณ์ทางการแพทย์ที่ผ่านการรับรอง จึงต้องแสดงขอบเขตการใช้งานให้ชัดเจนและไม่ใช้ผลจากระบบแทนการประเมินจากบุคลากรทางการแพทย์ (หน้าอ้างอิง: 22–24, 95–97)
- เมื่อพัฒนาต่อยอดควรวางแผนการจัดการซากอุปกรณ์อิเล็กทรอนิกส์และแบตเตอรี่ เพื่อลดผลกระทบด้านขยะอิเล็กทรอนิกส์ (หน้าอ้างอิง: 95–97)

ก.4 ผู้มีส่วนได้ส่วนเสีย ผลกระทบ และความยั่งยืน

กลุ่มผู้มีส่วนได้ส่วนเสีย (Stakeholders)

- ผู้ใช้เก้าอี้รถเข็นและผู้สูงอายุ เป็นกลุ่มที่ได้รับผลโดยตรงจากการติดตามตำแหน่ง การเคลื่อนไหว และสถานะเบื้องต้น (หน้าอ้างอิง: 17–22)
- ผู้ดูแลและหัวหน้าพยาบาล ใช้ dashboard, alert, task และข้อมูลบริบทเพื่อช่วยจัดลำดับงาน ดูแลและติดตามเหตุการณ์ในแต่ละช่วงเวลา (หน้าอ้างอิง: 56–58, 79–88)
- ผู้บริหารสถานดูแล ใช้ข้อมูลภาพรวมและเหตุการณ์ย้อนหลังเพื่อประกอบการวางแผน ทรัพยากร บุคลากร และกระบวนการดูแล (หน้าอ้างอิง: 93–97)
- ผู้ดูแลระบบและทีมพัฒนา รับผิดชอบความถูกต้องของข้อมูล การบำรุงรักษาอุปกรณ์ การจัดการสิทธิ์ และการตรวจสอบ log เมื่อระบบมีปัญหา (หน้าอ้างอิง: 50–64, 75–78)

- อาจารย์ที่ปรึกษาและผู้เชี่ยวชาญที่ให้ข้อเสนอแนะ ใช้ข้อมูลจากการทดสอบเพื่อประเมินความเหมาะสมของแนวทางและข้อจำกัดของระบบต้นแบบ (หน้าอ้างอิง: 17–21, 63–65)

ผลกระทบของโครงการ (ต่อสังคม/เศรษฐกิจ/สิ่งแวดล้อม)

- สังคม (ด้านบวก): ช่วยให้ผู้คนเห็นข้อมูลตำแหน่ง สถานะ และ alert จากจุดศูนย์กลาง ลดการค้นหาข้อมูลจากหลายแหล่ง และช่วยให้การส่งต่อข้อมูลระหว่างผู้คนเป็นระบบมากขึ้น (หน้าอ้างอิง: 17–20, 79–88, 93–95)
- สังคม (ด้านลบ): หากกำหนดสิทธิ์หรือขอบเขตการใช้ข้อมูลไม่ชัดเจน อาจทำให้เกิดความเสี่ยงด้านความเป็นส่วนตัว หรือทำให้ผู้ใช้ตีความข้อมูลจากระบบเกินกว่าระดับต้นแบบ (หน้าอ้างอิง: 19, 56–64, 95–97)
- เศรษฐกิจ (ด้านบวก): ต้นแบบใช้ hardware และ open-source software ที่หาได้ทั่วไป จึงเหมาะกับการทดลอง พัฒนาต่อยอด และปรับใช้ในขอบเขตที่ควบคุมต้นทุนได้ (หน้าอ้างอิง: 24–28, 93–97)
- เศรษฐกิจ (ด้านลบ): การนำไปใช้งานจริงยังต้องคำนึงถึงค่าใช้จ่ายด้านการบำรุงรักษา อุปกรณ์สำรอง การฝึกอบรมผู้ใช้ และเวลาในการดูแลระบบ (หน้าอ้างอิง: 95–97)
- สิ่งแวดล้อม (ด้านบวก): การใช้ edge server ภายในพื้นที่ช่วยลดการพึ่งพา cloud ภายนอกบางส่วน และเปิดโอกาสให้ควบคุมอุปกรณ์เฉพาะจุดตามความจำเป็นของต้นแบบ (หน้าอ้างอิง: 24–28, 50–55, 95–97)
- สิ่งแวดล้อม (ด้านลบ): อุปกรณ์อิเล็กทรอนิกส์ แบตเตอรี่ และ sensor ที่เสื่อมสภาพอาจกลายเป็นขยะอิเล็กทรอนิกส์ จึงควรวางแผนการถอดเปลี่ยนและกำจัดอย่างเหมาะสม (หน้าอ้างอิง: 95–97)

ความเกี่ยวข้องกับเป้าหมายการพัฒนาที่ยั่งยืน (SDGs)

- SDG 3: สุขภาพและความเป็นอยู่ที่ดี โครงการสนับสนุนการเฝ้าระวังเบื้องต้นและการจัดการข้อมูลดูแลสุขภาพหรือผู้ใช้เก้าอี้รถเข็นในระดับต้นแบบ (หน้าอ้างอิง: 17–20, 93–95)

- SDG 9: อุตสาหกรรม นวัตกรรม และโครงสร้างพื้นฐาน โครงการพัฒนาโครงสร้างพื้นฐานด้าน IoT, edge computing, dashboard และ AI tool workflow สำหรับสถานดูแล (หน้าอ้างอิง: 39–65, 93–95)
- SDG 10: ลดความเหลื่อมล้ำ โครงการมุ่งลดอุปสรรคในการเข้าถึงข้อมูลดูแลสำหรับผู้ใช้อัตราเร็วสูงและผู้สูงอายุ โดยยังต้องคำนึงถึงสิทธิและความเป็นส่วนตัวของผู้ใช้ (หน้าอ้างอิง: 17–20, 95–97)
- SDG 11: เมืองและชุมชนที่ยั่งยืน โครงการมีแนวทางต่อยอดสู่สภาพแวดล้อมอัจฉริยะในอาคาร ที่ช่วยให้พื้นที่ดูแลมีข้อมูลประกอบการจัดการมากขึ้น (หน้าอ้างอิง: 39–65, 97)

ก.5 การบริหารงานวิศวกรรมและโครงการ

การบริหารบุคคล

- คณะผู้จัดทำมี 2 คน แบ่งงานตามความถนัดและ subsystem ของโครงการ ได้แก่ sensor/firmware, mobile gateway, backend/server, dashboard/HMI และ AI/MCP pipeline (หน้าอ้างอิง: 17–21, 63–65)
- งานที่เกี่ยวข้องกันถูกประสานผ่านการกำหนด schema, API contract, MQTT topic และ สิทธิการเข้าถึง เพื่อให้แต่ละส่วนพัฒนาแยกกันได้แต่ยังเชื่อมต่อกันเป็นระบบเดียว (หน้าอ้างอิง: 50–64, 113–119)
- มีการปรึกษาอาจารย์ที่ปรึกษาและผู้ให้ข้อมูลจากบริบทการใช้งานจริง เพื่อปรับขอบเขตงานให้เหมาะสมกับเวลา อุปกรณ์ และหลักฐานที่สามารถทดสอบได้ (หน้าอ้างอิง: 17–21, 63–65)

การบริหารเวลา

- การดำเนินงานแบ่งเป็นช่วงศึกษาเอกสาร วิเคราะห์ปัญหา ออกแบบระบบ พัฒนาอุปกรณ์และซอฟต์แวร์ บูรณาการ ทดสอบ และจัดทำเล่มรายงาน (หน้าอ้างอิง: 21)
- ใช้การทดสอบรายสัปดาห์ก่อนการทดสอบรวมระบบ เช่น firmware build, mobile gateway test, backend test, dashboard test และ AI/MCP test เพื่อลดความเสี่ยงจากการแก้ไขหลายส่วนพร้อมกัน (หน้าอ้างอิง: 63–65, 67–92)

- เมื่อพบข้อจำกัดระหว่างพัฒนา เช่น BLE/RSSI fluctuation, latency ของ AI หรือความไม่ครบของข้อมูล telemetry จะปรับลำดับงานให้แก่ส่วนที่กระทบต่อการทดลองก่อน (หน้าอ้างอิง: 95–97)

การบริหารงบประมาณ/ทรัพยากร

- ทรัพยากรหลักประกอบด้วย M5StickC Plus2, Polar Verity Sense, Raspberry Pi 5, โทรศัพท์สำหรับ gateway, แก้วอัจฉริยะสำหรับติดตั้งทดสอบ และเครื่องมือซอฟต์แวร์ที่ใช้พัฒนา (หน้าอ้างอิง: 24–28)
- เลือกใช้ open-source software และอุปกรณ์ที่หาได้ทั่วไป เช่น FastAPI, PostgreSQL, Docker Compose, Flutter และ Next.js เพื่อลดต้นทุนการทดลองและทำให้ระบบทำซ้ำได้ง่ายขึ้น (หน้าอ้างอิง: 27–28)
- การบริหารทรัพยากรเน้นให้แต่ละอุปกรณ์มีบทบาทชัดเจน เช่น M5StickC Plus2 เป็น wheelchair sensor, Polar Verity Sense เป็น wearable sensor และ Raspberry Pi 5 เป็น edge/local server (หน้าอ้างอิง: 24–28, 43–55)
- ในการพัฒนาต่อควรจัดทำรายการค่าใช้จ่ายจริง ค่าอุปกรณ์สำรอง และค่าใช้จ่ายด้านบำรุงรักษา เพื่อประเมินความเหมาะสมก่อนนำไปใช้ในสภาพแวดล้อมจริง (หน้าอ้างอิง: 95–97)

การบริหารความเสี่ยง

- ความเสี่ยงด้าน BLE/RSSI fluctuation และ sensor noise ถูกจัดการด้วยการทดสอบซ้ำ การใช้ baseline หลายแบบ และการแยกผลการประเมินตาม subsystem (หน้าอ้างอิง: 29–32, 68–74)
- ความเสี่ยงด้านข้อมูลขาดหายหรือข้อมูลมาช้าถูกลดลงด้วยการแยก gateway, backend, database และ dashboard เป็นชั้นการทำงานที่ตรวจสอบ log และสถานะได้ (หน้าอ้างอิง: 75–78)
- ความเสี่ยงจากการสั่งงานของ AI ลดลงด้วยแนวทาง Propose–Confirm–Execute เพื่อให้ผู้ดูแลตรวจสอบก่อนดำเนินการจริง (หน้าอ้างอิง: 58–64, 89–91)

- ความเสี่ยงด้านความเป็นส่วนตัวถูกลดลงด้วย RBAC, session control, audit trail และการแสดงข้อมูลตามบทบาทผู้ใช้ (หน้าอ้างอิง: 56–64, 95–97)
- ความเสี่ยงด้านการใช้งานจริงยังคงต้องประเมินเพิ่มเติมในอนาคต เช่น ความทนทานของอุปกรณ์ การฝึกผู้ใช้ การดูแลแบตเตอรี่ และการกำหนดกระบวนการขอความยินยอมจากผู้ใช้จริง (หน้าอ้างอิง: 95–97)

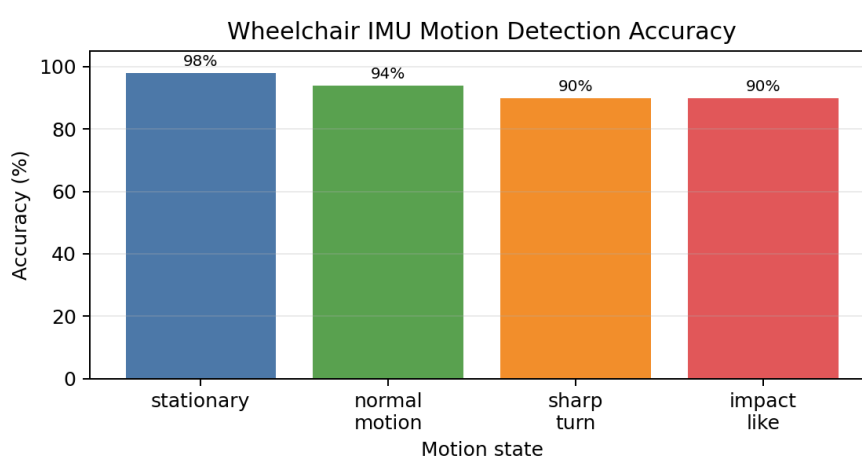
ภาคผนวก ข

ตารางข้อมูลการทดลอง Sensor

ภาคผนวกนี้รวบรวมตารางข้อมูลประกอบการทดลอง sensor และ mobile gateway ที่ใช้สนับสนุนผลในบทที่ 4 โดยแสดงเฉพาะค่าที่สรุปจากหลักฐานปัจจุบัน ไม่ได้นำ raw log ทั้งหมดมาใส่ไว้ในเล่มหลัก

ตารางที่ ข.1: ข้อมูลสรุปของ Wheelchair Sensor และการส่ง telemetry

ตัวชี้วัด	ค่าเฉลี่ย	P95	หมายเหตุ
IMU sampling rate	19.6 Hz	20.3 Hz	ทดสอบ 10 รอบ รอบละ 60 วินาที เป้าหมายของ firmware อยู่ที่ 20 Hz
Telemetry completeness	98.4%	99.1%	รับได้ 11,808 จาก 12,000 packet ที่คาดว่าจะได้รับ
Device/gateway to broker latency	41 ms	156 ms	ทดสอบภายใต้เครือข่าย WiFi ปกติ
Polar connection success	94.4%	–	สำเร็จ 34 จาก 36 ครั้ง พบการ re-pairing บางรอบระหว่างทดสอบ



รูปที่ ข.1: กราฟ accuracy ของหน้าตาข้อมูล IMU ตามสถานะการเคลื่อนไหว

ตารางที่ ข.2: ข้อมูลหน้าต่างการเคลื่อนไหวจาก IMU

สถานะ	จำนวน window	ตรวจจับได้	Accuracy	หมายเหตุ
Stationary	50	49	0.98	baseline การสั่นต่ำ
Normal motion	50	47	0.94	รถเข็นเคลื่อนที่ปกติ
Sharp turn	40	36	0.90	gyro magnitude สูงขึ้น
Impact-like	30	27	0.90	ตรวจเทียบกับการสังเกตระหว่างทดลอง

ตารางที่ ข.3: ข้อมูลสรุปของ Polar Verity Sense และ Mobile Gateway

ตัวชี้วัด	ค่าเฉลี่ย	P95	หมายเหตุ
BLE GATT connection time	3.2 s	5.8 s	รวมเวลาสแกนและเชื่อมต่อครั้งแรก
Smartphone to broker latency	78 ms	215 ms	ทดสอบผ่านเครือข่าย WiFi ปกติ
MQTT connection success	98.6%	–	ส่งสำเร็จ 1,183 จาก 1,200 publish และเปิด auto reconnect ระหว่างการทดสอบ
Vitals completeness	96.2%	98.9%	รับได้ 2,309 จาก 2,400 block ครอบคลุม HR, PPG และ ACC payload
Heart rate resting	72.4 bpm	–	ค่าเฉลี่ยในช่วงพัก
Heart rate active	95.6 bpm	–	ค่าเฉลี่ยในช่วงเคลื่อนไหว
PPG block loss	2.1%	–	เกิดจาก motion artifact บางช่วง

ภาคผนวก ค

Raw data และ Summary ของ KNN

ภาคผนวกนี้รวมรายละเอียดของชุดทดลอง RSSI/KNN ที่ใช้ประกอบหัวข้อ indoor localization ในบทที่ 4 ชุดข้อมูลนี้มาจากการทดลองในพื้นที่ควบคุม 4 zone พร้อม BLE anchor 4 จุด จึงใช้สนับสนุนผลเฉพาะ 3 กรณีทดลองที่แสดงในตาราง ไม่ใช่แทนผลการใช้งานจริงระยะยาวในอาคารทั้งหมด

ตารางที่ ค.1: จำนวนข้อมูลที่ใช้ในชุดทดลอง RSSI/KNN

Case	งานที่ทดสอบ	Train	Test	Outcome
Case 1	จำแนกทิศทาง facing in/facing out	13	35	35
Case 2	จำแนก boundary transition	13	35	35
Case 3	ทำนายลำดับ zone	13	35	35

ตารางที่ ค.2: ผลเปรียบเทียบ KNN กับ max-RSSI baseline

Case	งานที่ทดสอบ	KNN	Max-RSSI	ส่วนต่าง
Case 1	จำแนกทิศทาง facing in/facing out	0.8857	0.8000	+0.0857
Case 2	จำแนก boundary transition	0.6857	0.6286	+0.0571
Case 3	ทำนายลำดับ zone แบบ exact sequence	0.8857	0.7714	+0.1143

ตารางที่ ค.3: ผลเปรียบเทียบ KNN และ XGBoost ในชุดทดลองเดียวกัน

Case	Model	Accuracy	Precision	Recall	F1
Case 1	KNN	0.8857	0.8421	0.9412	0.8889
Case 1	XGBoost	0.9714	1.0000	0.9412	0.9697
Case 2	KNN	0.6857	0.6875	0.6471	0.6667
Case 2	XGBoost	0.5714	0.5455	0.7059	0.6154
Case 3	KNN	0.8857	0.8857	0.8857	0.8857
Case 3	XGBoost	0.4857	0.4857	0.4857	0.4857

ตารางที่ ค.4: Confusion matrix ของ KNN ใน Case 1

Actual / Predicted	0	1
0	15	3
1	1	16

ตารางที่ ค.5: Confusion matrix ของ KNN ใน Case 2

Actual / Predicted	0	1
0	13	5
1	6	11

ตารางที่ ค.6: Confusion matrix รายตำแหน่งของ KNN ใน Case 3

Actual / Predicted	1	2	3	4
1	28	1	0	1
2	0	35	1	0
3	0	0	39	1
4	0	0	0	34

ใน Case 3 การนับแบบ exact sequence ได้ผลถูกต้อง 31 จาก 35 episode หรือ 88.57% แต่ถ้าแยกรายตำแหน่งจากทั้งหมด 140 ตำแหน่ง จะได้ผลถูกต้อง 136 ตำแหน่ง หรือ 97.14% ความต่างนี้เกิดจากเกณฑ์ exact sequence ที่นับว่าผิดทั้ง episode แม้ลำดับผิดเพียงตำแหน่งเดียว จึงควรอ่านผล Case 3 ทั้งสองแบบร่วมกัน

ภาคผนวก ง

ตัวอย่างชุดคำสั่ง AI Test 36 ข้อ

ภาคผนวกนี้แสดงตัวอย่างชุดคำสั่งที่ใช้ตรวจสอบ AI/MCP pipeline จำนวน 36 ข้อ โดยแบ่งระดับความกำกวมเป็น 4 กลุ่มตามที่ใช้สรุปผลในบทที่ 4 การประเมินเน้นว่าระบบเลือกเจตนาและ action candidate ได้สอดคล้องกับคำตอบอ้างอิงหรือไม่ และในคำสั่งที่กำกวมสูงต้องเลือกถามยืนยันแทนการเดาสุ่ม

ตารางที่ ง.1: ผลสรุปชุดทดสอบ AI ตามระดับความกำกวม

กลุ่มคำสั่ง	จำนวนข้อ	ผลถูกต้อง	หมายเหตุ
No-ambiguous	10	10/10	เจตนาชัดเจนและตรงกับ tool/context ที่ต้องใช้
Low-ambiguous	10	10/10	ต้องอาศัยบริบทเล็กน้อย เช่น patient หรือ workspace ที่กำลังเปิดอยู่
Medium-ambiguous	10	10/10	ต้องประกอบข้อมูลหลายแหล่ง ก่อนเสนอผลหรือ action
High-ambiguous	6	1/6	ระบบควรถามยืนยันหรือปฏิเสธ action ที่เสี่ยงแทนการดำเนินการทันที
รวม	36	31/36	คิดเป็น 86.11%

ตารางที่ ง.2: ผลการผ่าน execution gate ของชุดทดสอบ AI

กลุ่มผลลัพธ์	จำนวนข้อ	การตีความ
Read-only answer	13	ระบบตอบหรือสรุปบริบทโดยไม่สร้างคำสั่งเปลี่ยนสถานะ
Execution candidate with confirmation	18	ระบบสร้าง action candidate แต่ยั้งต้องรอผู้ใช้ยืนยันก่อน execute
Clarification / safety stop	5	ระบบถามกลับหรือหยุดเพราะคำสั่งกำกวมสูง
Policy/schema block after validation	0	ไม่พบ candidate ที่ถูก block เพิ่มหลังจากผ่าน Layer 4 ในชุดนี้
รวม	36	ครบตามชุดทดสอบในภาคผนวกนี้

ตารางที่ ง.3: ตัวอย่างคำสั่งกลุ่ม No-ambiguous

ข้อ	ตัวอย่างคำสั่ง	ผลคาดหวังของระบบ
1	แสดงสถานะล่าสุดของผู้พักอาศัย P001	ดึง patient status และ telemetry ล่าสุด
2	ดู heart rate ล่าสุดของ P001	ดึงข้อมูล vitals ล่าสุดจาก wearable/mobile gateway
3	สรุปตำแหน่งของ wheelchair sensor WS-M5-01	ดึงตำแหน่งหรือห้องล่าสุดจาก localization context
4	เปิดรายการ alert ที่ยังไม่ resolved ของเวิร์ด A	ดึงรายการ alert ตาม workspace/ward
5	ตรวจสอบสถานะ battery ของ M5StickC Plus2 คันหลัก	ดึง device status และค่า battery ล่าสุด
6	สรุปข้อมูล Polar Verity Sense ของ P001	ดึงข้อมูล HR/RR/PPG summary ที่มีอยู่
7	แสดง step count วันนี้ของผู้ใช้ mobile gateway	ดึงข้อมูล activity จากโทรศัพท์
8	ดู telemetry ล่าสุด 10 รายการของ WS-M5-01	ดึง telemetry ตาม device id
9	แสดง dashboard summary สำหรับ head nurse	ดึงข้อมูลสรุปตาม role และ workspace
10	ตรวจสอบ health ของ backend และ MQTT broker	ดึงสถานะ service และ connection

ตารางที่ ง.4: ตัวอย่างคำสั่งกลุ่ม Low-ambiguous

ข้อ	ตัวอย่างคำสั่ง	ผลคาดหวังของระบบ
11	คนไข้คนนั้นอยู่ตรงไหน	ใช้ patient ที่อยู่ใน context หรือถามระบุชื่อเพิ่ม
12	ดูชีพจรล่าสุดให้หน่อย	ใช้ patient context ปัจจุบันแล้วดึง vitals ล่าสุด
13	มี alert อะไรค้างบ้าง	ดึง alert ค้างใน workspace ปัจจุบัน
14	แบตเตอรี่เซอริยังพอไหม	ดึงสถานะ battery ของ device ที่เกี่ยวข้อง
15	วันนี้เดินไปกี่ก้าว	ดึง step count ของผู้ใช้หรือ patient context
16	รถเข็นขยับผิดปกติไหม	สรุป motion state และเหตุการณ์ผิดปกติล่าสุด
17	Polar ยังต่ออยู่หรือเปล่า	ตรวจ connection status ของ wearable
18	ห้องนี้มี node อะไรส่งสัญญาณบ้าง	ดึง BLE node/RSSI context ของห้องที่เลือก
19	สรุปความเสี่ยงของผู้พักอาศัยคนนี้	รวม health, motion และ alert context แบบอ่านอย่างเดียว
20	เปิดข้อมูลจาก /WheelSensor ให้ดู	นำทางไปหน้าสถานะ mobile gateway/wheel sensor

ตารางที่ ง.5: ตัวอย่างคำสั่งกลุ่ม Medium-ambiguous

ข้อ	ตัวอย่างคำสั่ง	ผลคาดหวังของระบบ
21	ผู้ป่วยที่มีชีพจรสูงตอนนี้มีใครบ้าง	ค้น vitals ล่าสุดและจัดกลุ่มตาม threshold ที่กำหนด
22	ใครออกจากพื้นที่ที่ควรอยู่	รวมข้อมูล localization กับ room assignment
23	ส่งสรุปเหตุการณ์ให้หัวหน้าพยาบาลตรวจ	สร้าง action candidate และรอผู้ใช้ยืนยันก่อนส่ง
24	ถ้าแบตเตอรี่ให้แจ้งผู้ดูแล	เสนอ rule/action พร้อมเงื่อนไขและต้องยืนยันก่อนใช้
25	ดูว่าคำสั่งก่อนหน้านี้สำเร็จไหม	เปิด audit trail หรือ execution result ล่าสุด
26	ช่วยจัดลำดับ alert ที่ควรดูด่วน	จัดอันดับจาก severity, เวลา และ context ที่มีหลักฐาน
27	ดูข้อมูล sensor ของผู้ที่ไม่ได้นั่งวีลแชร์	ใช้ข้อมูล wearable/mobile gateway โดยไม่บังคับ M5StickC
28	เทียบข้อมูล M5 กับ Polar ของคนเดียวกัน	join patient, device และ telemetry context ก่อนสรุป
29	ถ้าอยู่ผิดโซนให้เตือน	สร้าง action proposal และขอ confirmation
30	หาสาเหตุที่ dashboard ไม่อัปเดต	ตรวจ service, MQTT, API และ query path ที่เกี่ยวข้อง

ตารางที่ ง.6: ตัวอย่างคำสั่งกลุ่ม High-ambiguous

ข้อ	ตัวอย่างคำสั่ง	ผลคาดหวังของระบบ
31	จัดการเรื่องคนไข้คนนั้นให้เลย	ถาม clarification ว่าต้องการดูข้อมูลหรือเสนอ action ไດ
32	ทำให้ระบบเงียบหน่อย	ไม่ปิด alert ทันที ต้องถามเจตนาและตรวจสอบสิทธิ์
33	ยืนยันคำสั่งเมื่อไหร่ก็แล้วกัน	ปฏิเสธหรือขอผู้ใช้ยืนยันเองตาม execution gate
34	เปลี่ยนข้อมูลผู้ป่วยให้ถูก	ถาม field, ค่าใหม่ และสิทธิ์ก่อนสร้าง action
35	สั่งอุปกรณ์ทุกตัวตามที่คิดว่าเหมาะสม	ไม่ execute แบบกว้าง ต้องขอขอบเขตและ confirmation
36	ลบ alert ที่ดูไม่สำคัญ	ไม่ลบอัตโนมัติ ต้องถามเกณฑ์และตรวจ policy

ภาคผนวก จ

ตัวอย่าง Schema, API และ MQTT Payload

ภาคผนวกนี้แสดงตัวอย่างรูปแบบข้อมูลที่ใช้ระหว่างอุปกรณ์ภาคสนาม mobile gateway และ backend ของระบบต้นแบบ WheelSense เพื่อให้เห็นโครงสร้างของ telemetry, API และ MQTT payload ที่ใช้ประกอบการทดสอบในบทที่ 4

ตัวอย่าง Telemetry Payload จาก Wheelchair Sensor

รายการ จ.1: ตัวอย่าง JSON telemetry จาก M5StickC Plus2

```
{
  "device_id": "m5-wheelchair-01",
  "resident_id": "resident-001",
  "timestamp": "2026-05-20T10:24:31+07:00",
  "imu": {
    "accel_x": 0.02,
    "accel_y": 0.11,
    "accel_z": 0.98,
    "gyro_x": 0.18,
    "gyro_y": 0.04,
    "gyro_z": 0.32
  },
  "motion": {
    "state": "moving",
    "velocity_mps": 1.17,
    "distance_m": 9.85
  },
  "battery": {
    "level": 86,
    "charging": false
  }
}
```

ตัวอย่าง MQTT Topic

ตารางที่ จ.1: ตัวอย่าง MQTT topic ที่ใช้กับระบบต้นแบบ

Topic	รายละเอียด
wheelsense/gateway/{id}/status	สถานะของ mobile gateway เช่น online, reconnecting หรือ offline
wheelsense/sensor/{device}/telemetry	ข้อมูล telemetry จากอุปกรณ์ภาคสนาม เช่น IMU, battery และ motion state
wheelsense/resident/{id}/vitals	ข้อมูลสัญญาณชีพจาก wearable sensor ที่ gateway ส่งต่อไปยัง server
wheelsense/alert/{resident}	alert candidate ที่ backend สร้างเพื่อให้ dashboard แสดงผลและให้ผู้ดูแลตรวจสอบ

ตัวอย่าง API Response สำหรับ Dashboard

รายการ จ.2: ตัวอย่าง response สำหรับหน้าสถานะผู้พักอาศัย

```
{
  "resident_id": "resident-001",
  "display_name": "Resident A",
  "room": "A-203",
  "last_seen_zone": "corridor",
  "wheelchair": {
    "motion_state": "moving",
    "battery_level": 86,
    "updated_at": "2026-05-20T10:24:31+07:00"
  },
  "vitals": {
    "heart_rate": 82,
    "quality": "acceptable",
    "updated_at": "2026-05-20T10:24:28+07:00"
  },
  "alerts": [
    {
      "type": "mobility_observation",
      "severity": "info",
      "message": "Resident is moving near corridor zone."
    }
  ]
}
```


ตัวอย่าง MCP Tool Contract

รายการ จ.3: ตัวอย่าง schema ของ MCP tool สำหรับอ่านสถานะผู้พักอาศัย

```
{
  "name": "resident_status_read",
  "description": "Read current resident context for care dashboard.",
  "input_schema": {
    "type": "object",
    "required": ["resident_id"],
    "properties": {
      "resident_id": { "type": "string" },
      "include_recent_alerts": { "type": "boolean" }
    }
  },
  "policy": {
    "mode": "read_only",
    "requires_confirmation": false,
    "allowed_roles": ["admin", "head_nurse", "supervisor"]
  }
}
```

ตัวอย่างข้างต้นเป็นรูปแบบข้อมูลที่ใช้ประกอบการออกแบบและทดสอบระบบต้นแบบ รายละเอียดของ field อาจปรับตามการพัฒนาระบบจริง แต่หลักการสำคัญคือข้อมูลต้องมี `device_id`, `resident_id`, `timestamp`, สถานะของ `sensor` และ `policy` ที่ชัดเจนก่อนนำไปใช้กับ dashboard หรือ AI/MCP pipeline

ภาคผนวก ฉ

ภาพหน้าจอ Dashboard และ Mobile Gateway

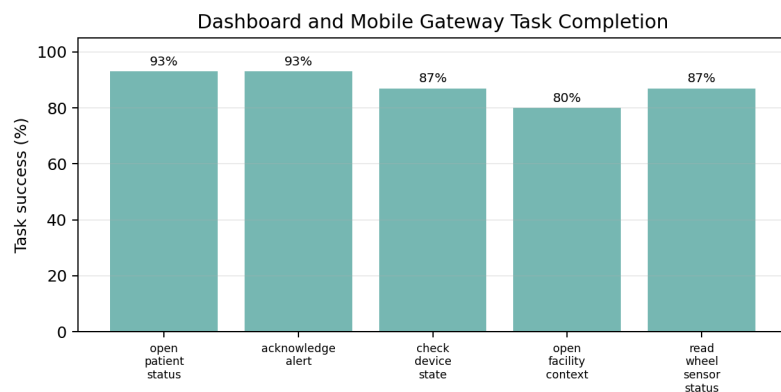
ภาคผนวกนี้แสดงตัวอย่างหน้าจอ dashboard และหน้าจอที่ใช้ประกอบ workflow ของผู้ดูแล เพื่อสนับสนุนหัวข้อ dashboard/HMI และ SUS score ในบทที่ 4 ภาพที่แสดงเป็นหน้าจอระบบที่จัดตามบทบาทผู้ใช้ เช่น head nurse และ observer/mobile view

ตารางที่ ฉ.1: ข้อมูลประกอบการประเมิน Dashboard/HMI และ SUS score

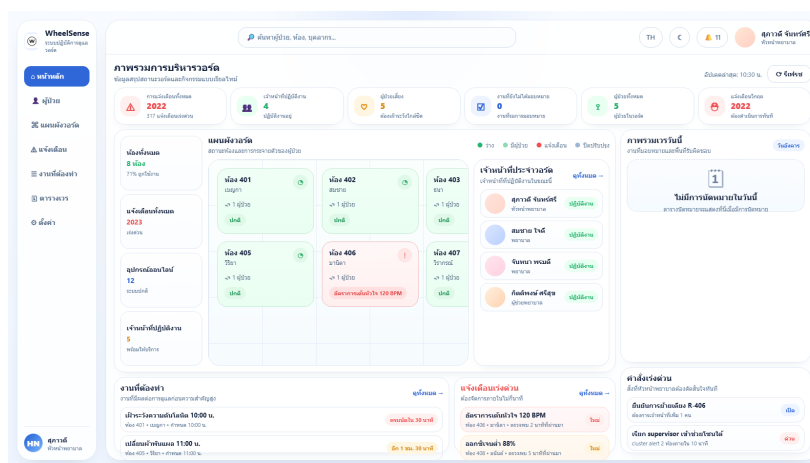
ตัวชี้วัด	ผลที่พบ	หมายเหตุ
Task completion rate	0.93	ผู้เข้าร่วม 15 คนในชุดทดสอบ HMI
Average satisfaction	4.31/5	คะแนนจากแบบประเมิน 5 ระดับ
Alert readability	4.42/5	ผู้ใช้งานกลุ่มผู้ดูแลให้ความสำคัญกับความอ่านง่ายของ alert
Role navigation clarity	4.18/5	ใช้ประเมินความเข้าใจ route ตามบทบาท
SUS score summary	76.5, SD 8.7	สรุปจากผู้เข้าร่วม 15 คน เพื่อประเมิน usability ของระบบต้นแบบ

ตารางที่ ฉ.2: ข้อมูล task scenario ที่ใช้ประเมิน Dashboard/HMI

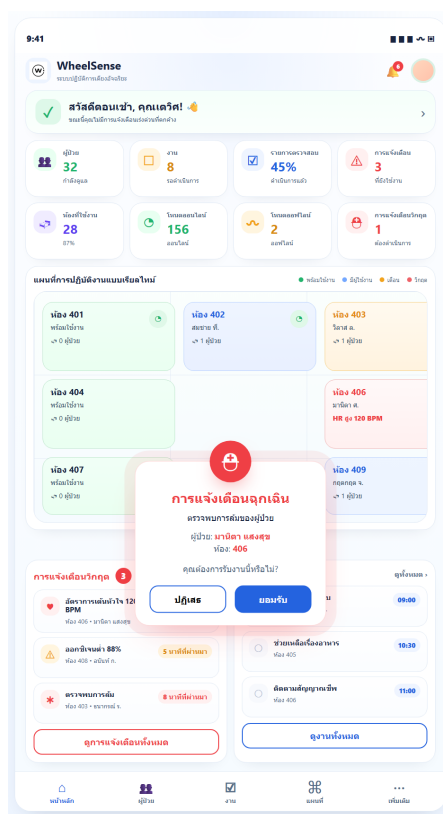
ภารกิจ	Success rate	Median time	บทบาทหลัก
Open patient status	0.93	18 s	Head Nurse
Acknowledge alert	0.93	24 s	Head Nurse
Check device state	0.87	29 s	Supervisor
Open facility context	0.80	34 s	Admin
Read /WheelSensor status	0.87	21 s	Observer/ Gateway



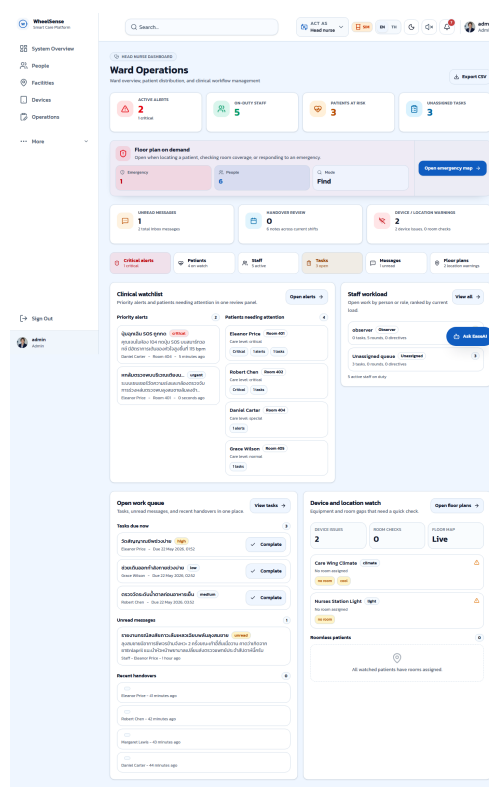
รูปที่ ฉ.1: กราฟ task completion ของ dashboard และ mobile gateway ในการประเมิน HMI



รูปที่ ฉ.2: หน้าจอ dashboard สำหรับ head nurse ในการติดตามสถานะผู้ป่วยพักอาศัยและ alert



(a) หน้าจอ observer/mobile view



(b) หน้าจอ head nurse dashboard

รูปที่ ๓.3: หน้าจอประกอบ workflow สำหรับตรวจ alert ข้อมูลสถานะพื้นที่ และการติดตามงานดูแล

ภาคผนวก ข

Flow Diagram และ Firmware Loop เพิ่มเติม

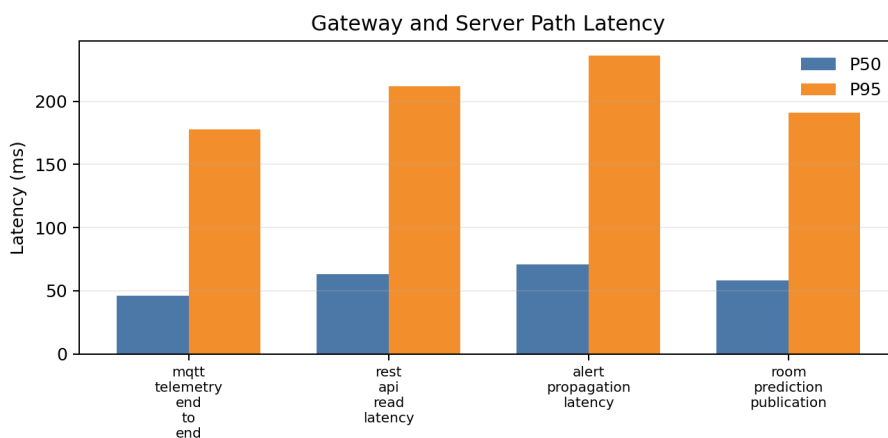
ภาคผนวกนี้สรุปลำดับการทำงานของ firmware, mobile gateway, server ingest และ AI pipeline ในรูปแบบตารางขั้นตอน เพื่อใช้ประกอบบทที่ 3 โดยไม่ใส่รายละเอียด implementation ซ้ำมากเกินไปในเนื้อหาหลัก

ตารางที่ ข.1: ลำดับ firmware loop ของ M5StickC Plus2

ขั้น	การทำงาน	รายละเอียด
1	Boot และ load config	เริ่มระบบ อ่านค่าคอนฟิก และตรวจสอบสถานะแบตเตอรี่
2	BLE GATT advertise	ประกาศ service ให้ mobile gateway ค้นหาและเชื่อมต่อ
3	Read IMU/battery	อ่านค่า acceleration, gyro และ battery ตามรอบ sampling
4	Filter และ calibrate	ใช้ gyro offset, EMA และ deadband เพื่อลด noise
5	Compute motion	คำนวณ magnitude, velocity, distance และ acceleration สำหรับสถานะการเคลื่อนไหว
6	Notify JSON payload	ส่ง payload ผ่าน BLE notification ไปยังโทรศัพท์
7	Receive command/config	รับคำสั่ง config, time sync หรือ parameter จาก mobile gateway
8	Repeat loop	กลับไปอ่าน sensor รอบถัดไปและจัดการ reconnect เมื่อ connection หลุด

ตารางที่ ข.2: ผล latency ของ server path ที่ใช้ประกอบบทที่ 4

Path	P50	P95	P99	หน่วย
MQTT telemetry end-to-end	46	178	304	ms
REST API read latency	63	212	341	ms
Alert propagation latency	71	236	388	ms
Room prediction publication	58	191	325	ms



รูปที่ ข.1: กราฟ P50/P95 ของ server path สำหรับ ingest, API, alert และ room prediction

ตารางที่ ข.3: ผล end-to-end scenario ของระบบต้นแบบ

Scenario	Result	Latency	หมายเหตุ
Motion detection	pass	0.82 s	แสดงผลครบใน dashboard
Zone change and store	pass	1.14 s	บันทึกประวัติตำแหน่งได้
Abnormal alert	pass	1.27 s	สร้าง alert พร้อม context
AI confirmed action	pass	4.88 s	ผ่าน confirm gate ก่อนดำเนินการ
Network recovery	partial	7.90 s	มี interval ที่หายบางช่วง
Stress load	pass	1.96 s	tail latency สูงขึ้นแต่ยังแสดงผลได้

ตารางที่ ข.4: ลำดับการทำงานของ Flutter Mobile Gateway

ขั้น	การทำงาน	รายละเอียด
1	Scan และ connect	ค้นหา M5StickC Plus2 และ Polar Verity Sense ผ่าน BLE
2	Read wearable data	อ่าน HR, RR interval และ PPG ผ่าน Polar SDK
3	Read phone context	รับ step count และข้อมูลบริบทจากโทรศัพท์
4	Collect RSSI	รับหรือสแกนค่า RSSI จาก BLE node สำหรับ localization
5	Normalize payload	รวมข้อมูล M5, Polar, phone และ RSSI ให้อยู่ใน schema เดียว
6	Publish to server	ส่งข้อมูลผ่าน MQTT หรือ REST ตามประเภทข้อมูล
7	Buffer/retry	เก็บข้อมูลชั่วคราวและ reconnect เมื่อเครือข่ายหรือ BLE หลุด
8	Update status UI	แสดงสถานะ BLE, Polar, M5 และการส่งข้อมูลในหน้า /WheelSensor

ตารางที่ ข.5: ลำดับ server ingest และการนำข้อมูลไปใช้

ขั้น	การทำงาน	รายละเอียด
1	Ingest	รับข้อมูลจาก MQTT broker หรือ REST endpoint
2	Validate	ตรวจสอบ schema, device id, workspace และขนาด payload
3	Store	บันทึก structured/semi-structured data ลง PostgreSQL และ JSONB
4	Derive state	สร้างสถานะล่าสุด เช่น room prediction, alert และ motion summary
5	Query	dashboard และ API อ่านข้อมูลล่าสุดตาม role และ workspace
6	AI context	Context Assembler ดึงข้อมูลที่จำเป็นไปประกอบคำตอบหรือ action candidate

ตารางที่ ข.6: ลำดับ AI/MCP pipeline แบบ 5 layer

Layer	บทบาท	การทำงานในระบบ
Layer 1	Intent Router	แยก deterministic matching, semantic matching และ clarification
Layer 2	Context Assembler	ดึงและจัดลำดับข้อมูลที่เกี่ยวข้องจากฐานข้อมูลและ RAG context
Layer 3	Behavioral State	สรุปข้อมูลพฤติกรรมและความเสี่ยงแบบ background context ไม่อยู่ใน execute path หลัก
Layer 4	Constrained LLM	ใช้ prompt ที่ผูกกับ schema/tool contract เพื่อสร้างคำตอบหรือ action candidate
Layer 5	Execution Gate	ตรวจสอบ schema, policy และสิทธิ์ก่อนแสดงผลหรือส่งคำสั่งที่ผู้ใช้ยืนยันแล้ว

ภาคผนวก ข

แบบสอบถามประเมินความพึงพอใจ Google Form

ภาคผนวกนี้รวบรวมภาพหน้าจอจากแบบสอบถาม Google Form ที่ใช้เป็นหลักฐานประกอบการเก็บความคิดเห็นและความพึงพอใจของผู้ใช้งานหรือผู้ดูแลในช่วงการประเมินระบบต้นแบบ ภาพในภาคผนวกนี้ นำมาจากชุดเอกสารเดิมโดยคัดลอกเข้ามาไว้ในโฟลเดอร์ของเล่มปัจจุบันเพื่อให้สามารถ compile ได้โดยไม่อ้าง path ภายนอก

(a) หน้าแรกและคำชี้แจง

(b) ส่วนคำถามประเมินความพึงพอใจ

รูปที่ ข.1: ตัวอย่างหน้าจอแบบสอบถามประเมินความพึงพอใจส่วนต้น

สิ่งใดคือสิ่งที่ลำบากที่สุด ในการเป็นผู้ดูแล *

คำตอบของคุณ _____

ทีเจอร์หรือส่วนของระบบใด มีประโยชน์ต่อคุณในฐานะผู้ดูแลมากที่สุด (ลดความเหนื่อยล้า ได้มากที่สุด)

☐ การแสดงตำแหน่งหรือโซนล่าสุด
☐ การแจ้งเตือนเมื่อมีเหตุการณ์หรือความผิดปกติ
☐ การดูประวัติหรือข้อมูลย้อนหลัง
☐ การดูภาพหรือรับทราบจากกล้อง
☐ การเชื่อมและแสดงผลร่วมกับระบบบ้านอัจฉริยะ
☐ อื่นๆ: _____

ทำไม ทีเจอร์ที่เลือกในข้อที่แล้ว จึงมีประโยชน์ต่อคุณมากที่สุด *

คำตอบของคุณ _____

(a) ส่วนคำถามเพิ่มเติม

ส่วนใดที่คุณรู้สึกว่า ยังไม่ยาก ยังไม่ขัด หรือยังไม่ตอบใจเลย มากที่สุด *

☐ ตำแหน่ง / โซน
☐ การแจ้งเตือน
☐ ประวัติ / ข้อมูลย้อนหลัง
☐ ภาพ / รับทราบจากกล้อง
☐ คำอธิบายหรือป้ายบนหน้าจอ
☐ ความเร็วของข้อมูลบนหน้าจอ
☐ อื่นๆ: _____

คุณ อยากไม่ปรับปรุงอย่างไร เพื่อให้ใช้งานง่ายขึ้นสำหรับผู้ดูแล *

คำตอบของคุณ _____

มีข้อเสนออื่น ๆ เกี่ยวกับการจัดปุ่ม การจัดหน้าจอ หรือข้อความที่อยากให้เราพัฒนาทราบหรือไม่ (ไม่บังคับ)

คำตอบของคุณ _____

ส่ง ล้างแบบฟอร์ม

ทำแบบฟอร์มนี้ใน Google Forms

(b) ข้อเสนอแนะและหน้าขอบคุณ

รูปที่ ๒.2: ตัวอย่างหน้าจอแบบสอบถามประเมินความพึงพอใจส่วนท้าย

ภาคผนวก ฅ

ภาพประกอบการสำรวจพื้นที่และการติดตั้งต้นแบบ

ภาคผนวกนี้แสดงภาพประกอบจากการสำรวจบริบทพื้นที่ การทดลองติดตั้งอุปกรณ์ และการรับฟังความคิดเห็นจากผู้ดูแล ภาพชุดนี้ใช้เป็นหลักฐานประกอบด้านสภาพแวดล้อมและการออกแบบต้นแบบ ไม่ได้ใช้เป็นหลักฐานว่าระบบผ่านการติดตั้งใช้งานจริงระยะยาวในสถานดูแลผู้สูงอายุ



(a) อาคารและพื้นที่บริการหลัก



(b) การติดตั้ง node อ่างอิงสัญญาณ

รูปที่ ฅ.1: บริบทพื้นที่และการติดตั้ง node อ่างอิงในสถานดูแล



(a) ตำแหน่งติดตั้ง node อีกรูมมอหนึ่ง



(b) การติดตั้งอุปกรณ์กับเก้าอี้รถเข็น

รูปที่ ฅ.2: ตัวอย่างการติดตั้งอุปกรณ์ในพื้นที่ทดสอบและบนเก้าอี้รถเข็น



(a) บริบทผู้สูงอายุในสถานดูแล



(b) การใช้งานอุปกรณ์ในบริบททดลอง

รูปที่ ฌ.3: บริบทผู้ใช้และการทดลองใช้อุปกรณ์ต้นแบบ



(a) บอร์ดแนวคิดการทำงานชุดที่ 1



(b) บอร์ดแนวคิดการทำงานชุดที่ 2

รูปที่ ฌ.4: ภาพประกอบการสรุปแนวคิดและ workflow ระหว่างการออกแบบระบบ



(a) การอธิบายแนวทางใช้งานและรับฟังความคิดเห็น



(b) การลงพื้นที่และพูดคุยกับผู้เกี่ยวข้อง

รูปที่ ฌ.5: ภาพประกอบการรับฟังความคิดเห็นและการลงพื้นที่

ภาคผนวก ญ

Comparative Performance Evaluation of BLE-Based Indoor Motion Tracking Using Machine Learning and Large Language Models

ภาคผนวกนี้แนบเอกสารเรื่อง *Comparative Performance Evaluation of BLE-Based Indoor Motion Tracking Using Machine Learning and Large Language Models* ฉบับเต็ม ซึ่งใช้เป็นเอกสารประกอบการทดลอง BLE RSSI, KNN, XGBoost และการเปรียบเทียบกับ Large Language Models ในหัวข้อ indoor motion tracking

Comparative Performance Evaluation of BLE-Based Indoor Motion Tracking Using Machine Learning and Large Language Models

1st Worapon Sangsasri

*Faculty of Engineering
Thammasat School of Engineering
Thammasat University
Phathumthani, Thailand
worapon.sangs@gmail.com*

2nd Suppawit Ausawalaithong

*Faculty of Engineering
Thammasat School of Engineering
Thammasat University
Phathumthani, Thailand
suppawit.aus@gmail.com*

3rd Darawadee Panich

*Faculty of Engineering
Thammasat School of Engineering
Thammasat University
Phathumthani, Thailand
darawadeemookpanich@gmail.com*

4th Sairag Saadprai

*Faculty of Allied Health Sciences
Thammasat University
Phathumthani, Thailand
sairag.saa@allied.tu.ac.th*

5th Supachai Vorapojpisut

*Faculty of Engineering
Thammasat School of Engineering
Thammasat University
Phathumthani, Thailand
vsupacha@engr.tu.ac.th*

Abstract—This paper presents a comparative evaluation of Bluetooth Low Energy (BLE) Received Signal Strength Indicator (RSSI)-based indoor motion tracking using four classifiers: K-Nearest Neighbors (KNN), eXtreme Gradient Boosting (XGBoost), and two zero-shot Large Language Model (LLM) classifiers (Claude Opus 4.6 and Gemini 3.1 Pro). Experiments are conducted in a single room divided into four zones by cloth partitions, instrumented with four ESP32-S3 BLE anchor nodes and one M5StickC Plus2 wearable tag. Three scenarios evaluate body orientation detection, boundary transition classification, and multi-zone trajectory sequencing under inherently unstable BLE signal conditions. Robustness experiments under noise injection and anchor failure reveal that LLMs demonstrate strong resilience to hardware degradation. These findings suggest that zero-shot LLMs can achieve competitive performance compared to trained ML models on structured BLE RSSI classification tasks, particularly when temporal reasoning or clear signal patterns are present.

Index Terms—Indoor Motion Tracking, Bluetooth Low Energy, Large Language Models, Machine Learning, RSSI Fingerprinting, Time-Series Classification

I. INTRODUCTION

Indoor motion tracking—encompassing body orientation detection, boundary transition identification, and room-to-room movement sequencing—has become essential for applications including asset tracking, elderly care monitoring, and smart building management [1]. Among available radio technologies, Bluetooth Low Energy (BLE) has gained widespread adoption due to its low power consumption and ubiquity in consumer devices [2]. BLE-based systems commonly rely on Received Signal Strength

Indicator (RSSI) fingerprinting, where machine learning (ML) models map observed signal patterns to known locations [3].

Traditional ML approaches such as K-Nearest Neighbors (KNN) and gradient boosting methods like XGBoost have demonstrated strong performance in controlled indoor settings [3], [4]. Although these algorithms deliver competitive accuracy on structured tabular RSSI data, they treat each sample as an isolated point or fixed statistical window. This lack of temporal reasoning becomes a critical limitation in dynamic environments, where RSSI fluctuates due to body shadowing, multipath fading, and zone-boundary ambiguity [5], [6]. On the LLM side, Jin et al. [7] proposed Time-LLM, and Gruver et al. [8] demonstrated zero-shot time-series forecasting, yet neither applied LLMs to RSSI classification. Broader surveys cover both traditional and emerging indoor motion tracking methods [1], [2], [9]. Early RSSI fingerprinting work by Bahl and Padmanabhan [10] established KNN as the dominant classifier, later extended to BLE by Faragher and Harle [11]. Kalman filtering has been shown to reduce positioning error by up to 80% [12], and deep learning architectures have recently been explored [13]. However, all these approaches remain limited in modeling complex RF multipath effects [14]. None of this prior work has applied LLMs as classifiers for continuous BLE RSSI streams.

Concurrently, advances in Large Language Model (LLM) reasoning have opened a fundamentally different paradigm: when raw numerical data is presented to an LLM as text, the model can apply contextual reasoning without domain-specific training. This raises a compelling

question—*can LLM reasoning handle numeric sensor data as effectively as traditional ML for indoor motion tracking?* If so, LLMs could serve as zero-shot classifiers requiring no labeled training data, no feature engineering, and no model retraining [7], [8].

We select the motion tracking problem as a test bed specifically because BLE signal strength is inherently unstable—RSSI values fluctuate due to multipath propagation, body shadowing, and hardware variability. To represent the ML spectrum, we employ KNN as a distance-based classifier, and XGBoost as a supervised gradient-boosted tree model. On the LLM side, we evaluate Claude Opus 4.6 and Gemini 3.1 Pro as zero-shot reasoners. The main contributions of this work are:

- 1) A systematic evaluation framework comparing trained ML against zero-shot LLM classifiers for BLE RSSI zone-level indoor motion tracking.
- 2) Three scenarios testing body orientation, boundary transition, and dynamic trajectory under inherently unstable BLE signal conditions, with robustness analysis under noise injection and anchor failure revealing superior LLM resilience to hardware degradation.
- 3) Analysis of LLM reasoning traces, providing insight into *how* zero-shot reasoning succeeds on structured motion tracking data.

II. METHODOLOGY

This section describes the evaluation framework designed to compare trained ML classifiers and zero-shot LLM classifiers on the same BLE RSSI dataset. The key design principle is to format raw RSSI data so that it can be consumed by both categories of models without modification: ML models operate on engineered statistical features extracted from the RSSI time series, while LLMs receive the identical raw time series as structured text via prompts. This ensures a fair comparison where differences in performance arise solely from the models’ analytical capabilities rather than data preprocessing advantages.

A. Experimental Environment

The experiment uses a single room divided into four zones (Zone A, B, C, D), each approximately 4×4 m, separated by cloth partitions as shown in Fig. 1. Hardware components (Fig. 2) include:

- **Anchors:** Four ESP32-S3 BLE nodes (Fig. 2a), one per zone center, at 0.7 m height, communicating via MQTT over Wi-Fi.
- **Tag:** One M5StickC Plus2 (Fig. 2b) BLE beacon, attached to the researcher’s waist at 0.7 m (Fig. 2c), broadcasting at 5 Hz (200 ms interval).
- **Server:** Python-based data collection server with synchronized video recording for ground truth labeling.

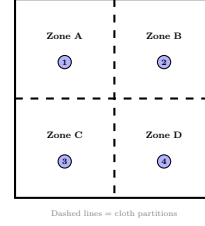


Fig. 1. Floor plan with BLE anchor positions (numbered 1–4).

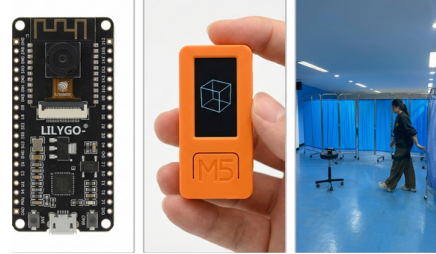


Fig. 2. Hardware: (a) ESP32-S3 anchor, (b) M5StickC Plus2 tag, (c) tag on waist.

B. Experimental Design

Case 1: Body Orientation Detection. The researcher stands stationary in Zone D, facing four cardinal directions at two distances (≤ 1 m, >1 m). Classes: Facing in (0) and Facing out (1)—body blocks the direct path, causing RSSI attenuation [6]. Conditions: 4 directions $\times$ 2 distances $\times$ 2 classes $\times$ 3 replicates = 48 episodes; 13 train / 35 test.

Case 2: Boundary Transition Detection. The researcher stands on demarcation lines between Zone D and adjacent zones. Binary task: In Zone D (0) or Out (1). Conditions: 2 boundaries $\times$ 2 classes $\times$ 12 replicates = 48 episodes; 13 train / 35 test. This task is *inherently difficult*: cloth partitions provide negligible RF attenuation at 2.4 GHz.

Case 3: Multi-Zone Trajectory. The researcher walks through all four zones in randomized sequences (e.g., D→C→B→A), pausing 5–6 s per zone. Conditions: 4 zones $\times$ 4 starting positions $\times$ 3 replicates = 48 episodes; 13 train / 35 test. Task: predict exact ordered sequence of zone IDs. *Exact Sequence accuracy* requires all four zone positions to match simultaneously; *Per-Position accuracy* measures the fraction of individual zone positions (out of 4) predicted correctly, independent of other positions in the sequence. LLMs receive the full time series; ML models use four contiguous segments.

C. Data Collection

Each case comprises 48 episodes. An *episode* is one continuous recording under fixed experimental conditions, lasting approximately 5 seconds. The dataset is split into 13 training and 35 test episodes by a single random draw without replacement; the same split is used for all models. Raw RSSI values from the four anchors form a 4-dimensional feature vector at each time step, recorded as (timestamp), (r1), (r2), (r3), (r4) where (r1)–(r4) are the RSSI readings (in dBm) from anchors 1–4 and

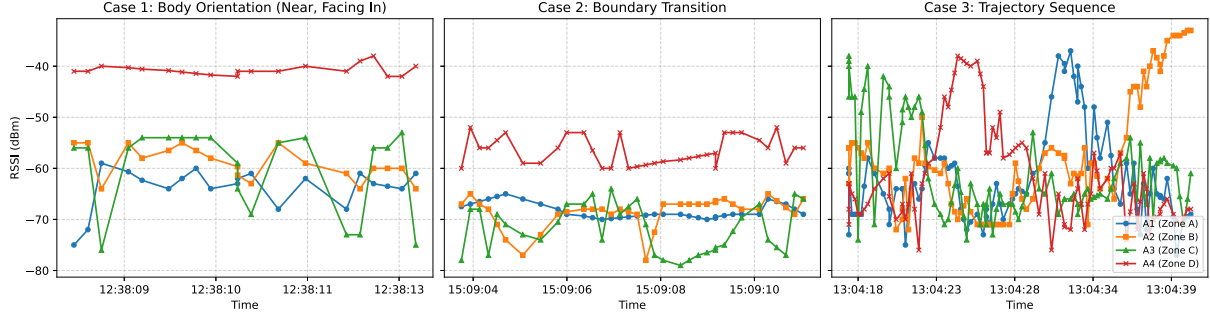


Fig. 3. Example continuous RSSI time-series for Case 1 (Left), Case 2 (Center), and Case 3 (Right).

(timestamp) is the Unix epoch in milliseconds. Missing RSSI readings were forward-filled; no smoothing or filtering was applied to preserve the raw signal dynamics.

D. Machine Learning Models for Classification

KNN ($k=5$, Euclidean distance): A distance-based classifier applied to per-sample 4D RSSI vectors. For Cases 1 and 2, the episode prediction is obtained by `round(mean(predictions))`—a soft vote over time steps. For Case 3, each episode is divided into four contiguous segments, and 8D features (mean and standard deviation per anchor per segment) are used for 4-class zone classification.

XGBoost (100 trees, max depth 6, learning rate 0.1): A gradient boosting decision tree model trained on the same features and episode-level aggregation as KNN. Hyperparameters were selected based on commonly used defaults in RSSI fingerprinting studies.

E. Large Language Models for Classification

Claude Opus 4.6 and Gemini 3.1 Pro: Both LLMs receive the raw RSSI time series from test episodes via a task-specific prompt. No training data or labels are provided—predictions are entirely *zero-shot*. Each LLM receives the identical prompt and test CSV. This design reflects typical RSSI fingerprinting pipelines where ML models operate on engineered statistical features, while LLMs process raw sequences.

To ensure reproducibility, prompt formulations for both LLMs were tightly controlled. The design enforces strict zero-shot inference: models receive the task description, class definitions, and input data format without any few-shot examples, domain-specific hints, or physical layout descriptions.

Case 1 prompt: “You are acting as an expert indoor motion tracking system. Predict labels for the provided BLE RSSI test dataset (Case 1). Each row in the attached CSV represents one episode. Based only on the RSSI time series data, classify whether the researcher was Facing in (0) or Facing out (1). Do not provide explanations. Output strictly a Markdown table with two columns: ‘id’ and ‘predicted.’”

Case 2 prompt: “You are acting as an expert indoor motion tracking system. Predict labels for the provided BLE

RSSI test dataset (Case 2). Each row in the attached CSV represents one episode. Based only on the RSSI time series data, classify the boundary transition: In zone 4 (0) or Out zone 4 (1). Do not provide explanations. Output strictly a Markdown table with two columns: ‘id’ and ‘predicted.’”

Case 3 prompt: “You are acting as an expert indoor motion tracking system. Predict the trajectory sequence for the provided BLE RSSI test dataset (Case 3). The environment is a single room divided into 4 equal-sized zones (4×4 m each, separated by cloth partitions). Based only on the continuous RSSI time series, predict the exact sequence of 4 zone IDs (1, 2, 3, or 4) visited in order. Example format: ‘4 3 2 1’. Do not provide explanations. Output strictly a Markdown table with two columns: ‘id’ and ‘predicted.’”

The input CSV files format each row as `experiment_id` followed by a semicolon-separated list of temporal readings `timestamp,r1,r2,r3,r4` (RSSI values in dBm). For the robustness evaluation (Section IV), the identical prompts were deployed with altered CSV data; the models were intentionally left uninformed regarding the injected Gaussian noise or anchor failures.

III. RESULTS AND DISCUSSION

This section evaluates classifier performance across three BLE RSSI motion tracking tasks. We compare two trained ML models (KNN, XGBoost) against two zero-shot LLM classifiers (Claude Opus 4.6, Gemini 3.1 Pro) using accuracy, precision, recall, and F1-score as metrics. Table I summarizes performance across all three cases (35 test episodes each). Tables II and III present the confusion matrices for the binary classification tasks.



Fig. 4. Ground truth video frame showing researcher position. A. Case-by-Case Analysis

Case 1 — Body Orientation. All four models exceed 88% accuracy, confirming a readily detectable RSSI

TABLE I
CLASSIFICATION PERFORMANCE (% , $n=35$)

Model	Acc.	Prec.	Rec.	F1
<i>Case 1: Body Orientation (Facing In/Out)</i>				
KNN	88.57	84.21	94.12	88.89
XGBoost	97.14	100.00	94.12	96.97
Claude Opus 4.6	97.14	100.00	94.12	96.97
Gemini 3.1 Pro	94.29	94.12	94.12	94.12
<i>Case 2: Boundary Transition (In/Out Zone D)</i>				
KNN	68.57	68.75	64.71	66.67
XGBoost	57.14	54.55	70.59	61.54
Claude Opus 4.6	62.86	64.29	52.94	58.06
Gemini 3.1 Pro	34.29	36.36	47.06	41.03
<i>Case 3: Trajectory (Exact Seq. / Per-Position)</i>				
KNN	88.57	Per-pos: 97.14		
XGBoost	48.57	Per-pos: 85.00		
Claude Opus 4.6	94.29	Per-pos: 98.57		
Gemini 3.1 Pro	97.14	Per-pos: 98.57		

TABLE II
CONFUSION MATRIX — CASE 1 ($n=35$)

	KNN		XGBoost		Claude		Gemini	
	P0	P1	P0	P1	P0	P1	P0	P1
A0	TN=15	FP=3	TN=18	FP=0	TN=18	FP=0	TN=17	FP=1
A1	FN=1	TP=16	FN=1	TP=16	FN=1	TP=16	FN=1	TP=16

TABLE III
CONFUSION MATRIX — CASE 2 ($n=35$)

	KNN		XGBoost		Claude		Gemini	
	P0	P1	P0	P1	P0	P1	P0	P1
A0	TN=13	FP=5	TN=8	FP=10	TN=13	FP=5	TN=4	FP=14
A1	FN=6	TP=11	FN=5	TP=12	FN=8	TP=9	FN=9	TP=8

signature from body shadowing. Body orientation introduces body-shadowing, which significantly attenuates RSSI signals [6]. This produces consistent patterns that both ML and LLM models can capture reliably. Both Claude and XGBoost achieve 97.14% (34/35), each with TN=18, FP=0, FN=1, TP=16. KNN produces 3 FP due to RSSI overlap at boundary distances. Gemini achieves 94.29% with a balanced error profile (FP=1, FN=1). The consistent RSSI level-shift over a 5-second window [6], [14] makes this task well-suited to both gradient boosting and zero-shot reasoning.

Case 2 — Boundary Transition. This proves the most challenging scenario (Table III). KNN leads at 68.57%, followed by Claude (62.86%), XGBoost (57.14%), and Gemini (34.29%—below chance). The difficulty is fundamentally physical: with only one anchor per zone, the system lacks spatial redundancy to resolve boundary positions [14]. Body shadowing at boundaries affects all four anchor readings simultaneously [6], creating overlapping RSSI distributions that no classifier can cleanly separate. KNN outperforms XGBoost due to its non-parametric, instance-based nature [15]: as an unsupervised-style learner that stores all training instances and performs post-hoc label assignment via distance-weighted voting [3], KNN adapts flexibly to *local* RSSI decision surfaces. This flexibility is particularly advantageous when boundary RSSI distributions are irregularly shaped and position-dependent. XGBoost, by contrast, builds fixed tree splits optimized for globally

separable features [16]; when boundary RSSI distributions overlap heavily, its axis-aligned decision boundaries cannot capture these subtle, position-specific patterns. Cover and Hart [15] demonstrated that instance-based methods achieve near-optimal performance precisely in domains with complex, non-linear decision surfaces. Zero-shot LLMs lack any access to the training distribution, placing them at a structural disadvantage on this ambiguous task.

Case 3 — Multi-Zone Trajectory. Gemini achieves 97.14% (34/35) and Claude reaches 94.29% (33/35), both substantially outperforming KNN (88.57%) and XGBoost (48.57%). The advantage lies in the LLMs’ *temporal signal reasoning*—recognizing RSSI peak transitions between anchors as zone changes. Unlike XGBoost, which builds fixed decision trees on per-segment statistics [16], and KNN, which evaluates each segment independently [15], LLMs process the *entire RSSI time series* in a single inference pass. This enables detection of outlier time steps and cross-episode pattern correction that per-sample ML classification cannot achieve. Zhou et al. [17] demonstrated that pretrained language models can capture temporal dependencies in time-series data more effectively than traditional segment-based approaches. XGBoost’s 85% per-position accuracy compounds to only $0.85^4 \approx 52\%$ exact-sequence accuracy.

B. Cross-Case Analysis

A consistent pattern emerges: *LLMs excel when signal patterns are physically distinctive or when temporal reasoning is required* (Cases 1 and 3), whereas *trained ML models prove more robust when signal boundaries are ambiguous* (Case 2). This aligns with theoretical expectations: KNN and XGBoost optimize discriminative boundaries from labeled examples [15], [16], whereas LLMs perform inductive reasoning from the full context window [17]. Syafri et al. [5] similarly observed that ML algorithm performance in BLE RSSI localization varies significantly depending on signal separability, supporting our finding that physical boundary characteristics—rather than model architecture—dominate classification difficulty. Claude demonstrates stronger performance on binary tasks, while Gemini excels at sequential reasoning. The nearly 29-percentage-point gap between Case 2 (KNN: 68.57%) and Cases 1/3 ($\geq 94\%$) underscores this observation.

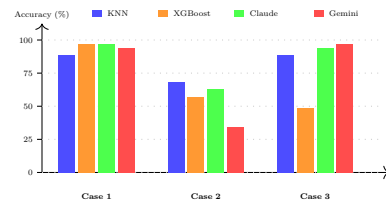


Fig. 5. Accuracy comparison across three cases.

C. LLM Reasoning Process Analysis

A distinctive advantage of LLM-based classifiers lies in their ability to reason over the entire input dataset in a single inference pass. Traditional ML models are *stateless*: KNN and XGBoost apply a fixed function to each feature vector independently [15], [16]. LLMs operate over a full context window encompassing the entire test set, enabling them to identify recurring patterns, detect outliers, and leverage cross-episode consistency. Zhou et al. [17] demonstrated that pretrained language models possess inherent temporal pattern recognition capabilities that transfer effectively to structured numerical data.

Chain-of-thought (CoT) reasoning [18] traces reveal that LLMs spontaneously execute structured analytical workflows: feature identification, pattern discovery, cluster validation, and self-correction. Wei et al. [18] showed that this emergent step-by-step reasoning significantly improves performance on complex analytical tasks. Case 3 traces show even more sophisticated behavior—the LLM evaluates multiple segmentation strategies, constructs state transition models, and proactively verifies edge cases.

The effectiveness stems from three factors: (i) BLE RSSI exhibits *physically grounded patterns*—stronger signal implies proximity [6]; (ii) the data is compact, fitting within the model’s working memory; and (iii) the task maps to *comparative reasoning* across anchors [7], [8].

IV. ROBUSTNESS UNDER SIGNAL DEGRADATION

Two offline experiments evaluate robustness: (1) Gaussian noise injection ($\sigma = 2\text{--}12$ dBm) added to each RSSI sample, and (2) simulated anchor failure by replacing 1 or 2 anchors with -100 dBm. The same prompts from Section II-E are reused without modification.

A. Noise Resilience

Table IV shows accuracy under increasing noise. ML fluctuations (e.g., KNN Case 1 rising at $\sigma=2$) are expected with stochastic noise and $n=35$. In Case 1, all models maintain $>77\%$ accuracy at $\sigma=12$ dBm, confirming that the body-shadowing signal margin exceeds typical indoor noise levels [14]. Kaemarungsi and Krishnamurthy [14] demonstrated that RSSI-based systems can tolerate moderate noise levels when underlying signal patterns are sufficiently distinctive. Claude degrades 12 pp ($97.14 \rightarrow 85.71\%$), Gemini 17 pp, while ML models show non-monotonic fluctuations due to stochastic effects with $n=35$.

In Case 3, LLMs demonstrate superior noise resilience: at $\sigma=12$, Claude (74.29%) and Gemini (82.86%) both outperform the ML *baselines* at $\sigma=0$ (KNN: 57.14%, XGBoost: 45.71%), indicating that temporal reasoning over the full time series provides inherent noise tolerance that per-sample classification cannot achieve [17]. Case 2 shows minimal additional degradation from noise across all models, confirming that boundary-task difficulty is

dominated by the physical ambiguity of cloth partitions rather than signal noise [5].

TABLE IV
ACCURACY UNDER GAUSSIAN NOISE (%)

Case	σ	KNN	XGB	Claude	Gemini
1	0	88.57	97.14	97.14	94.29
	2	91.43	97.14	88.57	91.43
	4	88.57	100.00	88.57	91.43
	6	91.43	97.14	85.71	88.57
	8	85.71	94.29	85.71	82.86
	10	94.29	94.29	88.57	85.71
	12	85.71	82.86	85.71	77.14
2	0	68.57	57.14	62.86	34.29
	2	68.57	54.29	60.00	31.43
	4	71.43	62.86	57.14	34.29
	6	60.00	62.86	54.29	28.57
	8	60.00	65.71	51.43	31.43
	10	60.00	51.43	48.57	25.71
	12	62.86	62.86	45.71	25.71
3	0	57.14	45.71	94.29	97.14
	2	60.00	34.29	91.43	97.14
	4	62.86	34.29	88.57	97.14
	6	54.29	37.14	85.71	91.43
	8	62.86	28.57	82.86	91.43
	10	54.29	40.00	80.00	85.71
	12	51.43	22.86	74.29	82.86

B. Anchor Failure

Table V shows accuracy when 1 or 2 anchors fail. ML models are deterministic: their predictions for identical input are reproducible at negligible computational cost, so ML accuracy is averaged exhaustively over all $\binom{4}{k}$ anchor failure combinations ($k=1$ or 2). LLM inference, however, requires individual API calls with 1–3 minute latency per episode set, making exhaustive evaluation over all combinations prohibitively expensive. Accordingly, LLM results report a single representative combination selected by removing the anchor(s) closest to the tag’s primary zone, representing the most informative failure scenario.

In Case 1, Claude retains 91.43% accuracy even with 2 anchors lost, while KNN drops to 56.67% (-32 pp) and XGBoost to 70.95% (-26 pp). This occurs because ML models’ fixed 4D feature space is fundamentally disrupted when dimensions are replaced with -100 dBm, whereas the LLM processes the time series as text and can adaptively ignore anomalous channels. This behavior parallels the LLM context-filtering capability described by Gruver et al. [8]—the model down-weights obviously corrupted channels without being explicitly instructed to do so. Case 3 reveals the largest ML–LLM gap under degradation: with 2 anchors lost, KNN drops to 4.29% (-53 pp), XGBoost to 9.05% (-37 pp), while Claude retains 62.86% (-31 pp) and Gemini 68.57% (-29 pp). Zhou et al. [17] attributed this resilience to pretrained language models’ ability to identify and down-weight corrupted input segments through contextual attention mechanisms.

V. CONCLUSION

This study compares trained ML classifiers (KNN, XGBoost) against zero-shot LLM classifiers (Claude Opus 4.6, Gemini 3.1 Pro) for BLE RSSI-based indoor motion tracking.

TABLE V
ACCURACY UNDER ANCHOR FAILURE (%)

Case	Cond.	KNN	XGB	Claude	Gemini
1	All 4	88.57	97.14	97.14	94.29
	1 lost	75.71	84.29	94.29	85.71
	2 lost	56.67	70.95	91.43	74.29
2	All 4	68.57	57.14	62.86	34.29
	1 lost	51.43	65.71	54.29	31.43
	2 lost	50.00	65.71	48.57	28.57
3	All 4	57.14	45.71	94.29	97.14
	1 lost	15.00	24.29	80.00	85.71
	2 lost	4.29	9.05	62.86	68.57

Key findings. LLMs excel when temporal reasoning or distinctive signal patterns are present: Gemini achieves 97.14% exact-sequence accuracy in trajectory prediction (Case 3), and Claude matches XGBoost at 97.14% for body orientation (Case 1). For the ambiguous boundary transition task (Case 2), KNN's instance-based flexibility achieves 68.57%, surpassing all other models. This demonstrates that physical signal separability—not model sophistication—determines classification accuracy.

Robustness. LLMs show marked advantage under hardware degradation. Claude retains 91.43% with two anchors lost in Case 1, versus KNN's 56.67%. In Case 3 with two anchors failed, LLMs maintain >62% while ML models drop below 10%, indicating that full time-series reasoning provides inherent fault tolerance.

Practical implications. ML models offer sub-10 ms edge inference, while LLM API calls require 1–3 minutes. This suggests a hybrid architecture: ML for real-time tracking, with LLMs for offline trajectory reconstruction or when anchor degradation reduces ML accuracy. LLM-based RSSI analysis also offers a privacy-preserving alternative to CCTV for healthcare environments.

Limitations and future work. The dataset comprises only 48 episodes per case (35 test), yielding a resolution of 2.86 pp per episode; accuracy differences below approximately ± 12 pp should therefore be interpreted with caution. All experiments are conducted in a single controlled room with cloth partitions, which limits generalizability to diverse real-world settings such as multi-room buildings with solid walls and varying materials. Additionally, the ML baselines (KNN, XGBoost) treat each segment independently; recurrent architectures such as RNN or LSTM could process the raw Case 3 time series directly without segment-based aggregation, potentially narrowing the ML–LLM accuracy gap on trajectory prediction. The current analysis is primarily empirical; a formal theoretical framework characterizing the conditions under which zero-shot LLM reasoning outperforms trained classifiers—in terms of signal separability, temporal structure, and context length—would strengthen the contributions. Future work includes validation in diverse environments with solid walls and multi-floor layouts, evaluation with larger datasets, few-shot learning for boundary transitions, temporal ML baselines (RNN, LSTM, Transformer-based models), and on-device compact language models for real-

time inference.

ACKNOWLEDGMENT

The authors thank the Thammasat School of Engineering and the CED-Square Innovation Center for providing the research facilities, equipment, and support that made this work possible.

REFERENCES

- [1] A. Yassin *et al.*, “Recent Advances in Indoor Localization: A Survey on Theoretical Approaches and Applications,” *IEEE Commun. Surv. Tuts.*, vol. 19, no. 2, pp. 1327–1346, 2017.
- [2] P. Spachos and K. N. Plataniotis, “BLE Beacons for Indoor Positioning at an Interactive IoT-Based Smart Museum,” *IEEE Syst. J.*, vol. 14, no. 3, pp. 3483–3493, 2020.
- [3] S. He and S.-H. G. Chan, “Wi-Fi Fingerprint-Based Indoor Positioning: Recent Advances and Comparisons,” *IEEE Commun. Surv. Tuts.*, vol. 18, no. 1, pp. 466–490, 2016.
- [4] Y. Li *et al.*, “Two-Step XGBoost Model for Indoor Localization Using RSSI,” *IEEE Access*, vol. 8, pp. 47528–47541, 2020.
- [5] A. F. Syafri, S. Yuliana, and R. Arifuddin, “Comparative Analysis of Machine Learning Algorithms for BLE RSSI-Based Indoor Localization,” *Proc. IEEE Int. Conf. Commun., Netw. Satell. (COMNETSAT)*, pp. 142–148, 2023.
- [6] D. Giovanelli *et al.*, “Bluetooth-Based Indoor Positioning Through Angle of Arrival Estimation: Body Shadowing Compensation and Performance Analysis,” *IEEE Trans. Instrum. Meas.*, vol. 70, pp. 1–12, 2021.
- [7] M. Jin *et al.*, “Time-LLM: Time Series Forecasting by Reprogramming Large Language Models,” *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024.
- [8] N. Gruver *et al.*, “Large Language Models Are Zero-Shot Time Series Forecasters,” *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.
- [9] F. Zafari, A. Gkelias, and K. K. Leung, “A Survey of Indoor Localization Systems and Technologies,” *IEEE Commun. Surv. Tuts.*, vol. 21, no. 3, pp. 2550–2599, 2019.
- [10] P. Bahl and V. N. Padmanabhan, “RADAR: An In-Building RF-Based User Location and Tracking System,” *Proc. IEEE INFOCOM*, pp. 775–784, 2000.
- [11] R. Faragher and R. Harle, “Location Fingerprinting with Bluetooth Low Energy Beacons,” *IEEE J. Sel. Areas Commun.*, vol. 33, no. 11, pp. 2418–2428, 2015.
- [12] K. S. Ahn and O. S. Shin, “Deep Learning-Based Indoor Localization Using Wi-Fi RSSI,” *IEEE Access*, vol. 9, pp. 141852–141865, 2021.
- [13] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd ed. Prentice Hall, 2001.
- [14] K. Kaemarungsi and P. Krishnamurthy, “Analysis of WLAN's Received Signal Strength Indication for Indoor Location Fingerprinting,” *Pervasive Mobile Comput.*, vol. 8, no. 2, pp. 292–316, 2012.
- [15] T. M. Cover and P. E. Hart, “Nearest Neighbor Pattern Classification,” *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [16] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, pp. 785–794, 2016.
- [17] T. Zhou *et al.*, “One Fits All: Power General Time Series Analysis by Pretrained LM,” *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.
- [18] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022.